Họ và tên: Nguyễn Toàn Thắng

MSSV: 25521679

Bạn làm chung Task D-T11: Đoàn Trần Việt Hà – 25520469

GIỚI THIỆU MÃ NGUỒN VÀ THƯ MỤC NỘP BÀI

Kho lưu trữ Git

Em lưu đường dẫn kho Git để thầy có thể đối chiếu mã nguồn và lịch sử cập nhật của bài thực hành. Thư mục sử dụng trong báo cáo và trong gói nộp bài là Lab03.

Đường dẫn: https://github.com/Meow-advocate/25521679_NT219.git

Trong thư mục Lab03, em chia riêng phần chương trình, header, kiểm thử, khóa thử nghiệm và tài liệu. Các Task A–E cùng được gọi từ chương trình CLI; em vẫn giữ từng file theo Task để việc đọc và kiểm tra từng nội dung thuận tiện hơn.

Cấu trúc thư mục và các file đi kèm

Đường dẫn	**Nội dung và mục đích sử dụng**
Lab03/CMakeLists.txt	Thiết lập dự án C++17, liên kết OpenSSL và khai báo các mục tiêu biên dịch, kiểm thử.
Lab03/build_windows.ps1	Biên dịch trên Windows; ưu tiên thư viện OpenSSL 4.1.0-beta1 trong môi trường thực hành.
Lab03/README.md	Hướng dẫn cài môi trường, build, danh sách lệnh CLI và cách chạy kiểm thử.
Lab03/src/main.cpp	Xử lý tham số dòng lệnh, điều phối các lệnh Task A–E.
Lab03/src/task_a.cpp	Các phép toán số nguyên lớn BIGNUM, gcd và nghịch đảo modulo.
Lab03/src/task_b.cpp	Tạo và kiểm tra khóa RSA, thông tin khóa, chuyển đổi PEM/DER.
Lab03/src/task_c.cpp	Chuyển đổi OS2IP/I2OSP, minh họa RSA học thuật và RSA-OAEP.
Lab03/src/task_d.cpp	Mã hóa/giải mã file bằng RSA-OAEP và AES-256-GCM, kiểm tra gói L2PK.
Lab03/src/task_e.cpp	RSASVE, HKDF-SHA256 và mã hóa/giải mã file bằng KEM kết hợp AES-GCM.
Lab03/include/	Các header dùng chung: ossl_raii.hpp, bn_math.hpp, rsa_params.hpp, key_io.hpp, rsa_encoding.hpp, rsa_kem.hpp và task_cli.hpp.
Lab03/tests/run_all, run_all.py, run_all.ps1	Điều khiển bộ acceptance test trên các môi trường; kiểm tra mã thoát và dữ liệu khôi phục.
Lab03/tests/tamper.py	Tạo dữ liệu kiểm thử bằng cách thay đổi byte để kiểm tra xử lý gói bị sửa.
Lab03/tests/api_checks.cpp, requirements.txt	Kiểm tra API và ghi yêu cầu thư viện Python phục vụ kiểm thử.
Lab03/tests/results/	Lưu log chạy thử, tóm tắt kết quả và thông tin môi trường.
Lab03/keys/	Chứa khóa RSA PEM/DER sinh riêng để kiểm thử; không sử dụng khóa riêng thực tế.
Lab03/samples/message.txt	Tệp dữ liệu mẫu để chạy các lệnh mã hóa/giải mã.
Lab03/report/	Chứa báo cáo PDF tóm tắt theo giới hạn trang của đề.
Lab03/docs/CHANGES.md, SHA256.json	Ghi nhận các thay đổi mã nguồn và thông tin kiểm tra tính toàn vẹn file.
Lab03/.vscode/settings.json, .gitignore	Cấu hình VS Code và danh sách file/thư mục không đưa vào Git.
Lab03/*.docx	Báo cáo Word chi tiết gồm mã nguồn đã đối chiếu và các ảnh thao tác.

Em thực hiện biên dịch trên Windows với GCC MinGW64 và OpenSSL 4.1.0-beta1 theo môi trường đã dùng trong lớp. Các file khóa trong keys/ chỉ phục vụ kiểm thử và không phải khóa cá nhân. Mã nguồn ở các mục dưới đây được đối chiếu với file trong src/; ảnh là kết quả chạy thử và được đặt sau bảng mã nguồn tương ứng.

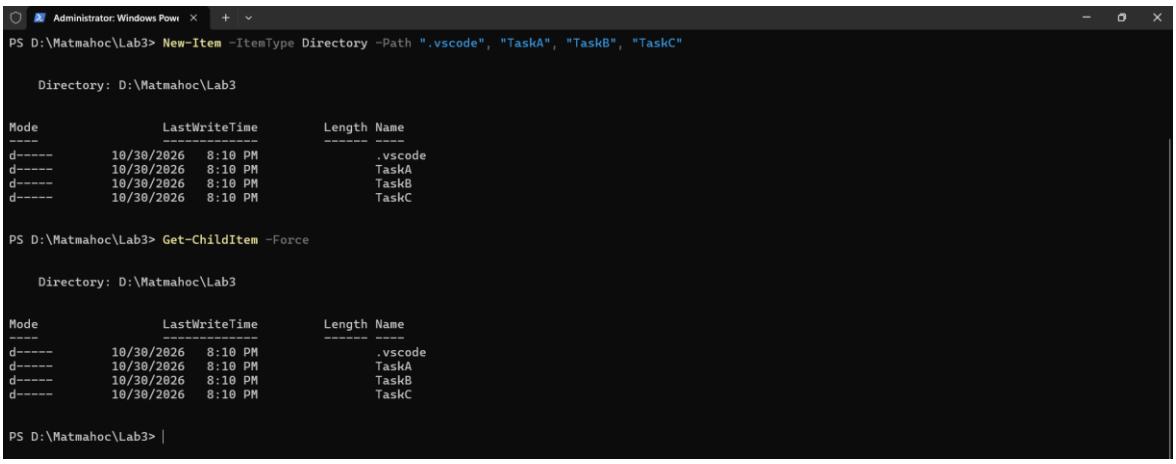
MỤC LỤC

GIỚI THIỆU MÃ NGUỒN VÀ THƯ MỤC NỘP BÀI 1
Kho lưu trữ Git 1
Cấu trúc thư mục và các file đi kèm 1
Chuẩn bị môi trường 3
Task A 5
1. Mã nguồn 5
2. Hình ảnh thực hiện và kết quả kiểm thử 10
Biên dịch và kiểm thử cơ bản 10
Các phép toán BIGNUM 11
Kiểm thử bộ dữ liệu A-T1 đến A-T8 14
Kiểm thử đầu vào không hợp lệ 15
Task B 15
1. Mã nguồn 15
2. Hình ảnh thực hiện và kết quả kiểm thử 24
Biên dịch và giao diện Task B 24
Tham số RSA và sinh khóa 25
Đọc và chuyển đổi khóa 26
Kiểm thử khóa không hợp lệ 28
Đối chiếu OpenSSL và quyền truy cập 29
Phân tích DER và thay đổi dữ liệu khóa 31
Task C 33
1. Mã nguồn 33
2. Hình ảnh thực hiện và kết quả kiểm thử 45
Biên dịch Task C 45
Biên dịch và kiểm thử bổ sung 45
Chuyển đổi OS2IP và I2OSP 48
RSA học thuật và RSA-OAEP 48
Kiểm thử khóa và dữ liệu không hợp lệ 49
Phân tích bản mã và tính ngẫu nhiên 50
Task D 55
1. Mã nguồn 55
2. Hình ảnh thực hiện và kết quả kiểm thử 64
Kiểm thử tổng hợp Task D 64
Phân tích định dạng L2PK 66
D-T1. Mã hóa và giải mã các loại tệp 68
D-T2. Ma trận khóa PEM và DER 75
D-T3. Tính ngẫu nhiên của gói dữ liệu 78
D-T4. Thay đổi ciphertext 79
D-T5 80
D-T6 81
D-T7 83
D-T8 86
D-T8 (tiếp theo) 88
D-T9 89
D-T10 91
D-T11 91
Task E 93
1. Mã nguồn 93

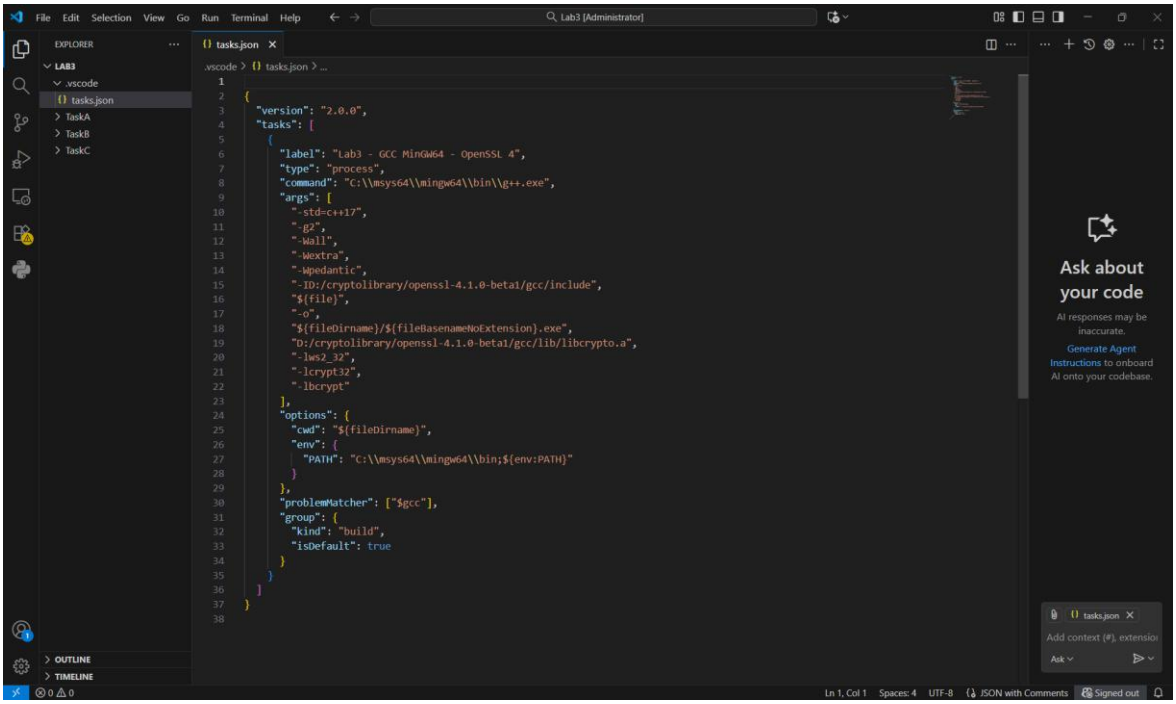
2. Hình ảnh thực hiện và kết quả kiểm thử 104
Biên dịch và kiểm thử tổng hợp Task E 104
E-T5 105
E-T6 107
E-T7 113
E-T8 115
E-T9 119
Tích hợp giao diện dòng lệnh CLI 122
1. Mã nguồn 122
2. Hình ảnh thực hiện và kết quả kiểm thử 123
Phụ lục ảnh 127

Chuẩn bị môi trường

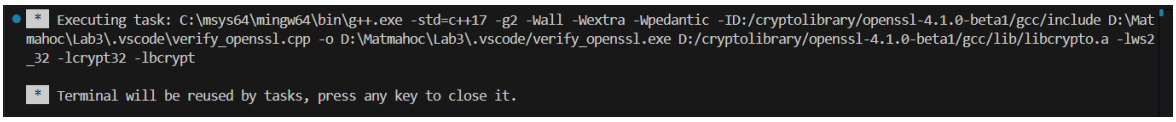
Các hình dưới đây ghi lại bước chuẩn bị thư mục, cấu hình trình biên dịch và kiểm tra OpenSSL sử dụng khi thực hiện bài lab.



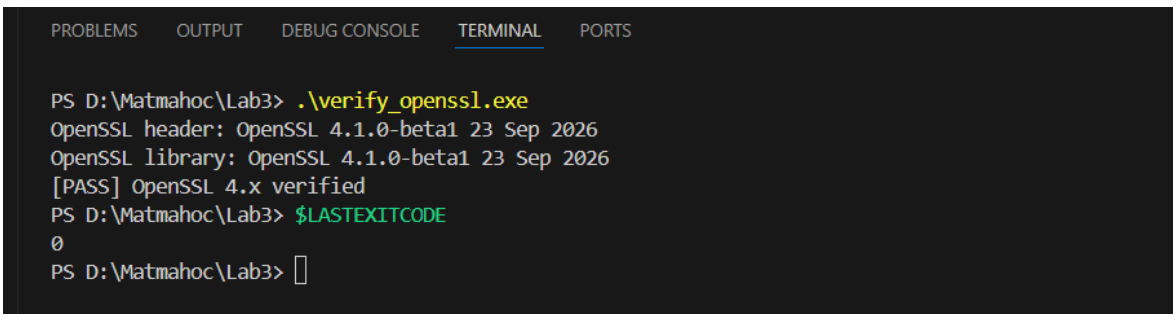
Hình 1. Tạo cấu trúc thư mục cho Lab 03.



Hình 2. Cấu hình biên dịch OpenSSL trong Visual Studio Code.



Hình 3. Biên dịch chương trình kiểm tra môi trường bằng GCC MinGW64, sử dụng header và thư viện tĩnh OpenSSL 4.1.0-beta1 trên Windows.



Hình 4. Xác minh phiên bản OpenSSL sử dụng khi biên dịch và thực thi chương trình, với kết quả thành công và mã thoát bằng 0.

Task A

1. Mã nguồn

Mã nguồn: src/task_a.cpp (phiên bản tích hợp CLI trong thư mục nộp bài).

```
#include "bn_math.hpp"
#include "task_cli.hpp"
#include "ossl_raii.hpp"
// LAB 3 - TASK A: Số nguyên lớn và các phép toán modulo với OpenSSL BIGNUM.
// Chương trình có menu thực hành và chế độ lệnh để kiểm thử tự động.
// Phần tự cài đặt bình phương - nhân và Euclid mở rộng chỉ phục vụ học tập.


#include <openssl/bn.h>
#include <openssl/crypto.h>
#include <openssl/err.h>
#include <openssl/opensslv.h>


#include <algorithm>
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>


namespace task_a {


using namespace std;


using BigNum = unique_ptr<BIGNUM, decltype(&BN_free)>;
using Context = unique_ptr<BN_CTX, decltype(&BN_CTX_free)>;


using LabError = lab::Error;


// Tạo đối tượng BIGNUM; unique_ptr tự giải phóng qua BN_free.
BigNum makeNumber() {
    BigNum result(BN_new(), BN_free);
    if (!result) throw runtime_error("Khong cap phat duoc BIGNUM");
    return result;
}


Context makeContext() {
    Context ctx(BN_CTX_new(), BN_CTX_free);
    if (!ctx) throw runtime_error("Khong cap phat duoc BN_CTX");
    return ctx;
}


void requireOK(int status, const string& operation) {
    if (status != 1) throw LabError(5, "OpenSSL that bai: " + operation + ": " + lab::drainErrors());
}


void copyNumber(BIGNUM* destination, const BIGNUM* source) {
    if (!BN_copy(destination, source))
        throw runtime_error("Sao chep BIGNUM that bai");
}


// Chỉ nhận số thập phân không âm gồm 1..10000 chữ số.
BigNum parseDecimal(const string& input) {
    if (input.empty() || input.size() > 10000 ||
        !all_of(input.begin(), input.end(), [](char c) { return c >= '0' && c <= '9'; })) {
        throw LabError(1, "Du lieu phai la so nguyen khong am, toi da 10000 chu so");
    }
    BIGNUM* raw = nullptr;
    int consumed = BN_dec2bn(&raw, input.c_str());
    if (consumed != static_cast<int>(input.size())) {
        BN_free(raw);
        throw LabError(1, "Khong chuyen duoc so thap phan sang BIGNUM");
    }
    return BigNum(raw, BN_free);
}
```

```
string displayNumber(const BIGNUM* number, bool hex = false) {
    char* raw = hex ? BN_bn2hex(number) : BN_bn2dec(number);
    if (!raw) throw runtime_error("Khong chuyen duoc BIGNUM sang chuoi");
    string answer(raw);
    OPENSSL_free(raw);
    return answer;
}

// Các phép toán tách khỏi phần nhập/xuất để dễ kiểm thử và tái sử dụng.
BigNum modOperation(const BIGNUM* a, const BIGNUM* b,
                    const BIGNUM* m, const string& operation) {
    if (BN_is_zero(m)) throw LabError(1, "Modulus m phai >= 1");
    auto result = makeNumber();
    auto ctx = makeContext();
    int ok = 0;
    if (operation == "mod-add")
        ok = BN_mod_add(result.get(), a, b, m, ctx.get());
    else if (operation == "mod-sub")
        ok = BN_mod_sub(result.get(), a, b, m, ctx.get());
    else if (operation == "mod-mul")
        ok = BN_mod_mul(result.get(), a, b, m, ctx.get());
    else if (operation == "mod-exp")
        ok = BN_mod_exp(result.get(), a, b, m, ctx.get());
    else
        throw LabError(1, "Phep toan khong hop le");
    requireOK(ok, operation);
    return result;
}

BigNum greatestCommonDivisor(const BIGNUM* a, const BIGNUM* b) {
    auto result = makeNumber();
    auto ctx = makeContext();
    requireOK(BN_gcd(result.get(), a, b, ctx.get()), "gcd");
    return result;
}

// Lũy thừa modulo theo thuật toán square-and-multiply (chỉ để đối chiếu).
// Không dùng phiên bản này với số mũ bí mật vì có thể rò rỉ thời gian chạy.
BigNum squareAndMultiply(const BIGNUM* a, const BIGNUM* exponent, const BIGNUM* m) {
    if (BN_is_zero(m)) throw LabError(1, "Modulus m phai >= 1");
    auto ctx = makeContext();
    auto result = makeNumber();
    auto base = makeNumber();
    auto remaining = makeNumber();
    auto temporary = makeNumber();
    requireOK(BN_one(result.get()), "Khoi tao gia tri 1");
    requireOK(BN_nnmod(result.get(), result.get(), m, ctx.get()), "Khoi tao modulo");
    requireOK(BN_nnmod(base.get(), a, m, ctx.get()), "Rut gon co so");
    copyNumber(remaining.get(), exponent);

    while (!BN_is_zero(remaining.get())) {
        if (BN_is_odd(remaining.get())) {
            requireOK(BN_mod_mul(temporary.get(), result.get(), base.get(), m, ctx.get()),
                      "Nhan voi co so");
            copyNumber(result.get(), temporary.get());
        }
        requireOK(BN_rshift1(remaining.get(), remaining.get()), "Dich phai so mu");
        if (!BN_is_zero(remaining.get())) {
            requireOK(BN_mod_mul(temporary.get(), base.get(), base.get(), m, ctx.get()),
                      "Binh phuong co so");
            copyNumber(base.get(), temporary.get());
        }
    }
    return result;
}

// Euclid mở rộng: tự tìm nghịch đảo để kiểm tra với BN_mod_inverse.
// Nếu gcd(a,m) != 1, trả về nullptr và ghi gcd vào gcdResult.
BigNum extendedEuclidInverse(const BIGNUM* a, const BIGNUM* m, BIGNUM* gcdResult) {
    auto ctx = makeContext();
    auto oldR = makeNumber();
    auto r = makeNumber();
    auto oldT = makeNumber();
    auto t = makeNumber();
    auto quotient = makeNumber();
    auto product = makeNumber();
```

```
    auto next = makeNumber();


    copyNumber(oldR.get(), m);
    requireOK(BN_nnmod(r.get(), a, m, ctx.get()), "Rut gon a");
    BN_zero(oldT.get());
    requireOK(BN_one(t.get()), "Khoi tao he so 1");


    while (!BN_is_zero(r.get())) {
        requireOK(BN_div(quotient.get(), nullptr, oldR.get(), r.get(), ctx.get()), "Euclid chia");
        requireOK(BN_mul(product.get(), quotient.get(), r.get(), ctx.get()), "Euclid nhan r");
        requireOK(BN_sub(next.get(), oldR.get(), product.get()), "Euclid tru r");
        copyNumber(oldR.get(), r.get());
        copyNumber(r.get(), next.get());


        requireOK(BN_mul(product.get(), quotient.get(), t.get(), ctx.get()), "Euclid nhan t");
        requireOK(BN_sub(next.get(), oldT.get(), product.get()), "Euclid tru t");
        copyNumber(oldT.get(), t.get());
        copyNumber(t.get(), next.get());
    }
    copyNumber(gcdResult, oldR.get());
    if (!BN_is_one(oldR.get())) return BigNum(nullptr, BN_free);
    auto inverse = makeNumber();
    requireOK(BN_nnmod(inverse.get(), oldT.get(), m, ctx.get()), "Rut gon nghich dao");
    return inverse;
}


BigNum modularInverse(const BIGNUM* a, const BIGNUM* m) {
    if (BN_cmp(m, BN_value_one()) <= 0)
        throw LabError(1, "Modulus cho nghich dao phai >= 2");
    auto g = greatestCommonDivisor(a, m);
    if (!BN_is_one(g.get())) {
        throw LabError(4, "no inverse: gcd(a,m) = " + displayNumber(g.get()));
    }
    auto ctx = makeContext();
    BigNum inverse(BN_mod_inverse(nullptr, a, m, ctx.get()), BN_free);
    if (!inverse) throw runtime_error("OpenSSL khong tinh duoc nghich dao modulo");
    return inverse;
}


// Chế độ lệnh độc lập: task_a.exe mod-exp 7 13 33 [--hex].
// Mã thoát: 0 thành công, 1 đầu vào sai, 4 không tồn tại nghịch đảo.
int runCommandLine(int argc, char* argv[]) {
    vector<string> args;
    bool hex = false;
    for (int i = 1; i < argc; ++i) {
        if (string(argv[i]) == "--hex") {
            if (hex) throw LabError(1, "--hex chi duoc xuat hien mot lan");
            hex = true;
        } else args.emplace_back(argv[i]);
    }
    if (args.empty()) throw LabError(1, "Thieu ten lenh");
    const string& operation = args[0];
    if (operation == "gcd" && args.size() == 3) {
        auto a = parseDecimal(args[1]);
        auto b = parseDecimal(args[2]);
        cout << displayNumber(greatestCommonDivisor(a.get(), b.get()).get(), hex) << '\n';
    } else if (operation == "mod-inverse" && args.size() == 3) {
        auto a = parseDecimal(args[1]);
        auto m = parseDecimal(args[2]);
        cout << displayNumber(modularInverse(a.get(), m.get()).get(), hex) << '\n';
    } else if ((operation == "mod-add" || operation == "mod-sub" ||
                operation == "mod-mul" || operation == "mod-exp") && args.size() == 4) {
        auto a = parseDecimal(args[1]);
        auto b = parseDecimal(args[2]);
        auto m = parseDecimal(args[3]);
        cout << displayNumber(modOperation(a.get(), b.get(), m.get(), operation).get(), hex) << '\n';
    } else {
        throw LabError(1, "Lenh sai. Dung: mod-add/mod-sub/mod-mul/mod-exp a b m; gcd a b; mod-inverse a m [--
hex]");
    }
    return 0;
}


string ask(const string& prompt) {
    cout << prompt << flush;
    string input;
    if (!getline(cin, input)) throw LabError(1, "Da ket thuc du lieu nhap");
```

```
    return input;
}

BigNum askNumber(const string& prompt) {
    return parseDecimal(ask(prompt));
}

void printMenu(const BIGNUM* modulus) {
    cout << "\n===== OPENSSL BIGNUM LAB - TASK A =====\n"
         << "Current modulus m: " << displayNumber(modulus) << '\n'
         << "1. Modular addition\n"
         << "2. Modular subtraction\n"
         << "3. Modular multiplication\n"
         << "4. Greatest common divisor\n"
         << "5. Modular exponentiation\n"
         << "6. Modular inverse\n"
         << "7. Generate a large prime for RSA (demo only)\n"
         << "0. Exit\n";
}

int runMenu() {
    cout << "OpenSSL header : " << OPENSSL_VERSION_TEXT << '\n';
    cout << "OpenSSL library: " << OpenSSL_version(OPENSSL_VERSION) << '\n';
    cout << "Nhap m mot lan; gia tri m duoc su dung lai trong cac phep modulo.\n";
    auto modulus = askNumber("Enter modulus m = ");
    if (BN_is_zero(modulus.get())) throw LabError(1, "Modulus m phai >= 1");


    while (true) {
        printMenu(modulus.get());
        string choice = ask("Choose: ");
        if (choice == "0") return 0;
        try {
            if (choice == "1" || choice == "2" || choice == "3" || choice == "5") {
                auto a = askNumber("a = ");
                auto b = askNumber("b = ");
                string operation = choice == "1" ? "mod-add" :
                                   choice == "2" ? "mod-sub" :
                                   choice == "3" ? "mod-mul" : "mod-exp";
                auto result = modOperation(a.get(), b.get(), modulus.get(), operation);
                cout << "Result: " << displayNumber(result.get()) << '\n';
                if (choice == "5") {
                    auto ownResult = squareAndMultiply(a.get(), b.get(), modulus.get());
                    if (BN_cmp(ownResult.get(), result.get()) != 0)
                        throw runtime_error("Square-and-multiply khong khop OpenSSL");
                    cout << "[PASS] Square-and-multiply khop BN_mod_exp\n";
                }
            } else if (choice == "4") {
                auto a = askNumber("a = ");
                auto b = askNumber("b = ");
                auto result = greatestCommonDivisor(a.get(), b.get());
                cout << "Result: " << displayNumber(result.get()) << '\n';
            } else if (choice == "6") {
                auto a = askNumber("a = ");
                if (BN_cmp(modulus.get(), BN_value_one()) <= 0)
                    throw LabError(1, "Modulus cho nghich dao phai >= 2");
                auto gcdResult = makeNumber();
                auto ownInverse = extendedEuclidInverse(a.get(), modulus.get(), gcdResult.get());
                if (!ownInverse) {
                    cout << "no inverse: gcd(a,m) = " << displayNumber(gcdResult.get()) << '\n';
                } else {
                    auto opensslInverse = modularInverse(a.get(), modulus.get());
                    if (BN_cmp(ownInverse.get(), opensslInverse.get()) != 0)
                        throw runtime_error("Euclid mo rong khong khop OpenSSL");
                    cout << "Result: " << displayNumber(opensslInverse.get()) << '\n';
                    cout << "[PASS] Extended Euclid khop BN_mod_inverse\n";
                }
            } else if (choice == "7") {
                string input = ask("Enter prime size in bits (16..8192): ");
                if (input.empty() || input.size() > 4 ||
                    !all_of(input.begin(), input.end(), [](char c) { return c >= '0' && c <= '9'; }))
                    throw LabError(1, "So bit phai trong khoang 16..8192");
                int bits = stoi(input);
                if (bits < 16 || bits > 8192)
                    throw LabError(1, "So bit phai trong khoang 16..8192");
                auto prime = makeNumber();
                auto ctx = makeContext();
                cout << "Dang sinh so nguyen to xac suat " << bits << " bit...\n";
                requireOK(BN_generate_prime_ex2(prime.get(), bits, 0, nullptr,
```

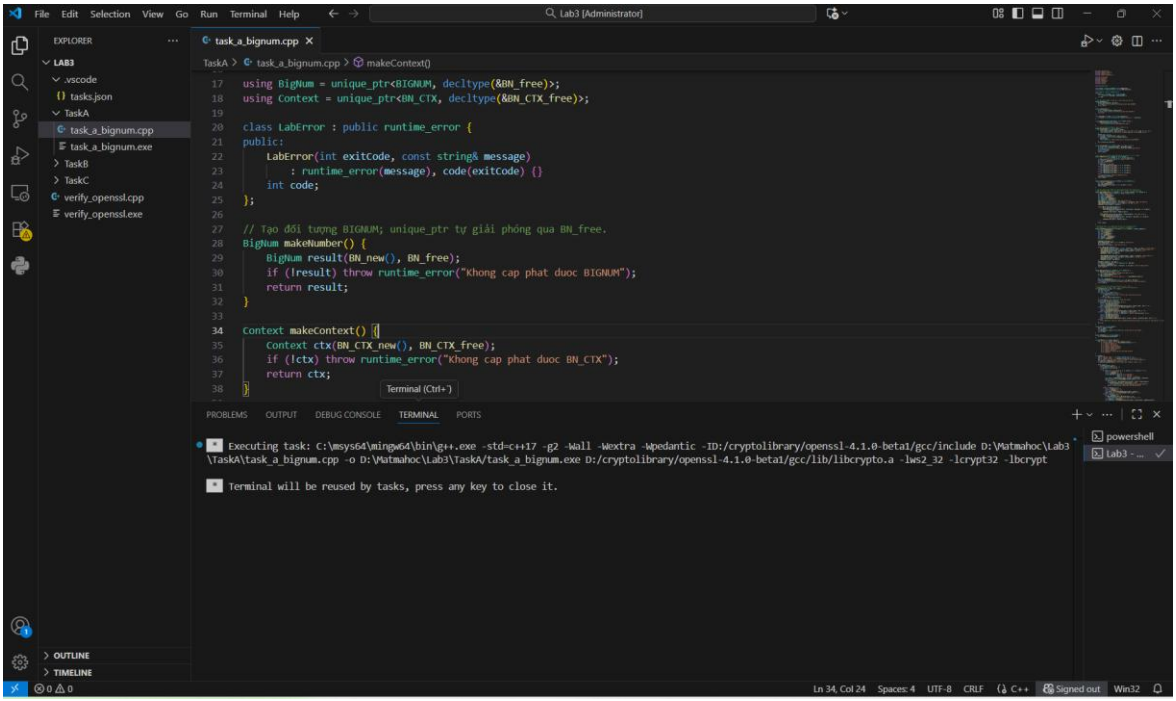
```
                                                nullptr, nullptr, ctx.get()), "Sinh so nguyen to");
                cout << "Prime generated successfully.\n";
                cout << "Actual bit length: " << BN_num_bits(prime.get()) << '\n';
                cout << "p = " << displayNumber(prime.get()) << '\n';
                cout << "Chi minh hoa: khong su dung p nay lam khoa RSA thuc te.\n";
            } else {
                cout << "Invalid choice.\n";
            }
        } catch (const exception& error) {
            cerr << "Error: " << error.what() << '\n';
            ERR_clear_error();
        }
    }
}

int entry(int argc, char* argv[]) {
    try {
        if (argc > 1) return runCommandLine(argc, argv);
        return runMenu();
    } catch (const LabError& error) {
        cerr << "Error: " << error.what() << '\n';
        return error.code;
    } catch (const exception& error) {
        cerr << "Error: " << error.what() << '\n';
        return 5;
    }
}

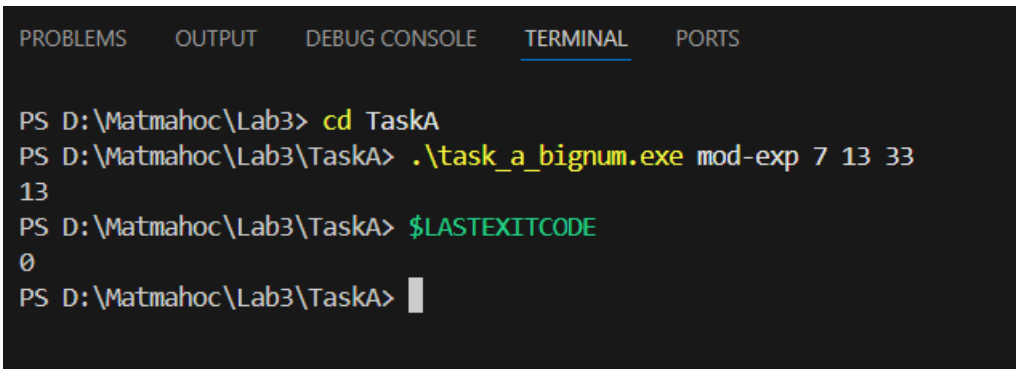
} // namespace task_a
```

2. Hình ảnh thực hiện và kết quả kiểm thử

Biên dịch và kiểm thử cơ bản



Hình 5. Biên dịch chương trình Task A bằng GCC MinGW64 và thư viện tĩnh OpenSSL 4.1.0-beta1.



Hình 6. Kiểm thử phép lũy thừa modulo với OpenSSL BIGNUM và xác nhận chương trình kết thúc thành công.

Các phép toán BIGNUM

```
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe
OpenSSL header : OpenSSL 4.1.0-beta1 23 Sep 2026
OpenSSL library: OpenSSL 4.1.0-beta1 23 Sep 2026
Nhap m mot lan; gia tri m duoc su dung lai trong cac phep modulo.
Enter modulus m = 33

===== OPENSSL BIGNUM LAB - TASK A =====
Current modulus m: 33
1. Modular addition
2. Modular subtraction
3. Modular multiplication
4. Greatest common divisor
5. Modular exponentiation
6. Modular inverse
7. Generate a large prime for RSA (demo only)
0. Exit
Choose: 1
a = 7
b = 13
Result: 20

===== OPENSSL BIGNUM LAB - TASK A =====
Current modulus m: 33
1. Modular addition
2. Modular subtraction
3. Modular multiplication
4. Greatest common divisor
5. Modular exponentiation
6. Modular inverse
7. Generate a large prime for RSA (demo only)
0. Exit
Choose: 2
a = 7
b = 13
Result: 27
```

Hình 7. Kiểm thử phép cộng và phép trừ modulo.

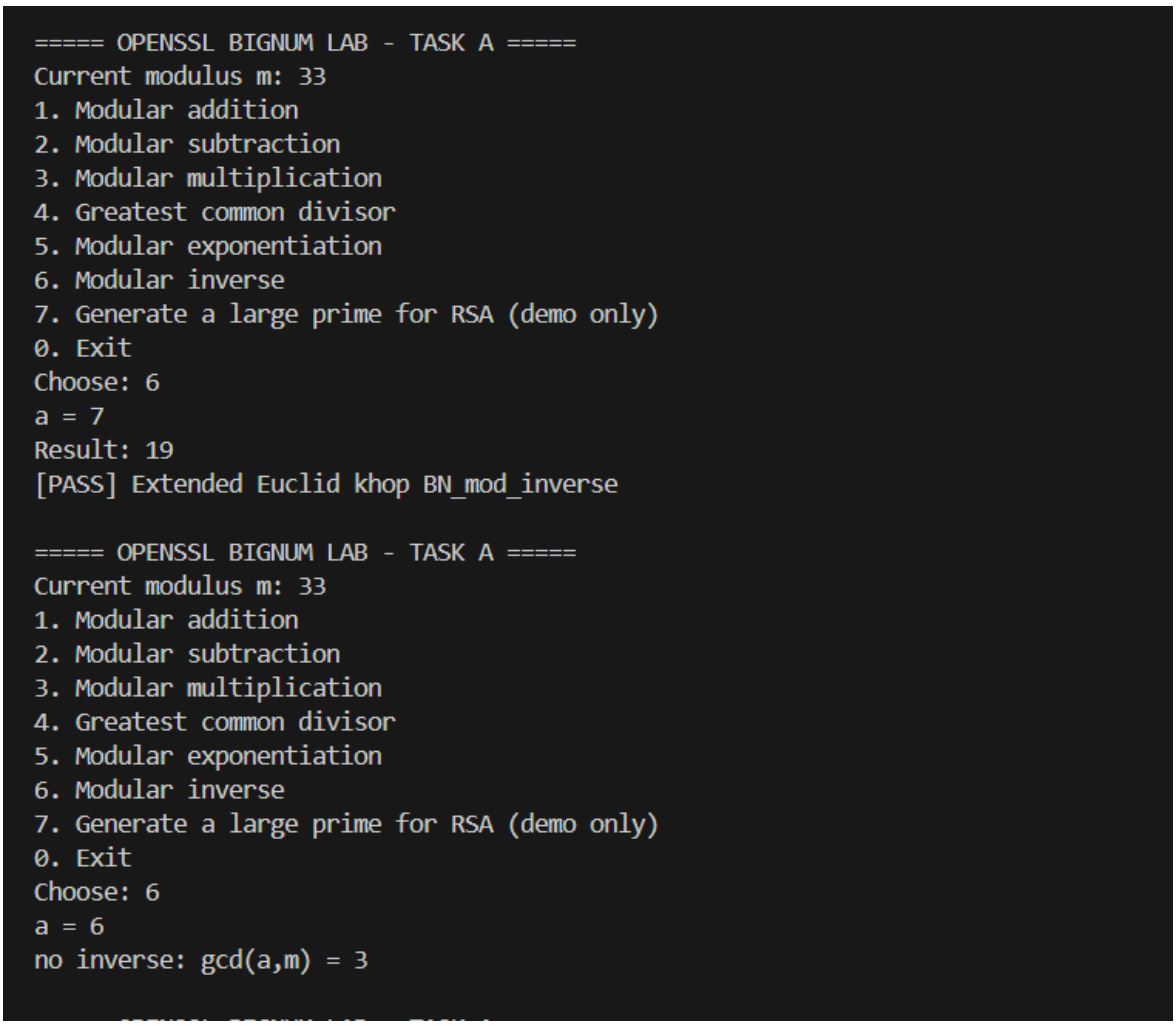
```
===== OPENSSL BIGNUM LAB - TASK A =====
Current modulus m: 33
1. Modular addition
2. Modular subtraction
3. Modular multiplication
4. Greatest common divisor
5. Modular exponentiation
6. Modular inverse
7. Generate a large prime for RSA (demo only)
0. Exit
Choose: 3
a = 7
b = 13
Result: 25

===== OPENSSL BIGNUM LAB - TASK A =====
Current modulus m: 33
1. Modular addition
2. Modular subtraction
3. Modular multiplication
4. Greatest common divisor
5. Modular exponentiation
6. Modular inverse
7. Generate a large prime for RSA (demo only)
0. Exit
Choose: 4
a = 252
b = 105
Result: 21
```

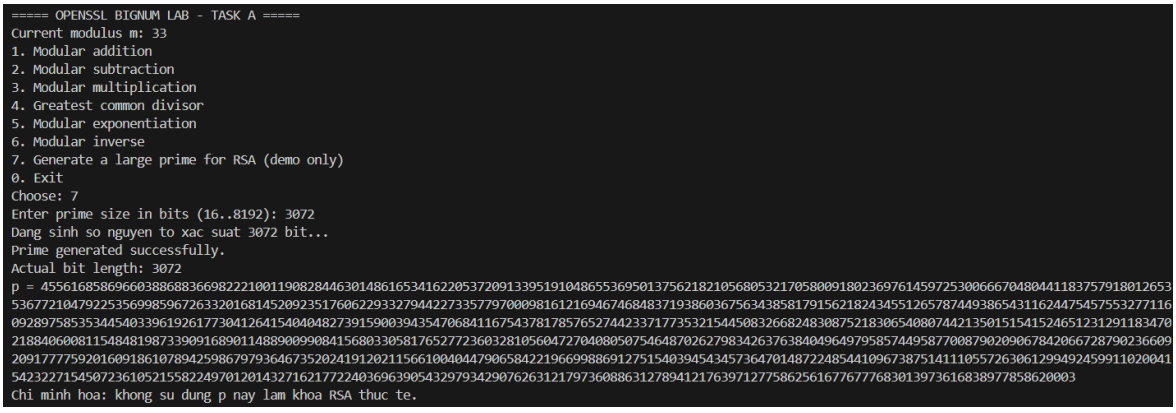
Hình 8. Kiểm thử phép nhân modulo và ước chung lớn nhất.

```
===== OPENSSL BIGNUM LAB - TASK A =====
Current modulus m: 33
1. Modular addition
2. Modular subtraction
3. Modular multiplication
4. Greatest common divisor
5. Modular exponentiation
6. Modular inverse
7. Generate a large prime for RSA (demo only)
0. Exit
Choose: 5
a = 7
b = 13
Result: 13
[PASS] Square-and-multiply khop BN_mod_exp
```

Hình 9. Kiểm thử lũy thừa modulo bằng thuật toán bình phương và nhân.

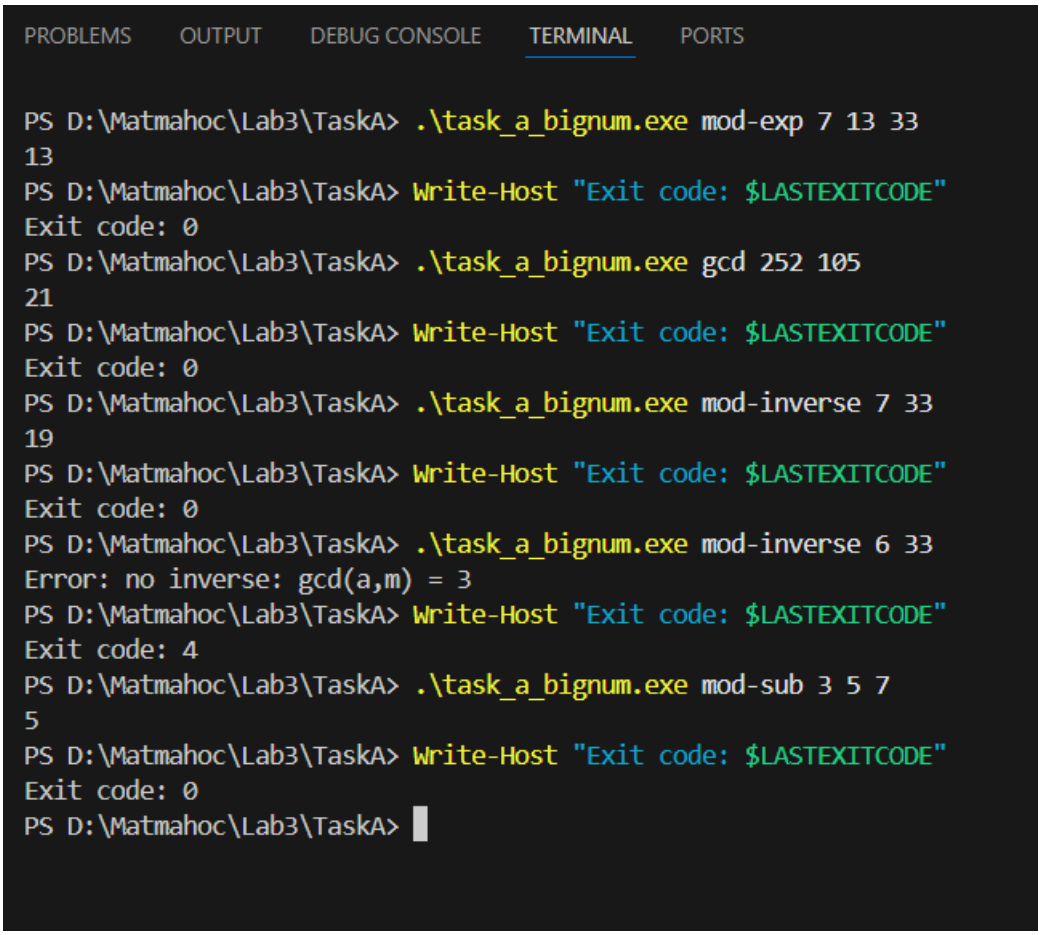


Hình 10. Kiểm thử nghịch đảo modulo và trường hợp không tồn tại nghịch đảo.

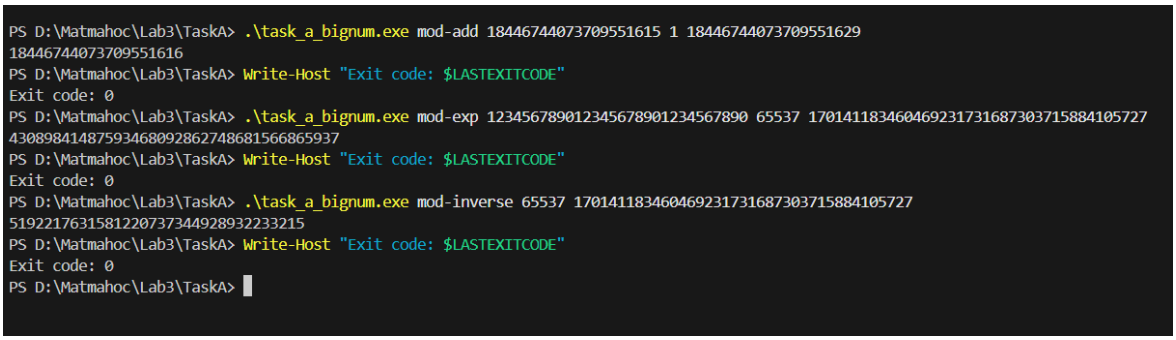


Hình 11. Sinh số nguyên tố xác suất có độ dài 3072 bit bằng OpenSSL BIGNUM để minh họa bước chuẩn bị tham số RSA.

Kiểm thử bộ dữ liệu A-T1 đến A-T8



Hình 12. Kiểm thử các phép toán BIGNUM theo bộ A-T1 đến A-T5 và xác minh mã thoát của chương trình.



Hình 13. Kiểm thử các phép toán trên số nguyên lớn vượt phạm vi 64 bit bằng OpenSSL BIGNUM, với các kết quả A-T6 đến A-T8 và mã thoát thành công.

Kiểm thử đầu vào không hợp lệ

```
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe mod-add 1 2 0
Error: Modulus m phai >= 1
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe mod-exp -3 2 5
Error: Du lieu phai la so nguyen khong am, toi da 10000 chu so
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe gcd 12a 4
Error: Du lieu phai la so nguyen khong am, toi da 10000 chu so
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe gcd "" 4
Error: Lenh sai. Dung: mod-add/mod-sub/mod-mul/mod-exp a b m; gcd a b; mod-inverse a m [--hex]
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskA>
```

Hình 14. Kiểm thử khả năng phát hiện dữ liệu không hợp lệ và trả về mã lỗi theo yêu cầu của Task A.

```
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe mod-add 200 200 257 --hex
8F
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe gcd 0 0
0
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe mod-add 40 13 33
20
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskA> $tooLong = '9' * 10001
PS D:\Matmahoc\Lab3\TaskA> .\task_a_bignum.exe gcd $tooLong 4
Error: Du lieu phai la so nguyen khong am, toi da 10000 chu so
PS D:\Matmahoc\Lab3\TaskA> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskA>
```

Hình 15. Kiểm thử dữ liệu hệ thập lục phân, số âm và giới hạn độ dài đầu vào.

Task B

1. Mã nguồn

Mã nguồn: src/task_b.cpp (phiên bản tích hợp CLI trong thư mục nộp bài).

```
#include "rsa_params.hpp"
#include <string_view>
#include "key_io.hpp"
#include "task_cli.hpp"
#include "ossl_raii.hpp"
// Lab 03 - Task B: Tham số RSA, tạo khóa, PEM/DER, kiểm tra và chuyển đổi khóa.
// Các khóa riêng không mã hóa chỉ phục vụ thực hành; không dùng cho hệ thống thật.


#include <openssl/bn.h>
#include <openssl/core_names.h>
```

```
#include <openssl/crypto.h>
#include <openssl/evp.h>
#include <openssl/pem.h>
#include <openssl/rand.h>
#include <openssl/rsa.h>
#include <openssl/sha.h>
#include <openssl/x509.h>
#include <openssl/opensslv.h>


#include <algorithm>
#include <cctype>
#include <climits>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>


#ifndef _WIN32
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#endif


namespace task_b {


using namespace std;
namespace fs = std::filesystem;


using Bn = unique_ptr<BIGNUM, decltype(&BN_clear_free)>;
using BnContext = unique_ptr<BN_CTX, decltype(&BN_CTX_free)>;
using Key = unique_ptr<EVP_PKEY, decltype(&EVP_PKEY_free)>;
using KeyContext = unique_ptr<EVP_PKEY_CTX, decltype(&EVP_PKEY_CTX_free)>;
using Bio = unique_ptr<BIO, decltype(&BIO_free)>;
using Pkcs8 = unique_ptr<PKCS8_PRIV_KEY_INFO, decltype(&PKCS8_PRIV_KEY_INFO_free)>;
using Bytes = vector<unsigned char>;


using LabError = lab::Error;


void ensure(bool success, const string& message, int code = 1) {
    if (!success) { const std::string queue=lab::drainErrors(); throw LabError(code, message + (queue.empty()
|| code==4 ? "" : ": " + queue)); }
}


void ensureCrypto(bool success,const string& message,int code=5) { ensure(success,message,code); }


Bn newBn() {
    Bn x(BN_new(), BN_clear_free);
    ensureCrypto(bool(x), "Khong tao duoc BIGNUM");
    return x;
}


BnContext newCtx() {
    BnContext ctx(BN_CTX_new(), BN_CTX_free);
    ensureCrypto(bool(ctx), "Khong tao duoc BN_CTX");
    return ctx;
}


// Chuyển BIGNUM sang chuỗi và giải phóng vùng nhớ do OpenSSL cấp.
string bnToText(const BIGNUM* value, bool hex = false) {
    char* raw = hex ? BN_bn2hex(value) : BN_bn2dec(value);
    ensureCrypto(raw != nullptr, "Khong chuyen duoc BIGNUM sang chuoi");
    string text(raw);
    OPENSSL_free(raw);
    return text;
}


int parseInt(const string& value, int minimum, int maximum, const string& name) {
    ensure(!value.empty() && all_of(value.begin(), value.end(), [](unsigned char c) {
        return c >= '0' && c <= '9';
```

```
    }), "Gia tri " + name + " khong hop le");
    long long n = 0;
    for (char c : value) {
        n = n * 10 + (c - '0');
        ensure(n <= maximum, "Gia tri " + name + " vuot gioi han");
    }
    ensure(n >= minimum, "Gia tri " + name + " qua nho");
    return static_cast<int>(n);
}


void requireRsa(EVP_PKEY* key, int minBits = 2048) {
    ensure(key && EVP_PKEY_is_a(key, "RSA") == 1,
           "Chi chap nhan khoa RSA", 3);
    ensure(EVP_PKEY_get_bits(key) >= minBits,
           "Khoa RSA khong du do dai toi thieu " + to_string(minBits) + " bit", 3);
}


Key generateKey(int bits) {
    ensure(bits == 2048 || bits == 3072 || bits == 4096,
           "Chi chap nhan 2048, 3072 hoac 4096 bit");
    KeyContext context(EVP_PKEY_CTX_new_from_name(nullptr, "RSA", nullptr), EVP_PKEY_CTX_free);
    ensureCrypto(bool(context), "Khong tao duoc context sinh khoa RSA");
    ensureCrypto(EVP_PKEY_keygen_init(context.get()) > 0, "Khong khoi tao duoc RSA keygen");
    ensureCrypto(EVP_PKEY_CTX_set_rsa_keygen_bits(context.get(), bits) > 0,
           "Khong dat duoc kich thuoc khoa RSA");
    EVP_PKEY* raw = nullptr;
    int ok = EVP_PKEY_generate(context.get(), &raw);
    Key key(raw, EVP_PKEY_free);
    ensureCrypto(ok > 0 && bool(key), "Sinh khoa RSA that bai");
    requireRsa(key.get());
    return key;
}


// Đọc nhị phân vào bộ nhớ: giới hạn kích thước nhằm tránh cấp phát không kiểm soát.
Bytes readBytes(const fs::path& filename) {
    ifstream stream(filename, ios::binary | ios::ate);
    ensure(bool(stream), "Khong mo duoc file: " + filename.string(), 2);
    streampos size = stream.tellg();
    ensure(size > 0 && size <= streampos(1024 * 1024), "File rong hoac qua lon", 3);
    Bytes data(static_cast<size_t>(size));
    stream.seekg(0);
    stream.read(reinterpret_cast<char*>(data.data()), static_cast<streamsize>(data.size()));
    ensure(bool(stream), "Khong doc het file: " + filename.string(), 2);
    return data;
}


void writeBytes(const fs::path& filename, const Bytes& data, bool overwrite,
                bool privatePart = false) {
    ensure(overwrite || !fs::exists(filename), "File da ton tai: " + filename.string(), 2);
#ifndef _WIN32
    // Trên POSIX, khóa riêng được tạo với quyền 0600 ngay từ lúc mở file.
    int flags = O_WRONLY | O_CREAT | (overwrite ? O_TRUNC : O_EXCL);
    int fd = ::open(filename.c_str(), flags, privatePart ? 0600 : 0644);
    ensure(fd >= 0, "Khong tao duoc file: " + filename.string(), 2);
    if (privatePart && ::fchmod(fd, 0600) != 0) {
        ::close(fd);
        throw LabError(2, "Khong cai duoc quyen 0600 cho file khoa rieng");
    }
    size_t written = 0;
    while (written < data.size()) {
        ssize_t count = ::write(fd, data.data() + written, data.size() - written);
        if (count <= 0) { ::close(fd); throw LabError(2, "Loi ghi file: " + filename.string()); }
        written += static_cast<size_t>(count);
    }
    ensure(::close(fd) == 0, "Khong dong duoc file: " + filename.string(), 2);
#else
    // Windows: quyền do NTFS ACL quyết định; không có chmod 0600 tương đương trực tiếp.
    ofstream stream(filename, ios::binary | ios::trunc);
    ensure(bool(stream), "Khong tao duoc file: " + filename.string(), 2);
    stream.write(reinterpret_cast<const char*>(data.data()), static_cast<streamsize>(data.size()));
    stream.flush();
    ensure(bool(stream), "Khong ghi duoc file: " + filename.string(), 2);
    (void)privatePart;
#endif
}


// Mã hóa SPKI DER hoặc PKCS#8 DER, tuyệt đối không dùng i2d_PrivateKey cho RSA.
Bytes encodeDer(EVP_PKEY* key, bool privatePart) {
```

```
    if (!privatePart) {
        int length = i2d_PUBKEY(key, nullptr);
        ensureCrypto(length > 0, "Khong ma hoa duoc SPKI DER");
        Bytes result(static_cast<size_t>(length));
        unsigned char* target = result.data();
        ensureCrypto(i2d_PUBKEY(key, &target) == length, "Loi ghi SPKI DER");
        return result;
    }
    Pkcs8 structure(EVP_PKEY2PKCS8(key), PKCS8_PRIV_KEY_INFO_free);
    ensureCrypto(bool(structure), "Khong tao duoc PKCS#8");
    int length = i2d_PKCS8_PRIV_KEY_INFO(structure.get(), nullptr);
    ensureCrypto(length > 0, "Khong ma hoa duoc PKCS#8 DER");
    Bytes result(static_cast<size_t>(length));
    unsigned char* target = result.data();
    ensureCrypto(i2d_PKCS8_PRIV_KEY_INFO(structure.get(), &target) == length,
           "Loi ghi PKCS#8 DER");
    return result;
}


Bytes encodePem(EVP_PKEY* key, bool privatePart) {
    Bio memory(BIO_new(BIO_s_mem()), BIO_free);
    ensureCrypto(bool(memory), "Khong tao duoc BIO trong bo nho");
    int ok = privatePart
        ? PEM_write_bio_PKCS8PrivateKey(memory.get(), key, nullptr, nullptr, 0, nullptr, nullptr)
        : PEM_write_bio_PUBKEY(memory.get(), key);
    ensureCrypto(ok == 1, "Khong ma hoa duoc khoa PEM");
    char* ptr = nullptr;
    long length = BIO_get_mem_data(memory.get(), &ptr);
    ensureCrypto(length > 0 && ptr != nullptr, "Khong doc duoc noi dung PEM");
    return Bytes(reinterpret_cast<unsigned char*>(ptr),
                 reinterpret_cast<unsigned char*>(ptr) + length);
}


bool startsPem(const Bytes& bytes) {
    const string prefix = "-----BEGIN ";
    return bytes.size() >= prefix.size() && equal(prefix.begin(), prefix.end(), bytes.begin());
}


// Chỉ chấp nhận đúng nhãn SPKI/PKCS#8; không nhận khóa PKCS#1 trá hình.
void validatePemEnvelope(const Bytes& bytes, bool privatePart) {
    std::string_view text(reinterpret_cast<const char*>(bytes.data()),bytes.size());
    string start = privatePart ? "-----BEGIN PRIVATE KEY-----" : "-----BEGIN PUBLIC KEY-----";
    string end = privatePart ? "-----END PRIVATE KEY-----" : "-----END PUBLIC KEY-----";
    ensure(text.rfind(start, 0) == 0, "Nhãn PEM khong dung SPKI/PKCS#8", 3);
    size_t headerEnd = start.size();
    ensure(headerEnd < text.size() && (text[headerEnd] == '\r' || text[headerEnd] == '\n'),
           "Dinh dang dau PEM khong hop le", 3);
    size_t endPos = text.find(end, headerEnd);
    ensure(endPos != string::npos && endPos > headerEnd && text[endPos - 1] == '\n',
           "Khong tim thay dong ket thuc PEM", 3);
    size_t after = endPos + end.size();
    ensure(after == text.size() || text[after] == '\n' || text[after] == '\r',
           "Du lieu du sau nhan PEM", 3);
    for (size_t i = after; i < text.size(); ++i) {
        ensure(text[i] == ' ' || text[i] == '\t' || text[i] == '\r' || text[i] == '\n',
               "Co du lieu sau khoa PEM", 3);
    }
}


// Trả về khóa đúng loại; DER phải được tiêu thụ hết và tái mã hóa y nguyên.
Key loadKey(const fs::path& filename, bool privatePart, const string& forcedFormat) {
    Bytes data = readBytes(filename);
    struct Wipe { Bytes& b; bool yes; ~Wipe(){ if(yes && !b.empty()) OPENSSL_cleanse(b.data(),b.size()); } }
wipe{data,privatePart};
    bool pem = forcedFormat.empty() ? startsPem(data) : forcedFormat == "PEM";
    ensure(forcedFormat.empty() || forcedFormat == "PEM" || forcedFormat == "DER",
           "--inform phai la PEM hoac DER");
    EVP_PKEY* raw = nullptr;
    if (pem) {
        validatePemEnvelope(data, privatePart);
        Bio memory(BIO_new_mem_buf(data.data(), static_cast<int>(data.size())), BIO_free);
        ensureCrypto(bool(memory), "Khong tao duoc BIO");
        raw = privatePart
            ? PEM_read_bio_PrivateKey(memory.get(), nullptr, nullptr, nullptr)
            : PEM_read_bio_PUBKEY(memory.get(), nullptr, nullptr, nullptr);
        ensure(raw != nullptr, "Khong giai ma duoc PEM", 3);
    } else {
        const unsigned char* cursor = data.data();
```

```
        if (privatePart) {
            Pkcs8 structure(d2i_PKCS8_PRIV_KEY_INFO(nullptr, &cursor,
                                                    static_cast<long>(data.size())),
                            PKCS8_PRIV_KEY_INFO_free);
            ensure(bool(structure), "DER phai la khoa rieng PKCS#8", 3);
            ensure(cursor == data.data() + data.size(), "DER co byte du", 3);
            raw = EVP_PKCS82PKEY(structure.get());
        } else {
            raw = d2i_PUBKEY(nullptr, &cursor, static_cast<long>(data.size()));
            ensure(raw != nullptr, "DER phai la khoa cong khai SPKI", 3);
            if (cursor != data.data() + data.size()) {
                EVP_PKEY_free(raw);
                throw LabError(3, "DER co byte du");
            }
        }
        ensure(raw != nullptr, "Khong giai ma duoc khoa DER", 3);
    }
    Key key(raw, EVP_PKEY_free);
    requireRsa(key.get());
    if (!pem) {
        Bytes canonical = encodeDer(key.get(), privatePart);
        bool match = canonical == data;
        if (privatePart) OPENSSL_cleanse(canonical.data(), canonical.size());
        ensure(match, "DER khong dung dang ma hoa chuan", 3);
    }
    if (privatePart) OPENSSL_cleanse(data.data(), data.size());
    return key;
}

string digestKeyId(EVP_PKEY* key) {
    Bytes der = encodeDer(key, false);
    unsigned char digest[EVP_MAX_MD_SIZE] = {};
    unsigned int digestLength = 0;
    ensureCrypto(EVP_Digest(der.data(), der.size(), digest, &digestLength, EVP_sha256(), nullptr) == 1
           && digestLength == 32, "Khong bam duoc SHA-256 key ID");
    const char* hex = "0123456789abcdef";
    string result;
    for (unsigned int i = 0; i < digestLength; ++i) {
        result += hex[digest[i] >> 4];
        result += hex[digest[i] & 15];
    }
    return result;
}

void validatePublic(EVP_PKEY* key) {
    KeyContext context(EVP_PKEY_CTX_new_from_pkey(nullptr, key, nullptr), EVP_PKEY_CTX_free);
    ensure(bool(context), "Khong tao duoc context kiem tra public key", 3);
    ensure(EVP_PKEY_public_check(context.get()) == 1, "Khoa cong khai khong hop le", 3);
}

void validatePrivate(EVP_PKEY* key) {
    KeyContext context(EVP_PKEY_CTX_new_from_pkey(nullptr, key, nullptr), EVP_PKEY_CTX_free);
    ensure(bool(context), "Khong tao duoc context kiem tra private key", 3);
    ensure(EVP_PKEY_private_check(context.get()) == 1, "Khoa rieng khong hop le", 3);
    ensure(EVP_PKEY_pairwise_check(context.get()) == 1, "RSA pairwise check that bai", 3);
}

void checkMatch(const fs::path& privateFile, const fs::path& publicFile) {
    Key privateKey = loadKey(privateFile, true);
    Key publicKey = loadKey(publicFile, false);
    validatePrivate(privateKey.get());
    validatePublic(publicKey.get());
    if (EVP_PKEY_eq(privateKey.get(), publicKey.get()) != 1) {
        cout << "MISMATCH\n";
        throw LabError(4, "Khoa cong khai va khoa rieng khong thuoc cung cap");
    }
    cout << "MATCH\n";
}

void keyInfo(const fs::path& publicFile, bool showPem = false) {
    Key key = loadKey(publicFile, false);
    validatePublic(key.get());
    BIGNUM* nRaw = nullptr;
    BIGNUM* eRaw = nullptr;
    int statusN = EVP_PKEY_get_bn_param(key.get(), OSSL_PKEY_PARAM_RSA_N, &nRaw);
    int statusE = EVP_PKEY_get_bn_param(key.get(), OSSL_PKEY_PARAM_RSA_E, &eRaw);
    Bn n(nRaw, BN_clear_free), e(eRaw, BN_clear_free);
    ensureCrypto(statusN == 1 && statusE == 1 && bool(n) && bool(e), "Khong lay duoc n, e");
```

```
    cout << "Algorithm: RSA\nBits: " << EVP_PKEY_get_bits(key.get()) << '\n';
    cout << "e: " << bnToText(e.get()) << "\n";
    cout << "n (hex): " << bnToText(n.get(), true) << "\n";
    cout << "Key ID (SHA-256 / SPKI DER): " << digestKeyId(key.get()) << '\n';
    if (showPem) {
        Bytes pem = encodePem(key.get(), false);
        cout << "Public key PEM:\n" << string(pem.begin(), pem.end());
    }
    cout << "Private-key contents are intentionally not displayed.\n";
}

void keygen(int bits, const fs::path& directory, bool force = false) {
    ensure(!fs::exists(directory) || fs::is_directory(directory), "Out-dir khong phai thu muc", 2);
    fs::create_directories(directory);
    const vector<string> files = {"private.pem", "private.der", "public.pem", "public.der"};
    for (const string& filename : files) {
        ensure(force || !fs::exists(directory / filename),
               "Files already exist. Move or rename them before generating a new key.", 2);
    }
    Key key = generateKey(bits);
    validatePrivate(key.get());
    Bytes privatePem = encodePem(key.get(), true);
    lab::BufferWiper wipePem{privatePem};
    Bytes privateDer = encodeDer(key.get(), true);
    lab::BufferWiper wipeDer{privateDer};
    Bytes publicPem = encodePem(key.get(), false);
    Bytes publicDer = encodeDer(key.get(), false);
    writeBytes(directory / "private.pem", privatePem, force, true);
    writeBytes(directory / "private.der", privateDer, force, true);
    writeBytes(directory / "public.pem", publicPem, force);
    writeBytes(directory / "public.der", publicDer, force);
    OPENSSL_cleanse(privatePem.data(), privatePem.size());
    OPENSSL_cleanse(privateDer.data(), privateDer.size());
    cout << "[PASS] Generated RSA-" << EVP_PKEY_get_bits(key.get()) << " key pair\n";
    cout << "[PASS] Saved private.pem, private.der, public.pem, public.der\n";
    cout << "WARNING: Unencrypted private key - lab use only. Protect file access.\n";
    cout << "Windows: review NTFS ACL; POSIX chmod 0600 does not apply directly.\n";
}

void keyConvert(bool privatePart, const fs::path& source, const fs::path& dest,
                const string& outform, const string& inform = "") {
    ensure(outform == "PEM" || outform == "DER", "--outform phai la PEM hoac DER");
    ensure(fs::absolute(source).lexically_normal() != fs::absolute(dest).lexically_normal(),
           "Khong cho ghi de len file dau vao");
    Key key = loadKey(source, privatePart, inform);
    Bytes output = outform == "PEM" ? encodePem(key.get(), privatePart)
                                    : encodeDer(key.get(), privatePart);
    lab::BufferWiper wipeOutput{output,privatePart};
    writeBytes(dest, output, false, privatePart);
    if (privatePart) OPENSSL_cleanse(output.data(), output.size());
    cout << "[PASS] Converted to " << outform << ": " << dest.string() << '\n';
}

// Tạo tham số từ đầu cho khóa minh họa (khác với sinh khóa EVP dùng thực tế).
void educationalRsa(int bits, bool showPrivate) {
    ensure(bits >= 128 && bits <= 1024 && bits % 2 == 0,
           "Educational --bits phai chan va trong [128,1024]");
    cout << "EDUCATIONAL KEY - NOT FOR USE\n";
    auto ctx = newCtx();
    auto p = newBn(), q = newBn(), pm1 = newBn(), qm1 = newBn();
    auto n = newBn(), phi = newBn(), lambda = newBn(), gcd = newBn();
    auto e = newBn(), tmp = newBn(), dP = newBn(), dQ = newBn();
    ensureCrypto(BN_set_word(e.get(), 65537) == 1, "Khong thiet lap duoc e");

    // Chỉ nhận số nguyên tố thỏa gcd(e,p-1)=1 và gcd(e,q-1)=1.
    auto drawPrime = [&](BIGNUM* prime, BIGNUM* minusOne) {
        do {
            ensureCrypto(BN_generate_prime_ex2(prime, bits / 2, 0, nullptr, nullptr,
                                               nullptr, ctx.get()) == 1, "Sinh so nguyen to loi");
            ensureCrypto(BN_copy(minusOne, prime) != nullptr && BN_sub_word(minusOne, 1) == 1,
                   "Tinh prime-1 loi");
            ensureCrypto(BN_gcd(gcd.get(), e.get(), minusOne, ctx.get()) == 1, "Tinh gcd loi");
        } while (!BN_is_one(gcd.get()));
    };
    drawPrime(p.get(), pm1.get());
    do { drawPrime(q.get(), qm1.get()); } while (BN_cmp(p.get(), q.get()) == 0);
    ensureCrypto(BN_mul(n.get(), p.get(), q.get(), ctx.get()) == 1, "Tinh n loi");
    ensureCrypto(BN_mul(phi.get(), pm1.get(), qm1.get(), ctx.get()) == 1, "Tinh phi loi");
```

```
    ensureCrypto(BN_gcd(gcd.get(), pm1.get(), qm1.get(), ctx.get()) == 1, "Tinh gcd(p-1,q-1) loi");
    ensureCrypto(BN_div(lambda.get(), nullptr, phi.get(), gcd.get(), ctx.get()) == 1,
           "Tinh lambda loi");
    Bn dPhi(BN_mod_inverse(nullptr, e.get(), phi.get(), ctx.get()), BN_clear_free);
    Bn dLam(BN_mod_inverse(nullptr, e.get(), lambda.get(), ctx.get()), BN_clear_free);
    ensureCrypto(bool(dPhi) && bool(dLam), "Khong tim duoc so mu d");
    ensureCrypto(BN_nnmod(dP.get(), dLam.get(), pm1.get(), ctx.get()) == 1 &&
           BN_nnmod(dQ.get(), dLam.get(), qm1.get(), ctx.get()) == 1,
           "Tinh CRT exponent loi");
    Bn qInv(BN_mod_inverse(nullptr, q.get(), p.get(), ctx.get()), BN_clear_free);
    ensureCrypto(bool(qInv), "Tinh qInv loi");


    auto verifyInverse = [&](BIGNUM* d, BIGNUM* modulus) {
        ensureCrypto(BN_mod_mul(tmp.get(), e.get(), d, modulus, ctx.get()) == 1,
               "Kiem tra ed mod loi");
        return BN_is_one(tmp.get());
    };
    bool checkPhi = verifyInverse(dPhi.get(), phi.get());
    bool checkLambda = verifyInverse(dLam.get(), lambda.get());
    bool roundtrips = true, crtPass = true;
    auto m = newBn(), cipher = newBn(), directPhi = newBn(), directLam = newBn();
    auto m1 = newBn(), m2 = newBn(), h = newBn(), crt = newBn(), hq = newBn();
    for (int i = 0; i < 20; ++i) {
        ensureCrypto(BN_rand_range(m.get(), n.get()) == 1, "Sinh thong diep ngau nhien loi");
        ensureCrypto(BN_mod_exp(cipher.get(), m.get(), e.get(), n.get(), ctx.get()) == 1,
               "Ma hoa RSA minh hoa loi");
        ensureCrypto(BN_mod_exp(directPhi.get(), cipher.get(), dPhi.get(), n.get(), ctx.get()) == 1 &&
               BN_mod_exp(directLam.get(), cipher.get(), dLam.get(), n.get(), ctx.get()) == 1,
               "Giai ma RSA minh hoa loi");
        roundtrips &= BN_cmp(directPhi.get(), m.get()) == 0 && BN_cmp(directLam.get(), m.get()) == 0;
        ensureCrypto(BN_mod_exp(m1.get(), cipher.get(), dP.get(), p.get(), ctx.get()) == 1 &&
               BN_mod_exp(m2.get(), cipher.get(), dQ.get(), q.get(), ctx.get()) == 1,
               "Tinh CRT m1,m2 loi");
        ensureCrypto(BN_mod_sub(h.get(), m1.get(), m2.get(), p.get(), ctx.get()) == 1 &&
               BN_mod_mul(h.get(), h.get(), qInv.get(), p.get(), ctx.get()) == 1 &&
               BN_mul(hq.get(), h.get(), q.get(), ctx.get()) == 1 &&
               BN_add(crt.get(), m2.get(), hq.get()) == 1,
               "Tinh CRT m loi");
        crtPass &= BN_cmp(crt.get(), directLam.get()) == 0;
    }
    cout << "n bits: " << BN_num_bits(n.get()) << "\n";
    cout << "n (hex): " << bnToText(n.get(), true) << "\n";
    cout << "e (hex): " << bnToText(e.get(), true) << "\n";
    if (showPrivate) {
        cout << "PRIVATE TOY PARAMETERS - NEVER USE FOR REAL KEYS\n";
        for (auto entry : vector<pair<string, const BIGNUM*>>{
                {"p", p.get()}, {"q", q.get()}, {"phi", phi.get()},
                {"lambda", lambda.get()}, {"d_phi", dPhi.get()},
                {"d_lambda", dLam.get()}, {"dP", dP.get()},
                {"dQ", dQ.get()}, {"qInv", qInv.get()}}) {
            cout << entry.first << ": " << bnToText(entry.second, true) << '\n';
        }
    }
    auto printCheck = [](const string& title, bool pass) {
        cout << (pass ? "[PASS] " : "[FAIL] ") << title << '\n';
    };
    printCheck("e*d_phi == 1 (mod phi)", checkPhi);
    printCheck("e*d_lambda == 1 (mod lambda)", checkLambda);
    printCheck("20 RSA round-trips with both private exponents", roundtrips);
    printCheck("20 CRT decryptions match direct decryptions", crtPass);
    ensureCrypto(checkPhi && checkLambda && roundtrips && crtPass,
           "Kiem thu RSA tham so that bai");
}


string optionValue(const vector<string>& args, const string& name,
                   const string& fallback = "") {
    auto it = find(args.begin(), args.end(), name);
    if (it == args.end()) return fallback;
    ensure(it + 1 != args.end(), "Thieu gia tri cho " + name);
    return *(it + 1);
}


bool hasFlag(const vector<string>& args, const string& flag) {
    return find(args.begin(), args.end(), flag) != args.end();
}


string requiredOption(const vector<string>& args, const string& name) {
    string value = optionValue(args, name);
```

```
    ensure(!value.empty(), "Thieu tham so " + name);
    return value;
}


void menu() {
    cout << "OpenSSL header: " << OPENSSL_VERSION_TEXT << '\n';
    cout << "OpenSSL library: " << OpenSSL_version(OPENSSL_VERSION) << '\n';
    while (true) {
        cout << "\n===== RSA KEY LAB - TASK B =====\n"
             << "1. Generate and save RSA-3072 (PEM/DER)\n"
             << "2. Load 4 key files and check MATCH\n"
             << "3. Display PUBLIC key (n, e, PEM, key ID)\n"
             << "4. Educational RSA parameters (512-bit toy)\n"
             << "0. Exit\nChoice: ";
        string choice;
        if (!getline(cin, choice) || choice == "0") return;
        try {
            if (choice == "1") keygen(3072, ".");
            else if (choice == "2") {
                const vector<string> privs = {"private.pem", "private.der"};
                const vector<string> pubs = {"public.pem", "public.der"};
                for (const auto& priv : privs) for (const auto& pub : pubs) {
                    cout << priv << " vs " << pub << ": ";
                    checkMatch(priv, pub);
                }
                cout << "[PASS] All 4 key combinations MATCH\n";
            } else if (choice == "3") keyInfo("public.pem", true);
            else if (choice == "4") educationalRsa(512);
            else cout << "Invalid menu choice.\n";
        } catch (const exception& e) {
            cerr << "[ERROR] " << e.what() << '\n';
        }
    }
}

void usage() {
    cerr << "Usage:\n"
         << "  task_b_rsa_keys.exe rsa-params --bits 512 [--show-private]\n"
         << "  task_b_rsa_keys.exe keygen --bits 3072 --out-dir keys [--force]\n"
         << "  task_b_rsa_keys.exe key-check --priv private.pem --pub public.pem\n"
         << "  task_b_rsa_keys.exe key-info --pub public.pem\n"
         << "  task_b_rsa_keys.exe key-convert --type public|private --in FILE --out FILE --outform PEM|DER [--
inform PEM|DER]\n";
}

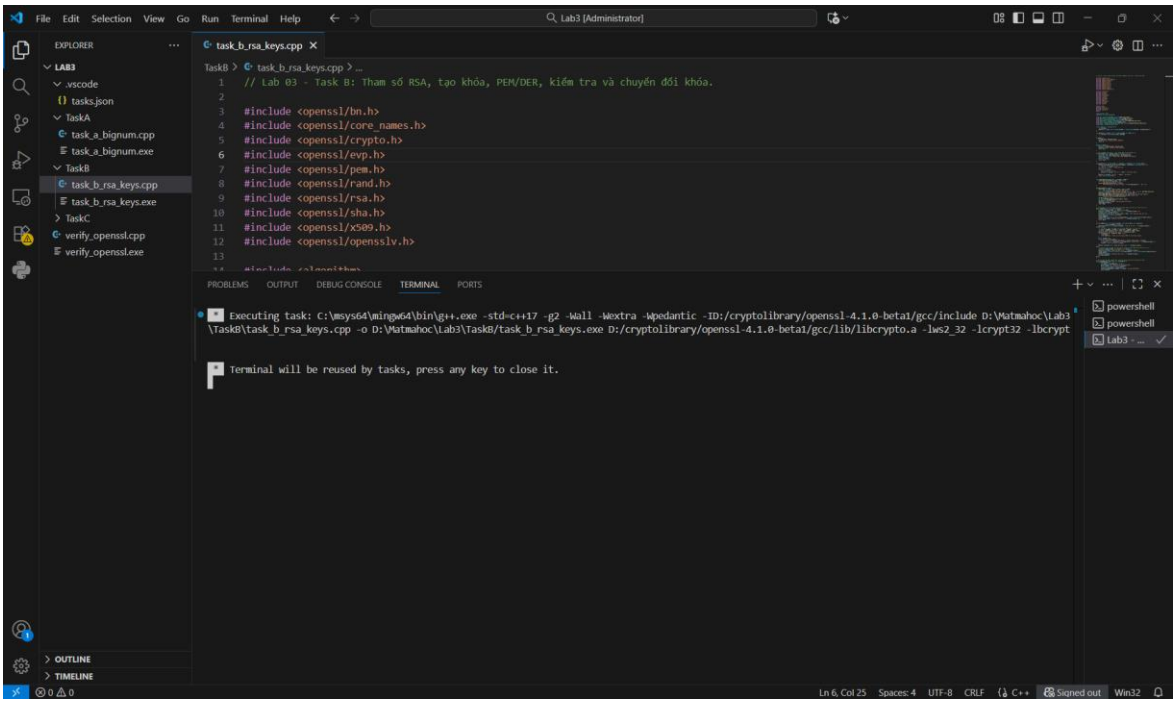
int entry(int argc, char** argv) {
    try {
        if (argc == 1) { menu(); return 0; }
        vector<string> args(argv + 2, argv + argc);
        string command = argv[1];
        vector<string> allowed;
        if(command=="rsa-params") allowed={"--bits","--show-private"};
        else if(command=="keygen") allowed={"--bits","--out-dir","--force"};
        else if(command=="key-check") allowed={"--priv","--pub"};
        else if(command=="key-info") allowed={"--pub"};
        else if(command=="key-convert") allowed={"--type","--in","--out","--outform","--inform"};
        vector<string> seen;
        for(size_t i=0;i<args.size();++i) {
            const auto flag=args[i];
            ensure(find(allowed.begin(),allowed.end(),flag)!=allowed.end(),"Tham so khong duoc ho tro: "+flag);
            ensure(find(seen.begin(),seen.end(),flag)==seen.end(),"Tham so lap lai: "+flag);
            seen.push_back(flag);
            if(flag!="--force" && flag!="--show-private") {
                ensure(i+1<args.size() && args[i+1].rfind("--",0)!=0,"Thieu gia tri cho "+flag); ++i;
            }
        }
        if (command == "rsa-params") {
            int bits = parseInt(optionValue(args, "--bits", "512"), 128, 1024, "--bits");
            educationalRsa(bits, hasFlag(args, "--show-private"));
        } else if (command == "keygen") {
            int bits = parseInt(optionValue(args, "--bits", "3072"), 2048, 4096, "--bits");
            keygen(bits, requiredOption(args, "--out-dir"), hasFlag(args, "--force"));
        } else if (command == "key-check") {
            checkMatch(requiredOption(args, "--priv"), requiredOption(args, "--pub"));
        } else if (command == "key-info") {
            keyInfo(requiredOption(args, "--pub"));
        } else if (command == "key-convert") {
            string type = requiredOption(args, "--type");
            ensure(type == "public" || type == "private", "--type phai la public hoac private");
```

```
            keyConvert(type == "private", requiredOption(args, "--in"),
                       requiredOption(args, "--out"), requiredOption(args, "--outform"),
                       optionValue(args, "--inform"));
        } else {
            usage();
            return 1;
        }
        return 0;
    } catch (const LabError& ex) {
        cerr << "Error: " << ex.what() << '\n';
        return ex.exitCode;
    } catch (const fs::filesystem_error& ex) {
        cerr << "Error: " << ex.what() << '\n';
        return 2;
    } catch (const exception& ex) {
        cerr << "Error: " << ex.what() << '\n';
        return 5;
    }
}

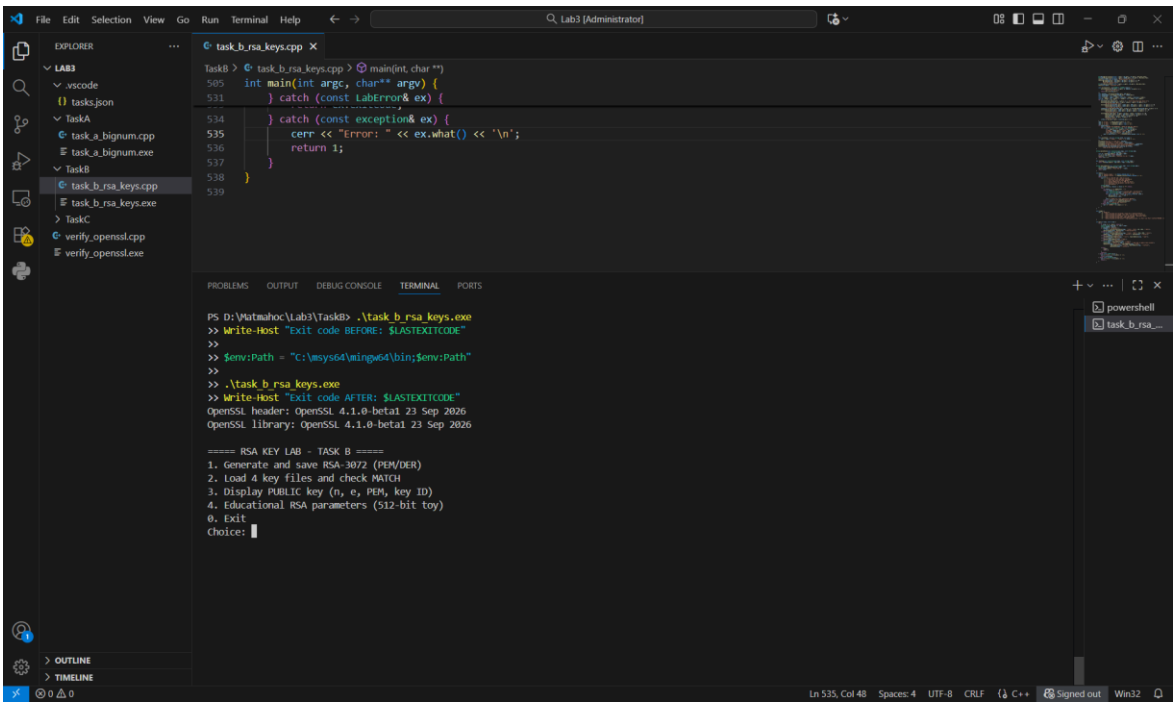
} // namespace task_b
```

2. Hình ảnh thực hiện và kết quả kiểm thử

Biên dịch và giao diện Task B



Hình 16. Biên dịch chương trình Task B bằng GCC MinGW64 và thư viện tĩnh OpenSSL 4.1.0-beta1.



Hình 17. Giao diện chương trình Task B.

Tham số RSA và sinh khóa

```
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe
>> Write-Host "Exit code BEFORE: $LASTEXITCODE"
>>
>> $env:Path = "C:\msys64\mingw64\bin;$env:Path"
>>
>> .\task_b_rsa_keys.exe
>> Write-Host "Exit code AFTER: $LASTEXITCODE"
OpenSSL header: OpenSSL 4.1.0-beta1 23 Sep 2026
OpenSSL library: OpenSSL 4.1.0-beta1 23 Sep 2026

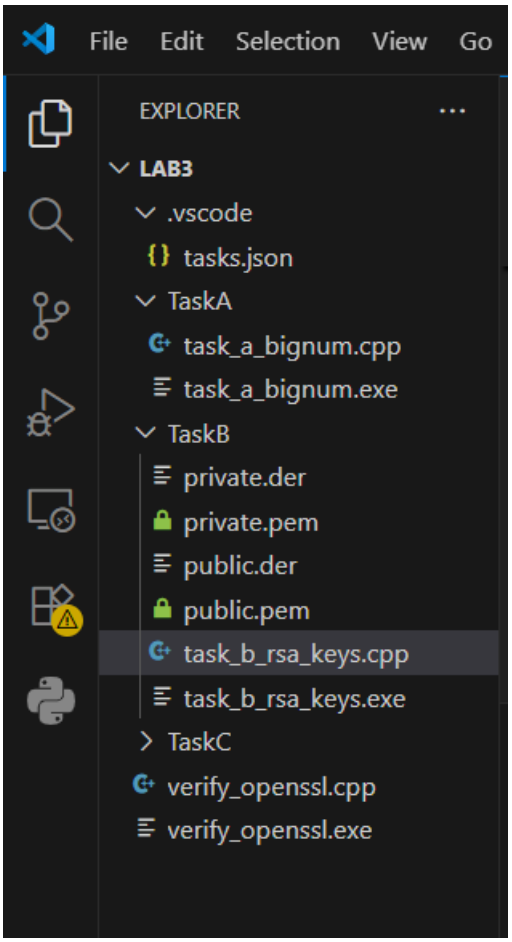
===== RSA KEY LAB - TASK B =====
1. Generate and save RSA-3072 (PEM/DER)
2. Load 4 key files and check MATCH
3. Display PUBLIC key (n, e, PEM, key ID)
4. Educational RSA parameters (512-bit toy)
0. Exit
Choice: 4
EDUCATIONAL KEY - NOT FOR USE
n bits: 512
n (hex): FF10A4C5E2A69D97C05E6A67CC6825594DF0730D04560109BEF0CFE4D8C41D3F8DB855E4D1C2E1BB234C4EBBA6BAD20CCC7E58690FFAC072C465F89B5D788283
e (hex): 010001
[PASS] e*d_phi == 1 (mod phi)
[PASS] e*d_lambda == 1 (mod lambda)
[PASS] 20 RSA round-trips with both private exponents
[PASS] 20 CRT decryptions match direct decryptions
```

Hình 18. Kiểm thử quá trình xây dựng tham số RSA và đối chiếu kết quả giải mã bằng số mũ bí mật và CRT.

```
===== RSA KEY LAB - TASK B =====
1. Generate and save RSA-3072 (PEM/DER)
2. Load 4 key files and check MATCH
3. Display PUBLIC key (n, e, PEM, key ID)
4. Educational RSA parameters (512-bit toy)
0. Exit
Choice: 1
[PASS] Generated RSA-3072 key pair
[PASS] Saved private.pem, private.der, public.pem, public.der
WARNING: Unencrypted private key - lab use only. Protect file access.
Windows: review NTFS ACL; POSIX chmod 0600 does not apply directly.
```

Hình 19. Sinh cặp khóa RSA 3072 bit bằng OpenSSL và lưu dưới định dạng PEM, DER.

Đọc và chuyển đổi khóa



Hình 20. Cấu trúc thư mục Task B sau khi tạo bốn file khóa RSA bằng OpenSSL.

```
===== RSA KEY LAB - TASK B =====
1. Generate and save RSA-3072 (PEM/DER)
2. Load 4 key files and check MATCH
3. Display PUBLIC key (n, e, PEM, key ID)
4. Educational RSA parameters (512-bit toy)
0. Exit
Choice: 2
private.pem vs public.pem: MATCH
private.pem vs public.der: MATCH
private.der vs public.pem: MATCH
private.der vs public.der: MATCH
[PASS] All 4 key combinations MATCH
```

Hình 21. Kiểm tra sự tương ứng của cặp khóa RSA thông qua bốn tổ hợp khóa riêng và khóa công khai ở định dạng PEM và DER.

```
===== RSA KEY LAB - TASK B =====
1. Generate and save RSA-3072 (PEM/DER)
2. Load 4 key files and check MATCH
3. Display PUBLIC key (n, e, PEM, key ID)
4. Educational RSA parameters (512-bit toy)
0. Exit
Choice: 3
Algorithm: RSA
Bits: 3072
e: 65537
n (hex): B25C79C9CE902660ADB8B33B9B25E6701163E213BAA0940132EF367B14267B1E3C6869FE38F80CE5C93A45F761BF7D8D9F9352946FB95B997AE4FA3E6E994E0913110BB207B4A41039DA
9D748A9F779CE3E6DAE1AF05B3231E21CD8DDABC37D4D9F707015316198E32688597A061647BA37627882A223243DEE9B00402DEDF024C35A452A1A995F333294150D1BAF6259052B283AAFD70751
955D92BF6BCD2C6BAF9144046307BEE654CD09E9C21D637B9F668499ADF04C1B74AD57921B5749A53E46867E9D1466C56D23626A68E5A86D8A183AB7E8A3863B5E8BF23673CD2B11BEB08A9099EEB
3656518C754CFACE8F7E54007F0C986712D5674018BA2E2CBCE3056E58A94660B3F73B542D2AF6CE2E1D4C93DE3155FC84F6FC1DEDD2E8BDEAD485990FF846A30A045F89767A1518C1B30F5455600
4977F878DE9086C0A5AB282F1DF59932ED3B0489ECF37AADED77409B8AF2D4448329F184A77620D69E0A14BAFD2616147E46CC60FD928F858DF13716D07E88749AB0B81FE31E1058771A1
Key ID (SHA-256 / SPKI DER): 0bde83ee668999360d84a2a694167edc012cce38f25c36055642526d24847c45
Public key PEM:
-----BEGIN PUBLIC KEY-----
MIIBojANBgkqhkiG9w0BAQEFAAOCAY8AMIIBigKCAYEAslx5yc6QJmCtuLM7myXm
cBFj4hO6oJQBMu82exQmex48aGn+OPgM5ck6Rfdhv32Nn5NSlG+5W5l65Po+bplO
CRMRC7IHtKQQOdqddIqfd5zj5trhrwWzIx4hzY3avDfU2fcHAVMWGY4yaIwXoGFk
e6N2J4gqIjJD3umwBALe3wJMNaRSoamV8zMpQVDRuvYlkFKyg6r9cHUZVdkr9rzS
xrr5FEBGMHvuZUzQnpwh1je59mhJmt8EwbdK1XkhtXSaU+RoZ+nRRmxW0jYmpo5a
htihg6t+ijhjtei/I2c80rEb6wipCZ7rNlZRjHVM+s6PflQAfwyYZxLVZ0AYui4s
vOMFblipRmCz9ztULSr2zi4dTJPeMVX8hPb8He3S6L3q1IWZD/hGowoEX4l2ehUY
wbMPVFVgBJd/h43pCGwKWrKC8d9Zky7TsEiezzeq3td0CbivLURIMp8YSndiDWng
oUuv0mFhR+Rsxg/ZKPhY3xNxbQfoh0mrC4H+MeEFh3GhAgMBAAE=
-----END PUBLIC KEY-----
Private-key contents are intentionally not displayed.
```

Hình 22. Hiển thị thông tin khóa công khai RSA 3072 bit, bao gồm modulus n, số mũ e, Key ID và biểu diễn khóa theo định dạng PEM.

```
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-info --pub public.pem
Algorithm: RSA
Bits: 3072
e: 65537
n (hex): B25C79C9CE902660ADB8B33B9B25E6701163E213BAA0940132EF367B14267B1E3C6869FE38F80CE5C93A45F761BF7D8D9F9352946FB95B997AE4FA3E6E994E0913110BB207B4A41039DA
9D748A9F779CE3E6DAE1AF05B3231E21CD8DDABC37D4D9F707015316198E32688597A061647BA37627882A223243DEE9B00402DEDF024C35A452A1A995F333294150D1BAF6259052B283AAFD70751
955D92BF6BCD2C6BAF9144046307BEE654CD09E9C21D637B9F668499ADF04C1B74AD57921B5749A53E46867E9D1466C56D23626A68E5A86D8A183AB7E8A3863B5E8BF23673CD2B11BEB08A9099EEB
3656518C754CFACE8F7E54007F0C986712D5674018BA2E2CBCE3056E58A94660B3F73B542D2AF6CE2E1D4C93DE3155FC84F6FC1DEDD2E8BDEAD485990FF846A30A045F89767A1518C1B30F5455600
4977F878DE9086C0A5AB282F1DF59932ED3B0489ECF37AADED77409B8AF2D4448329F184A77620D69E0A14BAFD2616147E46CC60FD928F858DF13716D07E88749AB0B81FE31E1058771A1
Key ID (SHA-256 / SPKI DER): 0bde83ee668999360d84a2a694167edc012cce38f25c36055642526d24847c45
Private-key contents are intentionally not displayed.
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-info --pub public.der
Algorithm: RSA
Bits: 3072
e: 65537
n (hex): B25C79C9CE902660ADB8B33B9B25E6701163E213BAA0940132EF367B14267B1E3C6869FE38F80CE5C93A45F761BF7D8D9F9352946FB95B997AE4FA3E6E994E0913110BB207B4A41039DA
9D748A9F779CE3E6DAE1AF05B3231E21CD8DDABC37D4D9F707015316198E32688597A061647BA37627882A223243DEE9B00402DEDF024C35A452A1A995F333294150D1BAF6259052B283AAFD70751
955D92BF6BCD2C6BAF9144046307BEE654CD09E9C21D637B9F668499ADF04C1B74AD57921B5749A53E46867E9D1466C56D23626A68E5A86D8A183AB7E8A3863B5E8BF23673CD2B11BEB08A9099EEB
3656518C754CFACE8F7E54007F0C986712D5674018BA2E2CBCE3056E58A94660B3F73B542D2AF6CE2E1D4C93DE3155FC84F6FC1DEDD2E8BDEAD485990FF846A30A045F89767A1518C1B30F5455600
4977F878DE9086C0A5AB282F1DF59932ED3B0489ECF37AADED77409B8AF2D4448329F184A77620D69E0A14BAFD2616147E46CC60FD928F858DF13716D07E88749AB0B81FE31E1058771A1
Key ID (SHA-256 / SPKI DER): 0bde83ee668999360d84a2a694167edc012cce38f25c36055642526d24847c45
Private-key contents are intentionally not displayed.
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 23. Đối chiếu thông tin khóa công khai RSA và giá trị Key ID giữa hai định dạng PEM và DER.

```
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-convert --type public --in public.pem --out public_roundtrip.der --outform DER
[PASS] Converted to DER: public_roundtrip.der
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-convert --type public --in public_roundtrip.der --out public_roundtrip.pem --outform PEM
[PASS] Converted to PEM: public_roundtrip.pem
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-convert --type private --in private.pem --out private_roundtrip.der --outform DER
[PASS] Converted to DER: private_roundtrip.der
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-convert --type private --in private_roundtrip.der --out private_roundtrip.pem --outform PEM
[PASS] Converted to PEM: private_roundtrip.pem
PS D:\Matmahoc\Lab3\TaskB> $pub1 = (Get-FileHash .\public.pem -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskB> $pub2 = (Get-FileHash .\public_roundtrip.pem -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskB> $priv1 = (Get-FileHash .\private.pem -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskB> $priv2 = (Get-FileHash .\private_roundtrip.pem -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Public PEM identical:" ($pub1 -eq $pub2)
Public PEM identical: True
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Private PEM identical:" ($priv1 -eq $priv2)
Private PEM identical: True
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 24. Kiểm thử chuyển đổi khóa RSA giữa định dạng PEM và DER, xác nhận nội dung khóa không thay đổi sau quá trình chuyển đổi hai chiều.

Kiểm thử khóa không hợp lệ

```
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe keygen --bits 2048 --out-dir mismatch_test
[PASS] Generated RSA-2048 key pair
[PASS] Saved private.pem, private.der, public.pem, public.der
WARNING: Unencrypted private key - lab use only. Protect file access.
Windows: review NTFS ACL; POSIX chmod 0600 does not apply directly.
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-check --priv private.pem --pub mismatch_test/public.pem
MISMATCH
Error: Khoa cong khai va khoa rieng khong thuoc cung cap
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 4
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 25. Kiểm thử phát hiện hai khóa RSA không thuộc cùng một cặp, chương trình trả về MISMATCH và mã lỗi 4.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Matmahoc\Lab3\TaskB> $bytes = [System.IO.File]::ReadAllBytes(
>> (Resolve-Path ".\public.der").Path
>> )
PS D:\Matmahoc\Lab3\TaskB> $broken = [byte[]]::new($bytes.Length - 10)
PS D:\Matmahoc\Lab3\TaskB> [System.Array]::Copy($bytes, $broken, $broken.Length)
PS D:\Matmahoc\Lab3\TaskB> [System.IO.File]::WriteAllBytes(
>> (Join-Path (Get-Location).Path "public_truncated.der"),
>> $broken
>> )
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-info --pub public_truncated.der
Error: DER phai la khoa cong khai SPKI
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 3
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 26. Kiểm thử khả năng phát hiện và từ chối file khóa công khai DER bị cắt mất dữ liệu, với mã lỗi 3.

```
PS D:\Matmahoc\Lab3\TaskB> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" rsa -pubin -in public.pem -RSAPublicKey_out -out public_pkcs1.pem
writing RSA key
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-info --pub public_pkcs1.pem
Error: Nhận PEM khong dung SPKI/PKCS#8
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 3
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 27. Kiểm thử khả năng từ chối khóa công khai RSA ở cấu trúc PKCS#1 thay vì SPKI, với mã lỗi 3.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Matmahoc\Lab3\TaskB> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:1024 -out weak_privat
e.pem
PS D:\Matmahoc\Lab3\TaskB> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" pkey -in weak_private.pem -pubout -out weak_public.pem
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-info --pub weak_public.pem
Error: Khoa RSA khong du do dai toi thieu 2048 bit
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 3
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 28. Kiểm thử khả năng từ chối khóa RSA 1024 bit không đáp ứng yêu cầu độ dài khóa tối thiểu 2048 bit.

Đối chiếu OpenSSL và quyền truy cập

```
PS D:\Matmahoc\Lab3\TaskB> $openssl = "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe"
PS D:\Matmahoc\Lab3\TaskB> & $openssl pkey -in private.pem -check -noout
Key is valid
PS D:\Matmahoc\Lab3\TaskB> Write-Host "PEM exit code: $LASTEXITCODE"
PEM exit code: 0
PS D:\Matmahoc\Lab3\TaskB> & $openssl pkey -inform DER -in private.der -check -noout
Key is valid
PS D:\Matmahoc\Lab3\TaskB> Write-Host "DER exit code: $LASTEXITCODE"
DER exit code: 0
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 29. Kiểm tra tính hợp lệ của khóa riêng RSA 3072 bit ở hai định dạng PEM và DER bằng OpenSSL CLI.

```
PS D:\Matmahoc\Lab3\TaskB> whoami
laptop-h5aerr5o\acer
PS D:\Matmahoc\Lab3\TaskB> icacls .\private.pem
.\private.pem BUILTIN\Administrators:(I)(F)
              NT AUTHORITY\SYSTEM:(I)(F)
              NT AUTHORITY\Authenticated Users:(I)(M)
              BUILTIN\Users:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB> icacls .\private.der
.\private.der BUILTIN\Administrators:(I)(F)
              NT AUTHORITY\SYSTEM:(I)(F)
              NT AUTHORITY\Authenticated Users:(I)(M)
              BUILTIN\Users:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 30. Kiểm tra danh sách quyền truy cập NTFS đối với các file khóa riêng RSA trên Windows

```
PS D:\Matmahoc\Lab3\TaskB> icacls .\private.pem /save .\acl_private_pem_before.txt
processed file: .\private.pem
Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB> icacls .\private.der /save .\acl_private_der_before.txt
processed file: .\private.der
Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB> $me = (whoami).Trim()
PS D:\Matmahoc\Lab3\TaskB> foreach ($file in @(".\private.pem", ".\private.der")) {
>> icacls $file /grant:r "${me}:(F)" "*S-1-5-18:(F)" "*S-1-5-32-544:(F)"
>> icacls $file /inheritance:r
>> }
processed file: .\private.pem
Successfully processed 1 files; Failed processing 0 files
processed file: .\private.pem
Successfully processed 1 files; Failed processing 0 files
processed file: .\private.der
Successfully processed 1 files; Failed processing 0 files
processed file: .\private.der
Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB> icacls .\private.pem
.\private.pem BUILTIN\Administrators:(F)
              NT AUTHORITY\SYSTEM:(F)
              LAPTOP-H5AERR5O\ACER:(F)

Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB> icacls .\private.der
.\private.der BUILTIN\Administrators:(F)
              NT AUTHORITY\SYSTEM:(F)
              LAPTOP-H5AERR5O\ACER:(F)

Successfully processed 1 files; Failed processing 0 files
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 31. Cấu hình và xác minh quyền truy cập NTFS đối với hai file khóa riêng RSA trên Windows.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Matmahoc\Lab3\TaskB> $openssl = "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe"
PS D:\Matmahoc\Lab3\TaskB> & $openssl pkey -pubin -in public.pem -outform DER -out public_cli_check.der
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Convert exit code: $LASTEXITCODE"
Convert exit code: 0
PS D:\Matmahoc\Lab3\TaskB> & $openssl dgst -sha256 public_cli_check.der
SHA2-256(public_cli_check.der)= 0bde83ee668999360d84a2a694167edc012cce38f25c36055642526d24847c45
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Hash exit code: $LASTEXITCODE"
Hash exit code: 0
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 32. Xác minh giá trị Key ID bằng thuật toán SHA-256 trên khóa công khai SPKI DER thông qua OpenSSL CLI.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Matmahoc\Lab3\TaskB> $before = (Get-FileHash .\private.pem -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe keygen --bits 3072 --out-dir .
Error: Files already exist. Move or rename them before generating a new key.
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskB> $after = (Get-FileHash .\private.pem -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Private key unchanged:" ($before -eq $after)
Private key unchanged: True
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 33. Kiểm thử cơ chế ngăn ghi đè các file khóa RSA đã tồn tại và xác minh khóa riêng không bị thay đổi.

Phân tích DER và thay đổi dữ liệu khóa

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Matmahoc\Lab3\TaskB> Get-Item .\public.der | Select-Object Name, Length

Name        Length
----        ------
public.der     422


PS D:\Matmahoc\Lab3\TaskB> Format-Hex .\public.der | Select-Object -First 6


           Path: D:\Matmahoc\Lab3\TaskB\public.der

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000000   30 82 01 A2 30 0D 06 09 2A 86 48 86 F7 0D 01 01  0.¢0...*H÷...
00000010   01 05 00 03 82 01 8F 00 30 82 01 8A 02 82 01 81  ......0...
00000020   00 B2 5C 79 C9 CE 90 26 60 AD B8 B3 3B 9B 25 E6  .²\yÉÎ&` ¸³;%æ
00000030   70 11 63 E2 13 BA A0 94 01 32 EF 36 7B 14 26 7B  p.câ.º .2ï6{.&{
00000040   1E 3C 68 69 FE 38 F8 0C E5 C9 3A 45 F7 61 BF 7D  .<hiþ8ø.åÉ:E÷a¿}
00000050   8D 9F 93 52 94 6F B9 5B 99 7A E4 FA 3E 6E 99 4E  Ro¹[zäú>nN


PS D:\Matmahoc\Lab3\TaskB>
```

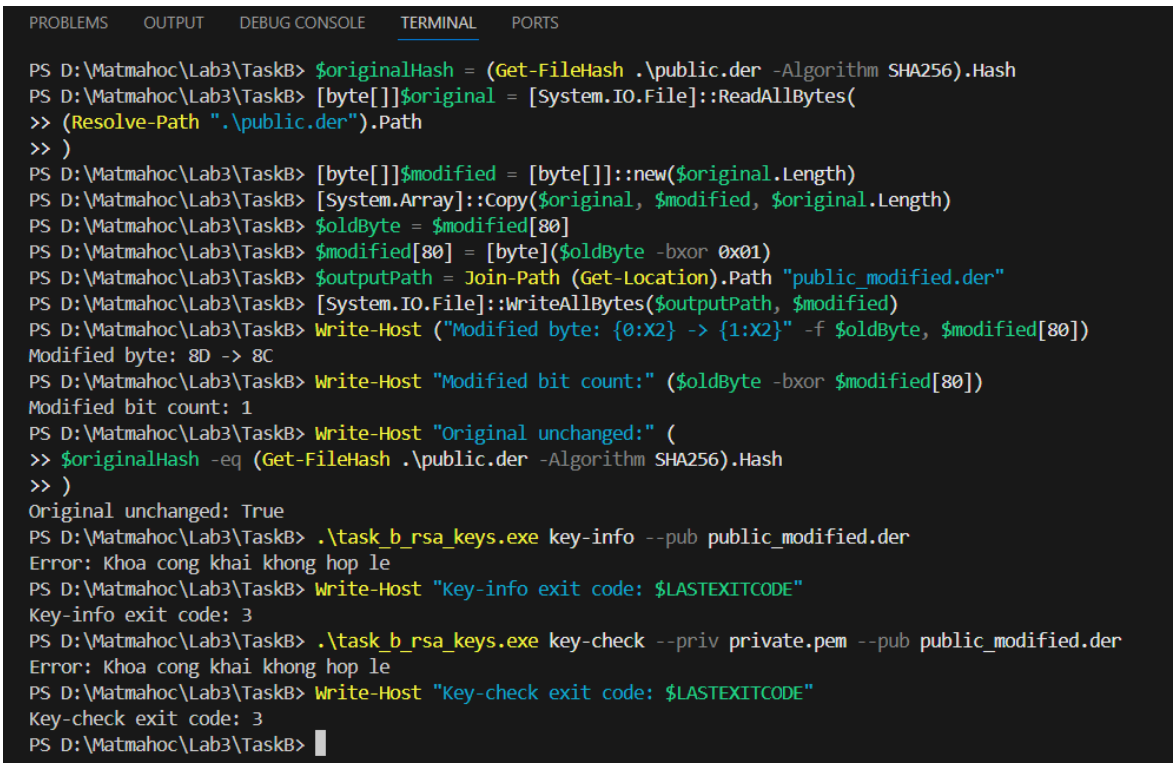
Hình 34. Quan sát dữ liệu nhị phân và phân tích các byte mở đầu của khóa công khai RSA dưới định dạng DER.

```
PS D:\Matmahoc\Lab3\TaskB> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" asn1parse -inform DER -in .\public.der -i
    0:d=0  hl=4 l= 418 cons: SEQUENCE
    4:d=1  hl=2 l=  13 cons:  SEQUENCE
    6:d=2  hl=2 l=   9 prim:   OBJECT            :rsaEncryption
   17:d=2  hl=2 l=   0 prim:   NULL
   19:d=1  hl=4 l= 399 prim:  BIT STRING
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 35. Phân tích cấu trúc ASN.1 của khóa công khai RSA dưới định dạng DER bằng OpenSSL 4.1.0-beta1.

```
PS D:\Matmahoc\Lab3\TaskB> $bytes[80] = [byte]($old -bxor 0x01)
PS D:\Matmahoc\Lab3\TaskB> [System.IO.File]::WriteAllBytes(
>> (Join-Path (Get-Location).Path "public_modified.der"),
>> $bytes
>> )
PS D:\Matmahoc\Lab3\TaskB> Write-Host ("Modified byte: {0:X2} -> {1:X2}" -f $old, $bytes[80])
Modified byte:  -> 01
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Original unchanged:" (
>> $originalHash -eq (Get-FileHash .\public.der -Algorithm SHA256).Hash
>> )
Original unchanged: False
PS D:\Matmahoc\Lab3\TaskB> .\task_b_rsa_keys.exe key-check --priv private.pem --pub public_modified.der
Error: Khoa cong khai khong hop le
PS D:\Matmahoc\Lab3\TaskB> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 3
PS D:\Matmahoc\Lab3\TaskB>
```

Hình 36. Thực nghiệm thay đổi một bit trong dữ liệu modulus của khóa công khai RSA và kiểm tra sự không tương ứng với khóa riêng ban đầu.



Hình 37. Thực nghiệm thay đổi một bit trong bản sao khóa công khai RSA dưới định dạng DER và kiểm tra tác động đến tính hợp lệ và sự tương ứng của cặp khóa.

Task C

1. Mã nguồn

Mã nguồn: src/task_c.cpp (phiên bản tích hợp CLI trong thư mục nộp bài).

```
#include "rsa_encoding.hpp"
#include "key_io.hpp"
#include "task_cli.hpp"
#include "ossl_raii.hpp"
// Lab 03 - Task C: Chuyển đổi OS2IP/I2OSP, RSA không đệm và RSA-OAEP.


#include <openssl/bn.h>
#include <openssl/core_names.h>
#include <openssl/crypto.h>
#include <openssl/err.h>
#include <openssl/evp.h>
#include <openssl/rand.h>
#include <openssl/rsa.h>
#include <openssl/pem.h>
#include <openssl/x509.h>


#include <fstream>
#include <filesystem>
#include <string_view>
#include <limits>


#include <algorithm>
#include <array>
#include <climits>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <memory>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>


namespace task_c {


using namespace std;
using Bytes = vector<unsigned char>;
using Big = unique_ptr<BIGNUM, decltype(&BN_clear_free)>;
using BnContext = unique_ptr<BN_CTX, decltype(&BN_CTX_free)>;
using RsaKey = unique_ptr<EVP_PKEY, decltype(&EVP_PKEY_free)>;
using RsaContext = unique_ptr<EVP_PKEY_CTX, decltype(&EVP_PKEY_CTX_free)>;


// Mã thoát theo quy định của đề bài (G1-G3):
// 0: thành công; 1: lỗi cú pháp hoặc đầu vào; 2: lỗi đọc/ghi;
// 3: dữ liệu sai định dạng; 4: lỗi mật mã; 5: lỗi nội bộ hoặc OpenSSL.
using LabError = lab::Error;


int totalChecks = 0;
int failedChecks = 0;


void check(const string& label, bool passed) {
    ++totalChecks;
    if (!passed) ++failedChecks;
    cout << (passed ? "[PASS] " : "[FAIL] ") << label << '\n';
}
void require(bool ok, const string& description, int exitCode = 5) {
    if (!ok) throw LabError(exitCode, description);
}
Big newBig() {
    Big x(BN_new(), BN_clear_free);
    require(bool(x), "Khong cap phat duoc BIGNUM");
    return x;
}
BnContext newBnContext() {
    BnContext ctx(BN_CTX_new(), BN_CTX_free);
    require(bool(ctx), "Khong cap phat duoc BN_CTX");
    return ctx;
```

```
}

// OS2IP theo RFC 8017: byte đầu tiên có trọng số cao nhất.
Big os2ip(const Bytes& bytes) {
    require(bytes.size() <= 1024, "OS2IP gioi han 1024 bytes", 1);
    Big number = newBig();
    if (bytes.empty()) {
        BN_zero(number.get());
    } else {
        require(BN_bin2bn(bytes.data(), static_cast<int>(bytes.size()), number.get()) != nullptr,
                "OS2IP that bai");
    }
    return number;
}

// I2OSP trả về đúng số byte được yêu cầu, bổ sung byte 00 ở đầu nếu cần.
// Từ chối giá trị âm hoặc giá trị không biểu diễn được trong độ dài đã cho.
Bytes i2osp(const BIGNUM* x, size_t width) {
    require(x && !BN_is_negative(x), "I2OSP khong chap nhan so am", 1);
    require(width <= 1024, "I2OSP: width vuot gioi han 1024 bytes", 1);
    if (width == 0) {
        require(BN_is_zero(x), "I2OSP: integer too large", 1);
        return {};
    }
    Bytes output(width, 0);
    require(BN_bn2binpad(x, output.data(), static_cast<int>(width)) >= 0,
            "I2OSP: integer too large", 1);
    return output;
}

Bytes fromHex(const string& hex) {
    require(hex.size() <= 2048, "OS2IP gioi han 1024 bytes", 1);
    require(hex.size() % 2 == 0, "Hex phai co so ky tu chan", 1);
    auto digit = [](char c) -> int {
        if (c >= '0' && c <= '9') return c - '0';
        if (c >= 'a' && c <= 'f') return c - 'a' + 10;
        if (c >= 'A' && c <= 'F') return c - 'A' + 10;
        throw LabError(1, "Ky tu hex khong hop le");
    };
    Bytes result;
    for (size_t i = 0; i < hex.size(); i += 2)
        result.push_back(static_cast<unsigned char>((digit(hex[i]) << 4) | digit(hex[i + 1])));
    return result;
}
string toHex(const Bytes& bytes, size_t showMax = 32) {
    ostringstream out;
    out << hex << setfill('0');
    for (size_t i = 0; i < min(bytes.size(), showMax); ++i)
        out << setw(2) << static_cast<unsigned int>(bytes[i]);
    if (bytes.size() > showMax) out << "... (" << dec << bytes.size() << " bytes)";
    return out.str();
}

// 1. Kiểm tra các trường hợp chuyển đổi C-T1 đến C-T6.
void part1() {
    cout << "\n===== PART 1: OS2IP / I2OSP =====\n";
    struct Case { const char* id; const char* bytes; size_t width; const char* expected; };
    const Case cases[] = {
        {"C-T1", "000001", 3, "000001"},
        {"C-T2", "000001", 1, "01"},
        {"C-T3", "0100",   2, "0100"},
        {"C-T4", "0100",   1, nullptr},  // Số 256 cần ít nhất hai byte để biểu diễn.
        {"C-T5", "00ff00", 3, "00ff00"},
        {"C-T6", "",       2, "0000"}
    };
    for (const auto& test : cases) {
        auto number = os2ip(fromHex(test.bytes));
        char* decimal = BN_bn2dec(number.get());
        require(decimal != nullptr, "BN_bn2dec that bai");
        cout << test.id << " | OS2IP=" << decimal << " | I2OSP(" << test.width << ")=";
        OPENSSL_free(decimal);
        bool succeeded = false;
        Bytes output;
        try {
            output = i2osp(number.get(), test.width);
            succeeded = true;
        } catch (const runtime_error&) {
            // Đây là trường hợp lỗi được mong đợi khi số nguyên vượt quá độ dài cho phép.
```

```
        }
        cout << (succeeded ? toHex(output) : "integer too large") << '\n';
        check(test.id, test.expected ? (succeeded && toHex(output) == test.expected) : !succeeded);
    }
}

// 2. RSA không đệm với bộ tham số minh họa n = 3233, e = 17, d = 2753.
// Không có cơ chế đệm nên bản mã có tính xác định và dễ bị biến đổi.
void part2() {
    cout << "\n===== PART 2: TEXTBOOK RSA (TOY EXAMPLE) =====\n";
    auto ctx = newBnContext();
    auto n = newBig(), e = newBig(), d = newBig();
    auto m = newBig(), c = newBig(), cAgain = newBig(), back = newBig();
    auto six = newBig(), seven = newBig(), encSix = newBig(), encSeven = newBig();
    auto forged = newBig();
    require(BN_set_word(n.get(), 3233) && BN_set_word(e.get(), 17) &&
            BN_set_word(d.get(), 2753) && BN_set_word(m.get(), 65) &&
            BN_set_word(six.get(), 6) && BN_set_word(seven.get(), 7), "Khoi tao khoa toy that bai");
    require(BN_mod_exp(c.get(), m.get(), e.get(), n.get(), ctx.get()) == 1, "Ma hoa toy that bai");
    require(BN_mod_exp(back.get(), c.get(), d.get(), n.get(), ctx.get()) == 1, "Giai ma toy that bai");
    cout << "Toy key: n=3233, e=17, d=2753\n";
    cout << "Enc(65)=" << BN_get_word(c.get()) << ", Dec(Enc(65))=" << BN_get_word(back.get()) << '\n';
    check("Enc(65)=2790; decrypt=65", BN_is_word(c.get(), 2790) && BN_is_word(back.get(), 65));
    require(BN_mod_exp(cAgain.get(), m.get(), e.get(), n.get(), ctx.get()) == 1, "Ma hoa lan hai loi");
    check("Determinism: Enc(65) lan 1 = lan 2", BN_cmp(c.get(), cAgain.get()) == 0);
    require(BN_mod_exp(encSix.get(), six.get(), e.get(), n.get(), ctx.get()) == 1 &&
            BN_mod_exp(encSeven.get(), seven.get(), e.get(), n.get(), ctx.get()) == 1 &&
            BN_mod_mul(forged.get(), encSix.get(), encSeven.get(), n.get(), ctx.get()) == 1 &&
            BN_mod_exp(back.get(), forged.get(), d.get(), n.get(), ctx.get()) == 1,
            "Phep thu tinh uon deo RSA loi");
    cout << "C-T9: Enc(6)=" << BN_get_word(encSix.get())
         << ", Enc(7)=" << BN_get_word(encSeven.get())
         << ", forged=(Enc(6)*Enc(7)) mod n=" << BN_get_word(forged.get())
         << ", decrypted=" << BN_get_word(back.get()) << '\n';
    check("C-T9: 824 * 2369 mod 3233 = 2557; decrypt = 42",
          BN_is_word(encSix.get(), 824) && BN_is_word(encSeven.get(), 2369)
          && BN_is_word(forged.get(), 2557) && BN_is_word(back.get(), 42));
}

// Sinh khóa RSA 2048 bit trong bộ nhớ để kiểm tra OAEP, không ghi khóa ra tệp.
RsaKey newRsaKey() {
    RsaContext ctx(EVP_PKEY_CTX_new_from_name(nullptr, "RSA", nullptr), EVP_PKEY_CTX_free);
    require(bool(ctx), "Khong tao duoc EVP_PKEY_CTX");
    require(EVP_PKEY_keygen_init(ctx.get()) > 0, "EVP_PKEY_keygen_init loi");
    require(EVP_PKEY_CTX_set_rsa_keygen_bits(ctx.get(), 2048) > 0, "Dat bits RSA loi");
    EVP_PKEY* raw = nullptr;
    require(EVP_PKEY_generate(ctx.get(), &raw) > 0 && raw != nullptr, "Sinh khoa RSA that bai");
    return RsaKey(raw, EVP_PKEY_free);
}
Big rsaParam(EVP_PKEY* key, const char* name) {
    BIGNUM* raw = nullptr;
    require(EVP_PKEY_get_bn_param(key, name, &raw) == 1 && raw != nullptr,
            "Khong lay duoc tham so RSA");
    return Big(raw, BN_clear_free);
}

// Thực hiện phép toán RSA cơ bản bằng lũy thừa modulo với BIGNUM.
Bytes rawRsa(const Bytes& x, const BIGNUM* exponent, const BIGNUM* modulus, size_t k) {
    auto value = os2ip(x);
    require(BN_cmp(value.get(), modulus) < 0, "Dai dien RSA vuot pham vi n", 1);
    auto result = newBig();
    auto ctx = newBnContext();
    require(BN_mod_exp(result.get(), value.get(), exponent, modulus, ctx.get()) == 1,
            "Phep luy thua RSA that bai");
    return i2osp(result.get(), k);
}

Bytes sha256(const Bytes& data) {
    Bytes digest(32);
    unsigned int written = 0;
    require(EVP_Digest(data.data(), data.size(), digest.data(), &written, EVP_sha256(), nullptr) == 1
            && written == 32, "SHA-256 that bai");
    return digest;
}

// MGF1 với SHA-256 theo RFC 8017: băm seed ghép với bộ đếm dài bốn byte.
Bytes mgf1(const Bytes& seed, size_t requested) {
    Bytes out;
```

```
    out.reserve(requested);
    for (uint32_t counter = 0; out.size() < requested; ++counter) {
        Bytes input = seed;
        input.push_back(static_cast<unsigned char>((counter >> 24) & 0xff));
        input.push_back(static_cast<unsigned char>((counter >> 16) & 0xff));
        input.push_back(static_cast<unsigned char>((counter >> 8) & 0xff));
        input.push_back(static_cast<unsigned char>(counter & 0xff));
        Bytes block = sha256(input);
        out.insert(out.end(), block.begin(), block.end());
    }
    out.resize(requested);
    return out;
}

// EME-OAEP: EM = 0x00 || maskedSeed || maskedDB; sử dụng nhãn rỗng.
Bytes oaepEncode(const Bytes& message, size_t k) {
    constexpr size_t hLen = 32;
    require(k >= 2 * hLen + 2, "OAEP: modulus too small");
    require(message.size() <= k - 2 * hLen - 2,
            "OAEP: message too long", 1);
    Bytes db = sha256({});
    db.insert(db.end(), k - message.size() - 2 * hLen - 2, 0x00);
    db.push_back(0x01);
    db.insert(db.end(), message.begin(), message.end());
    Bytes seed(hLen);
    require(RAND_bytes(seed.data(), static_cast<int>(hLen)) == 1, "RAND_bytes that bai");
    Bytes dbMask = mgf1(seed, db.size());
    for (size_t i = 0; i < db.size(); ++i) db[i] ^= dbMask[i];
    Bytes seedMask = mgf1(db, hLen);
    for (size_t i = 0; i < hLen; ++i) seed[i] ^= seedMask[i];
    Bytes em = {0x00};
    em.insert(em.end(), seed.begin(), seed.end());
    em.insert(em.end(), db.begin(), db.end());
    return em;
}

// Hàm giải mã OAEP tự cài đặt chỉ phục vụ thử nghiệm, không bảo đảm thời gian thực thi hằng.
// Khi cần giải mã an toàn, sử dụng EVP_PKEY_decrypt của OpenSSL.
bool oaepDecode(const Bytes& em, Bytes& message) {
    constexpr size_t hLen = 32;
    if (em.size() < 2 * hLen + 2) return false;
    Bytes seed(em.begin() + 1, em.begin() + 1 + hLen);
    Bytes db(em.begin() + 1 + hLen, em.end());
    Bytes seedMask = mgf1(db, hLen);
    for (size_t i = 0; i < hLen; ++i) seed[i] ^= seedMask[i];
    Bytes dbMask = mgf1(seed, db.size());
    for (size_t i = 0; i < db.size(); ++i) db[i] ^= dbMask[i];
    Bytes expectedHash = sha256({});
    if (em[0] != 0 || CRYPTO_memcmp(db.data(), expectedHash.data(), hLen) != 0) return false;
    size_t index = hLen;
    while (index < db.size() && db[index] == 0) ++index;
    if (index == db.size() || db[index] != 0x01) return false;
    message.assign(db.begin() + index + 1, db.end());
    return true;
}

RsaContext newOaepContext(EVP_PKEY* key, bool encrypt) {
    RsaContext ctx(EVP_PKEY_CTX_new_from_pkey(nullptr, key, nullptr), EVP_PKEY_CTX_free);
    require(bool(ctx), "Khong tao duoc OAEP context");
    require((encrypt ? EVP_PKEY_encrypt_init(ctx.get()) : EVP_PKEY_decrypt_init(ctx.get())) > 0,
            "Khoi tao EVP OAEP that bai");
    require(EVP_PKEY_CTX_set_rsa_padding(ctx.get(), RSA_PKCS1_OAEP_PADDING) > 0 &&
            EVP_PKEY_CTX_set_rsa_oaep_md(ctx.get(), EVP_sha256()) > 0 &&
            EVP_PKEY_CTX_set_rsa_mgf1_md(ctx.get(), EVP_sha256()) > 0,
            "Thiet lap OAEP SHA-256/MGF1 loi");
    return ctx;
}
Bytes opensslEncrypt(EVP_PKEY* key, const Bytes& plain) {
    const int keySize = EVP_PKEY_get_size(key);
    require(keySize >= 66, "OAEP: invalid key length", 3);
    require(plain.size() <= static_cast<size_t>(keySize - 66),
            "OAEP: message too long", 1);
    auto ctx = newOaepContext(key, true);
    size_t length = 0;
    require(EVP_PKEY_encrypt(ctx.get(), nullptr, &length, plain.data(), plain.size()) > 0,
            "Khong tinh duoc do dai ciphertext");
    Bytes result(length);
    require(EVP_PKEY_encrypt(ctx.get(), result.data(), &length, plain.data(), plain.size()) > 0,
            "EVP_PKEY_encrypt that bai");
```

```
    result.resize(length);
    return result;
}
// C-T11 gọi trực tiếp EVP để kiểm tra việc từ chối thông điệp dài 319 byte.
// Phép kiểm tra này độc lập với bước kiểm tra độ dài trong opensslEncrypt.
bool evpRejectsOversizedMessage(EVP_PKEY* key, const Bytes& message) {
    auto ctx = newOaepContext(key, true);
    size_t outputLength = static_cast<size_t>(EVP_PKEY_get_size(key));
    Bytes ciphertext(outputLength);
    int ok = EVP_PKEY_encrypt(ctx.get(), ciphertext.data(), &outputLength,
                              message.data(), message.size());
    if (ok <= 0) ERR_clear_error();
    return ok <= 0;
}

bool opensslDecrypt(EVP_PKEY* key, const Bytes& cipher, Bytes& plain) {
    auto ctx = newOaepContext(key, false);
    size_t length = 0;
    if (EVP_PKEY_decrypt(ctx.get(), nullptr, &length, cipher.data(), cipher.size()) <= 0) {
        ERR_clear_error();
        return false;
    }
    Bytes output(length);
    if (EVP_PKEY_decrypt(ctx.get(), output.data(), &length, cipher.data(), cipher.size()) <= 0) {
        ERR_clear_error();
        return false;
    }
    output.resize(length);
    plain = move(output);
    return true;
}

void part3() {
    cout << "\n===== PART 3: RSA-OAEP SHA-256 / MGF1 =====\n";
    auto key = newRsaKey();
    auto n = rsaParam(key.get(), OSSL_PKEY_PARAM_RSA_N);
    auto e = rsaParam(key.get(), OSSL_PKEY_PARAM_RSA_E);
    auto d = rsaParam(key.get(), OSSL_PKEY_PARAM_RSA_D);
    const size_t k = static_cast<size_t>(EVP_PKEY_get_size(key.get()));
    const string text = "attack at dawn";
    Bytes message(text.begin(), text.end());
    cout << "RSA=" << EVP_PKEY_get_bits(key.get()) << " bits, k=" << k
         << " bytes, message=\"" << text << "\"\n";

    // Chiều 1: tự mã hóa OAEP, thực hiện RSA cơ bản rồi giải mã bằng OpenSSL.
    Bytes ourEncoded = oaepEncode(message, k);
    Bytes ourCipher = rawRsa(ourEncoded, e.get(), n.get(), k);
    Bytes recovered;
    cout << "Manual OAEP encoded (hex): " << toHex(ourEncoded, 16) << '\n';
    check("Custom OAEP encrypt -> OpenSSL decrypt",
          opensslDecrypt(key.get(), ourCipher, recovered) && recovered == message);

    // Chiều 2: mã hóa bằng OpenSSL, thực hiện RSA với d rồi tự giải mã OAEP.
    Bytes officialCipher = opensslEncrypt(key.get(), message);
    Bytes officialEncoded = rawRsa(officialCipher, d.get(), n.get(), k);
    Bytes decoded;
    cout << "OpenSSL OAEP encoded (hex): " << toHex(officialEncoded, 16) << '\n';
    check("OpenSSL encrypt -> Custom OAEP decode",
          oaepDecode(officialEncoded, decoded) && decoded == message);

    // OAEP sử dụng seed ngẫu nhiên nên hai lần mã hóa cùng thông điệp cho bản mã khác nhau.
    Bytes secondCipher = opensslEncrypt(key.get(), message);
    check("OAEP is randomized (two ciphertexts differ)", officialCipher != secondCipher);

    // RSA 2048 bit với SHA-256: độ dài thông điệp tối đa là 256 - 64 - 2 = 190 byte.
    const size_t maxLength = k - 2 * 32 - 2;
    Bytes maxMessage(maxLength, 'A');
    Bytes maxCipher = opensslEncrypt(key.get(), maxMessage);
    check("OAEP maximum 190-byte message round-trip",
          maxLength == 190 && opensslDecrypt(key.get(), maxCipher, recovered) && recovered == maxMessage);
    bool tooLongRejected = false;
    try { (void)oaepEncode(Bytes(maxLength + 1, 'B'), k); }
    catch (const runtime_error&) { tooLongRejected = true; }
    check("OAEP rejects 191-byte message", tooLongRejected);

    // Thay đổi một bit trong bản mã phải làm cho quá trình giải mã OAEP thất bại.
```

```
    Bytes tampered = officialCipher;
    tampered[10] ^= 0x01;
    check("OAEP rejects tampered ciphertext (1 bit flipped)",
          !opensslDecrypt(key.get(), tampered, recovered));


    // Kiểm tra phép biến đổi nhân: nhân bản mã OAEP với giá trị mã hóa của số 2.
    // Cấu trúc đệm OAEP khiến bản mã bị biến đổi không vượt qua bước kiểm tra hợp lệ.
    auto multiplier = newBig();
    auto cipherInteger = os2ip(officialCipher);
    auto encryptedTwo = newBig();
    auto changedInteger = newBig();
    auto bnctx = newBnContext();
    require(BN_set_word(multiplier.get(), 2) == 1 &&
            BN_mod_exp(encryptedTwo.get(), multiplier.get(), e.get(), n.get(), bnctx.get()) == 1 &&
            BN_mod_mul(changedInteger.get(), cipherInteger.get(), encryptedTwo.get(), n.get(), bnctx.get()) ==
1,
            "Khong tao duoc ban ma RSA da bien doi");
    Bytes changedCipher = i2osp(changedInteger.get(), k);
    check("OAEP rejects multiplicatively modified ciphertext",
          !opensslDecrypt(key.get(), changedCipher, recovered));
}



// C-T10: sinh khóa RSA học thuật từ 512 đến 1024 bit.
// Không đọc khóa từ tệp và không sử dụng khóa học thuật cho lệnh OAEP an toàn.
void educationalRawRsa(int bits) {
    require(bits >= 512 && bits <= 1024, "raw-rsa: --bits phai trong [512, 1024]", 1);
    cout << "\nEDUCATIONAL - INSECURE: TEXTBOOK RSA, NEVER USE FOR REAL DATA\n";
    cout << "===== C-T10: RAW RSA (" << bits << " bits) =====\n";
    // Áp dụng quy trình sinh tham số RSA học thuật đã sử dụng ở Task B.
    // Sinh p, q bằng BIGNUM rồi tính n, phi, lambda và d; không dùng EVP_RSA_gen.
    auto ctxGen = newBnContext();
    auto p = newBig(), q = newBig(), pm1 = newBig(), qm1 = newBig();
    auto n = newBig(), e = newBig(), phi = newBig(), lambda = newBig();
    auto g = newBig();
    require(BN_set_word(e.get(), 65537) == 1, "Khong thiet lap duoc e=65537");
    auto drawPrime = [&](BIGNUM* prime, BIGNUM* primeMinusOne) {
        do {
            require(BN_generate_prime_ex2(prime, bits/2, 0, nullptr, nullptr,
                                          nullptr, ctxGen.get()) == 1,
                    "Sinh so nguyen to RSA hoc thuat that bai");
            require(BN_copy(primeMinusOne, prime) != nullptr &&
                    BN_sub_word(primeMinusOne, 1) == 1 &&
                    BN_gcd(g.get(), e.get(), primeMinusOne, ctxGen.get()) == 1,
                    "Kiem tra gcd(e, prime-1) that bai");
        } while (!BN_is_one(g.get()));
    };
    drawPrime(p.get(), pm1.get());
    do { drawPrime(q.get(), qm1.get()); } while (BN_cmp(p.get(), q.get()) == 0);
    require(BN_mul(n.get(), p.get(), q.get(), ctxGen.get()) == 1 &&
            BN_mul(phi.get(), pm1.get(), qm1.get(), ctxGen.get()) == 1 &&
            BN_gcd(g.get(), pm1.get(), qm1.get(), ctxGen.get()) == 1 &&
            BN_div(lambda.get(), nullptr, phi.get(), g.get(), ctxGen.get()) == 1,
            "Tinh n, phi, lambda cua RSA that bai");
    Big d(BN_mod_inverse(nullptr, e.get(), lambda.get(), ctxGen.get()), BN_clear_free);
    require(bool(d), "Tinh d=e^-1 mod lambda that bai");
    const size_t k = static_cast<size_t>(BN_num_bytes(n.get()));
    cout << "Generated with BN_generate_prime_ex2 + Task B formulas: "
         << BN_num_bits(n.get()) << "-bit modulus, e=65537\n";
    vector<Bytes> samples = {
        fromHex("000001"), fromHex("0100"), fromHex("00ff00"), Bytes{},
        fromHex("01"), fromHex("000000")
    };
    Bytes random32(32);
    require(RAND_bytes(random32.data(), 32) == 1, "RAND_bytes raw RSA that bai");
    random32[0] = 0;
    samples.push_back(random32);
    bool allRoundTrips = true;
    for (size_t i = 0; i < samples.size(); ++i) {
        const auto& x = samples[i];
        auto xi = os2ip(x);
        if (BN_cmp(xi.get(), n.get()) >= 0) { allRoundTrips = false; continue; }
        Bytes ciphertext = rawRsa(x, e.get(), n.get(), k);
        Bytes recoveredFull = rawRsa(ciphertext, d.get(), n.get(), k);
        auto recoveredInteger = os2ip(recoveredFull);
        Bytes recovered = i2osp(recoveredInteger.get(), x.size());
        bool ok = recovered == x;
        allRoundTrips &= ok;
        cout << "Round-trip X=" << toHex(x, 32) << " -> " << toHex(recovered, 32)
```

```
             << (ok ? " [OK]" : " [FAIL]") << '\n';
    }
    check("C-T10 all fixed-width RSA round-trips", allRoundTrips);
    // Từ chối giá trị x = n vì RSA chỉ chấp nhận 0 <= x < n.
    bool rejectedRange = false;
    try { (void)rawRsa(i2osp(n.get(), k), e.get(), n.get(), k); }
    catch (const LabError& error) { rejectedRange = error.code == 1; }
    check("C-T10 range check rejects x >= n", rejectedRange);
    Bytes m = fromHex("41");
    Bytes c1 = rawRsa(m, e.get(), n.get(), k);
    Bytes c2 = rawRsa(m, e.get(), n.get(), k);
    check("C-T10 determinism (same plaintext -> same ciphertext)", c1 == c2);
    Bytes enc6 = rawRsa(fromHex("06"), e.get(), n.get(), k);
    Bytes enc7 = rawRsa(fromHex("07"), e.get(), n.get(), k);
    auto a = os2ip(enc6), b = os2ip(enc7), forged = newBig();
    auto ctx = newBnContext();
    require(BN_mod_mul(forged.get(), a.get(), b.get(), n.get(), ctx.get()) == 1,
            "C-T10 tao ciphertext lien he that bai");
    Bytes decrypted = rawRsa(i2osp(forged.get(), k), d.get(), n.get(), k);
    auto plaintext = os2ip(decrypted);
    check("C-T10 malleability: decrypt(Enc(6)*Enc(7))=42", BN_is_word(plaintext.get(),42));
}

// Phân tích tham số dòng lệnh, kiểm tra định dạng và giới hạn độ dài đầu vào.
size_t parseSize(const string& s, size_t maximum) {
    require(!s.empty() && s.find_first_not_of("0123456789") == string::npos,
            "Tham so phai la so nguyen khong am", 1);
    size_t result = 0;
    for (char c : s) {
        size_t digit = static_cast<size_t>(c-'0');
        require(digit <= maximum && result <= (maximum-digit)/10, "Tham so qua lon", 1);
        result = result*10+digit;
    }
    require(result <= maximum, "Tham so qua lon", 1);
    return result;
}
string bnDecimal(const BIGNUM* x) {
    char* txt = BN_bn2dec(x);
    require(txt != nullptr, "BN_bn2dec failed");
    string result(txt);
    OPENSSL_free(txt);
    return result;
}

// Đọc khóa PEM hoặc DER theo các định dạng đã sử dụng ở Task B.
// Khóa công khai sử dụng SPKI, khóa riêng sử dụng PKCS#8.
// Phần đọc khóa được tách riêng để có thể sử dụng lại khi tích hợp dự án.
// Bộ đệm chứa khóa riêng được xóa khi ra khỏi phạm vi, kể cả khi có ngoại lệ.
// Tránh sao chép khóa PEM riêng sang std::string để hạn chế lưu dữ liệu nhạy cảm.
struct FileKeyBytes {
    Bytes data;
    bool secret;
    explicit FileKeyBytes(bool isPrivate) : secret(isPrivate) {}
    ~FileKeyBytes() {
        if (secret && !data.empty()) OPENSSL_cleanse(data.data(), data.size());
    }
    FileKeyBytes(const FileKeyBytes&) = delete;
    FileKeyBytes& operator=(const FileKeyBytes&) = delete;
};

RsaKey readKey(const string& name, bool privateKey) {
    constexpr size_t MAX_KEY_FILE_BYTES = 1024 * 1024;
    ifstream stream(name, ios::binary | ios::ate);
    require(bool(stream), "Khong mo duoc file khoa: " + name, 2);
    const auto end = stream.tellg();
    require(end >= 0, "Khong xac dinh duoc kich thuoc file khoa", 2);
    const auto length = static_cast<uint64_t>(end);
    require(length > 0 && length <= MAX_KEY_FILE_BYTES,
            "Kich thuoc file khoa khong hop le", 3);
    stream.seekg(0, ios::beg);
    require(bool(stream), "Khong doc duoc file khoa: " + name, 2);
    FileKeyBytes input(privateKey);
    input.data.resize(static_cast<size_t>(length));
    require(bool(stream.read(reinterpret_cast<char*>(input.data.data()),
                             static_cast<streamsize>(length))),
            "Khong doc duoc day du file khoa: " + name, 2);

    const Bytes& bytes = input.data;
```

```
    string_view data(reinterpret_cast<const char*>(bytes.data()), bytes.size());
    const bool pem = data.rfind("-----BEGIN ", 0) == 0;
    EVP_PKEY* raw = nullptr;
    if (pem) {
        // Chỉ chấp nhận PKCS#8 cho khóa riêng và SPKI cho khóa công khai; từ chối dữ liệu thừa.
        const string_view beginLabel = privateKey ? "-----BEGIN PRIVATE KEY-----" :
                                                    "-----BEGIN PUBLIC KEY-----";
        const string_view endLabel = privateKey ? "-----END PRIVATE KEY-----" :
                                                  "-----END PUBLIC KEY-----";
        require(data.rfind(beginLabel, 0) == 0, "PEM khong dung SPKI/PKCS#8", 3);
        const auto pos = data.find(endLabel, beginLabel.size());
        require(pos != string_view::npos, "Khong tim thay ket thuc PEM", 3);
        require(pos > 0 && data[pos - 1] == '\n', "Thieu dong ket thuc PEM", 3);
        const auto tail = data.substr(pos + endLabel.size());
        require(tail.find_first_not_of(" \r\n\t") == string_view::npos,
                "PEM co du lieu thua", 3);
        unique_ptr<BIO, decltype(&BIO_free)> bio(
            BIO_new_mem_buf(bytes.data(), static_cast<int>(bytes.size())), BIO_free);
        require(bool(bio), "BIO_new_mem_buf loi");
        raw = privateKey ? PEM_read_bio_PrivateKey(bio.get(), nullptr, nullptr, nullptr)
                         : PEM_read_bio_PUBKEY(bio.get(), nullptr, nullptr, nullptr);
    } else {
        const unsigned char* cursor = bytes.data();
        if (privateKey) {
            unique_ptr<PKCS8_PRIV_KEY_INFO, decltype(&PKCS8_PRIV_KEY_INFO_free)> info(
                d2i_PKCS8_PRIV_KEY_INFO(nullptr, &cursor, static_cast<long>(bytes.size())),
                PKCS8_PRIV_KEY_INFO_free);
            require(bool(info), "DER phai la PKCS#8 private key", 3);
            require(cursor == bytes.data() + bytes.size(), "DER co du lieu thua", 3);
            raw = EVP_PKCS82PKEY(info.get());
        } else {
            raw = d2i_PUBKEY(nullptr, &cursor, static_cast<long>(bytes.size()));
            RsaKey key(raw, EVP_PKEY_free);
            require(bool(key) && cursor == bytes.data() + bytes.size(),
                    "DER phai la SPKI public key va khong du byte", 3);
            require(EVP_PKEY_is_a(key.get(), "RSA") == 1 && EVP_PKEY_get_bits(key.get()) >= 2048,
                    "Phai la RSA >=2048 bit", 3);
            return key;
        }
    }
    RsaKey key(raw, EVP_PKEY_free);
    require(bool(key), "Khong doc duoc khoa PEM/DER", 3);
    require(EVP_PKEY_is_a(key.get(), "RSA") == 1 && EVP_PKEY_get_bits(key.get()) >= 2048,
            "Phai la RSA >=2048 bit", 3);
    return key;
}


void oaepDemoFiles(const string& pubName, const string& privName) {
    auto publicKey = readKey(pubName, false);
    auto privateKey = readKey(privName, true);
    auto nPub = rsaParam(publicKey.get(), OSSL_PKEY_PARAM_RSA_N);
    auto nPriv = rsaParam(privateKey.get(), OSSL_PKEY_PARAM_RSA_N);
    auto ePub = rsaParam(publicKey.get(), OSSL_PKEY_PARAM_RSA_E);
    auto ePriv = rsaParam(privateKey.get(), OSSL_PKEY_PARAM_RSA_E);
    require(BN_cmp(nPub.get(), nPriv.get()) == 0 && BN_cmp(ePub.get(), ePriv.get()) == 0,
            "Khoa cong khai/bi mat khong khop", 4);
    const size_t k = static_cast<size_t>(EVP_PKEY_get_size(publicKey.get()));
    const size_t maxLen = k - 2*32 - 2;
    Bytes msg({'a','t','t','a','c','k',' ','a','t',' ','d','a','w','n'});
    cout << "\n===== OAEP DEMO WITH SAVED KEYS (C-F4) =====\n"
         << "Key bits=" << EVP_PKEY_get_bits(publicKey.get()) << ", k=" << k
         << ", maximum OAEP message=" << maxLen << " bytes\n";
    if (EVP_PKEY_get_bits(publicKey.get()) == 3072)
        cout << "C-T11 RSA-3072 vector: 318 accepted / 319 rejected\n";
    else
        cout << "C-F4 test for this key size; C-T11 separately uses RSA-3072\n";
    auto c1 = opensslEncrypt(publicKey.get(), msg);
    auto c2 = opensslEncrypt(publicKey.get(), msg);
    Bytes p1, p2;
    check("C-T11 twice encrypted -> different ciphertexts", c1 != c2);
    check("C-T11 both decrypt to original", opensslDecrypt(privateKey.get(), c1, p1)
          && opensslDecrypt(privateKey.get(), c2, p2) && p1 == msg && p2 == msg);
    Bytes big(maxLen, 'A'), back;
    auto bigCipher = opensslEncrypt(publicKey.get(), big);
    check("C-T11 max message accepted", opensslDecrypt(privateKey.get(), bigCipher, back) && back == big);
    bool rejected = false;
    rejected = evpRejectsOversizedMessage(publicKey.get(), Bytes(maxLen+1, 'B'));
    check("C-T11 max+1 message rejected", rejected);
    c1[10] ^= 0x01;
    check("C-T11 one bit flipped is rejected", !opensslDecrypt(privateKey.get(),c1,back));
}
```

```
void part1ExtraC7() {
    cout << "\n===== C-T7: RANDOM 32-BYTE LEADING ZERO =====\n";
    Bytes data(32);
    require(RAND_bytes(data.data(), static_cast<int>(data.size())) == 1,
            "RAND_bytes that bai");
    data.front() = 0x00;
    auto value = os2ip(data);
    auto recovered = i2osp(value.get(), data.size());
    cout << "Input:     " << toHex(data, 32) << '\n';
    cout << "Recovered: " << toHex(recovered, 32) << '\n';
    check("C-T7 exact 32-byte round-trip including leading zero", recovered == data);
}


// Demo OAEP sử dụng RSA 2048 bit; C-T11 kiểm tra riêng với RSA 3072 bit.
RsaKey newRsaKey3072() {
    RsaContext ctx(EVP_PKEY_CTX_new_from_name(nullptr, "RSA", nullptr), EVP_PKEY_CTX_free);
    require(bool(ctx), "Khong tao duoc RSA key context");
    require(EVP_PKEY_keygen_init(ctx.get()) > 0, "RSA keygen init that bai");
    require(EVP_PKEY_CTX_set_rsa_keygen_bits(ctx.get(), 3072) > 0, "Khong dat duoc 3072 bits");
    EVP_PKEY* raw = nullptr;
    require(EVP_PKEY_generate(ctx.get(), &raw) > 0 && raw,
            "Khong sinh duoc RSA 3072 bits");
    return RsaKey(raw, EVP_PKEY_free);
}
void part3ExtraC11() {
    cout << "\n===== C-T11: RSA-3072 OAEP-SHA256/MGF1 =====\n";
    auto key = newRsaKey3072();
    const size_t k = static_cast<size_t>(EVP_PKEY_get_size(key.get()));
    const size_t maxLen = k - 2*32 - 2;
    cout << "RSA=" << EVP_PKEY_get_bits(key.get()) << " bits, k=" << k
         << " bytes, maximum=" << maxLen << " bytes\n";
    Bytes message = {'a','t','t','a','c','k',' ','a','t',' ','d','a','w','n'};
    Bytes c1 = openssslEncrypt(key.get(), message), c2 = openssslEncrypt(key.get(), message);
    Bytes r1, r2;
    check("C-T11 randomized: two ciphertexts differ", c1 != c2);
    check("C-T11 both decrypt to original", openssslDecrypt(key.get(), c1, r1)
          && openssslDecrypt(key.get(), c2, r2) && r1 == message && r2 == message);
    Bytes maxMessage(maxLen, 'A');
    Bytes maxCipher = openssslEncrypt(key.get(), maxMessage), restored;
    check("C-T11 318-byte message accepted", maxLen == 318
          && openssslDecrypt(key.get(), maxCipher, restored) && restored == maxMessage);
    bool oversizedRejected = false;
    oversizedRejected = evpRejectsOversizedMessage(key.get(), Bytes(maxLen + 1, 'B'));
    check("C-T11 319-byte message rejected by EVP", oversizedRejected);
    c1[10] ^= 0x01;
    check("C-T11 flipped ciphertext bit rejected", !openssslDecrypt(key.get(), c1, restored));
}


void supplementalTests() {
    part1ExtraC7();
    educationalRawRsa(512);
    part3ExtraC11();
}


void fullTaskC() {
    part1();   // Chuyển đổi OS2IP/I2OSP, C-T1 đến C-T6.
    part2();   // Thử nghiệm các đặc tính của RSA không đệm.
    part3();   // Thử nghiệm OAEP với RSA 2048 bit và giới hạn 190/191 byte.
    supplementalTests();
}



void usage(const string& command = "") {
    if (command == "os2ip") {
        cout << "Usage: task_c_oaep.exe os2ip --hex <even-hex-string> (max 1024 bytes)\n";
    } else if (command == "i2osp") {
        cout << "Usage: task_c_oaep.exe i2osp --int <nonnegative-decimal> --len <0..1024>\n";
    } else if (command == "raw-rsa") {
        cout << "Usage: task_c_oaep.exe raw-rsa --bits <512..1024>\n"
             << "EDUCATIONAL - INSECURE: not for real data\n";
    } else if (command == "oaep-demo") {
        cout << "Usage: task_c_oaep.exe oaep-demo --pub <public.pem/der> --priv <private.pem/der>\n"
             << "C-T11 requires an RSA-3072 key pair; RSA-2048 is also allowed for C-F4.\n";
    } else {
        cout << "Usage:\n"
             << "  task_c_oaep.exe [--all | --all-full | --help]\n"
```

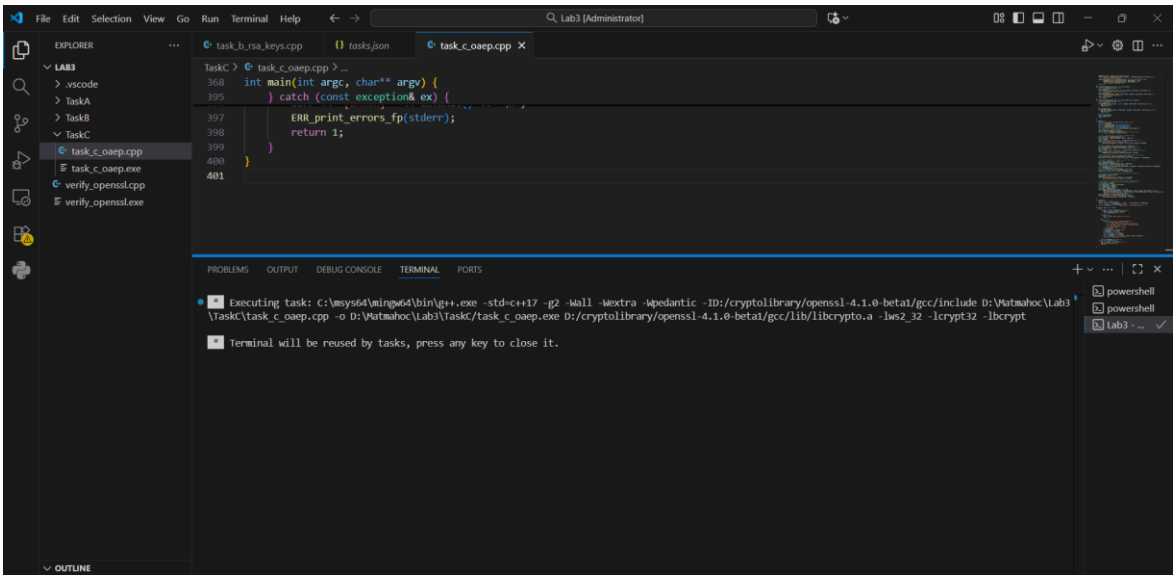
```
             << "  task_c_oaep.exe os2ip --hex <even-hex-string>\n"
             << "  task_c_oaep.exe i2osp --int <nonnegative-decimal> --len <0..1024>\n"
             << "  task_c_oaep.exe raw-rsa --bits <512..1024>  [EDUCATIONAL - INSECURE]\n"
             << "  task_c_oaep.exe oaep-demo --pub <public.pem/der> --priv <private.pem/der>\n"
             << "  task_c_oaep.exe <subcommand> --help\n";
    }
}
void summary() {
    cout << "\n===== SUMMARY =====\n";
    cout << "Checks: " << totalChecks << ", passed: " << (totalChecks - failedChecks)
         << ", failed: " << failedChecks << '\n';
    cout << (failedChecks == 0 ? "ALL CHECKS PASSED" : "SOME CHECKS FAILED") << '\n';
}
int entry(int argc, char** argv) {
    try {
        // Tùy chọn --all chạy các phép thử của phần thực hành trên lớp với RSA 2048 bit.
        if (argc == 2 && string(argv[1]) == "--all") {
            part1(); part2(); part3(); summary();
            return failedChecks == 0 ? 0 : 5;
        }
        if (argc == 2 && string(argv[1]) == "--all-full") {
            fullTaskC(); summary();
            return failedChecks == 0 ? 0 : 5;
        }
        if (argc == 2 && (string(argv[1]) == "--help" || string(argv[1]) == "help")) {
            usage(); return 0;
        }
        if (argc == 3 && string(argv[2]) == "--help") {
            string command(argv[1]);
            if (command == "os2ip" || command == "i2osp" ||
                command == "raw-rsa" || command == "oaep-demo") {
                usage(command); return 0;
            }
            usage(); return 1;
        }
        if (argc > 1) {
            vector<string> args(argv+1, argv+argc);
            if (args.size() == 3 && args[0] == "os2ip" && args[1] == "--hex") {
                cout << bnDecimal(os2ip(fromHex(args[2])).get()) << '\n';
                return 0;
            }
            if (args.size() == 5 && args[0] == "i2osp" && args[1] == "--int"
                && args[3] == "--len") {
                require(!args[2].empty() && args[2].size() <= 10000 &&
                        args[2].find_first_not_of("0123456789") == string::npos,
                        "--int phai la so nguyen khong am (1..10000 chu so)", 1);
                BIGNUM* number = nullptr;
                int consumed = BN_dec2bn(&number, args[2].c_str());
                Big value(number, BN_clear_free);
                require(value && consumed == static_cast<int>(args[2].size()),
                        "Gia tri decimal khong hop le", 1);
                cout << toHex(i2osp(value.get(), parseSize(args[4], 1024)), 1024) << '\n';
                return 0;
            }
            if (args.size() == 3 && args[0] == "raw-rsa" && args[1] == "--bits") {
                int bits = static_cast<int>(parseSize(args[2], 1024));
                educationalRawRsa(bits); summary();
                return failedChecks == 0 ? 0 : 5;
            }
            if (args.size() == 5 && args[0] == "oaep-demo" && args[1] == "--pub"
                && args[3] == "--priv") {
                oaepDemoFiles(args[2],args[4]); summary();
                return failedChecks == 0 ? 0 : 5;
            }
            usage(); return 1;
        }
        while (true) {
            cout << "\n===== TASK C - RSA/OAEP LAB =====\n"
                 << "1. OS2IP / I2OSP (C-T1..C-T6)\n"
                 << "2. Textbook RSA: determinism and malleability\n"
                 << "3. RSA-OAEP: custom vs OpenSSL, tampering tests\n"
                 << "4. Run ALL tests\n"
                 << "5. Run EXTRA tests (C-T7, C-T10, C-T11)\n"
                 << "0. Exit\nChoice: " << flush;
            string selected;
            if (!getline(cin, selected)) break;
            if (selected == "0") break;
            if (selected == "1") part1();
            else if (selected == "2") part2();
            else if (selected == "3") part3();
            else if (selected == "4") { part1(); part2(); part3(); summary(); }
```

```
            else if (selected == "5") { supplementalTests(); summary(); }
            else cout << "Please choose 0..5.\n";
        }
        return failedChecks == 0 ? 0 : 5;
    } catch (const LabError& error) {
        cerr << "[ERROR] " << error.what() << '\n';
        // Không hiển thị chi tiết lỗi đệm OAEP để tránh tiết lộ thông tin nội bộ.
        ERR_clear_error();
        return error.code;
    } catch (const exception& ex) {
        cerr << "[ERROR] Internal failure: " << ex.what() << '\n';
        ERR_clear_error();
        return 5;
    }
}

} // namespace task_c
```

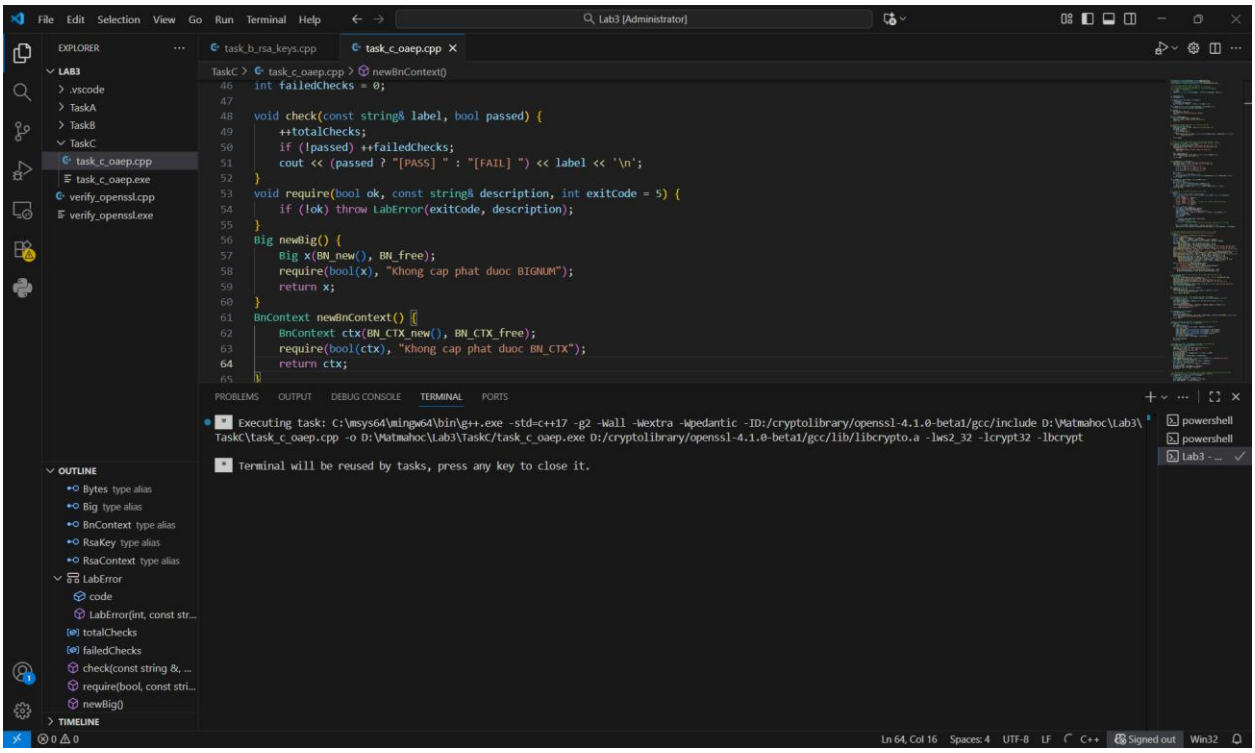
2. Hình ảnh thực hiện và kết quả kiểm thử

Biên dịch Task C

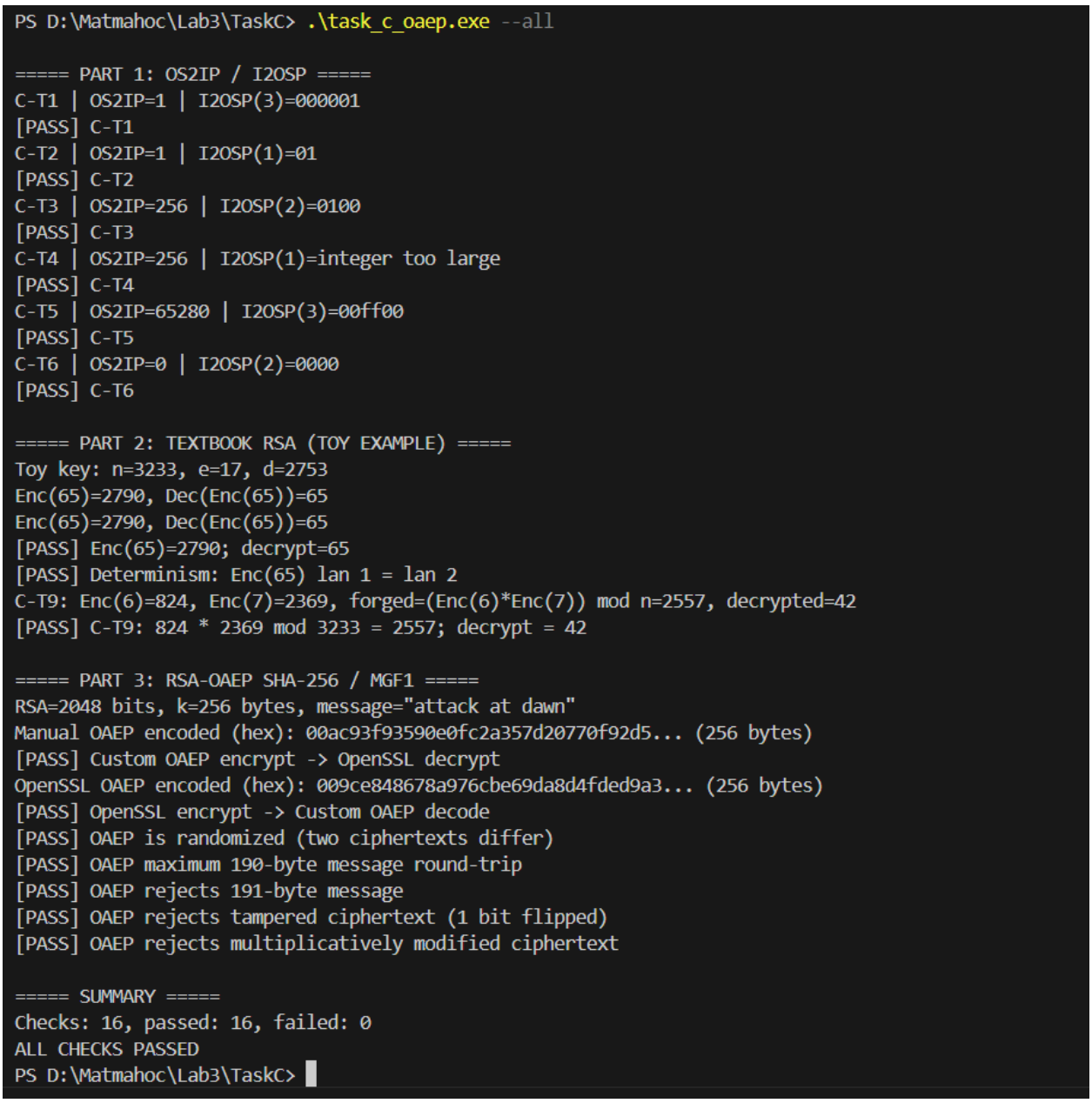


Hình 38. Biên dịch chương trình Task C bằng GCC MinGW64, sử dụng thư viện OpenSSL 4.1.0-beta1 trên Windows.

Biên dịch và kiểm thử bổ sung



Hình 39. Biên dịch mã nguồn Task C bằng GCC MinGW64 với chuẩn C++17 và thư viện OpenSSL 4.1.0-beta1 trên Windows.



Hình 40. Kết quả thực hiện ba phần của Task C gồm chuyển đổi OS2IP/I2OSP, kiểm tra các tính chất của Textbook RSA và thử nghiệm RSA-OAEP với khóa 2048 bit. Tất cả 16 kiểm thử đều thành công.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Matmahoc\Lab3\TaskC> .\task_c_oaep.exe --all-full

===== PART 1: OS2IP / I2OSP =====
C-T1 | OS2IP=1 | I2OSP(3)=000001
[PASS] C-T1
C-T2 | OS2IP=1 | I2OSP(1)=01
[PASS] C-T2
C-T3 | OS2IP=256 | I2OSP(2)=0100
[PASS] C-T3
C-T4 | OS2IP=256 | I2OSP(1)=integer too large
[PASS] C-T4
C-T5 | OS2IP=65280 | I2OSP(3)=00ff00
[PASS] C-T5
C-T6 | OS2IP=0 | I2OSP(2)=0000
[PASS] C-T6

===== PART 2: TEXTBOOK RSA (TOY EXAMPLE) =====
Toy key: n=3233, e=17, d=2753
Enc(65)=2790, Dec(Enc(65))=65
[PASS] Enc(65)=2790; decrypt=65
[PASS] Determinism: Enc(65) lan 1 = lan 2
C-T9: Enc(6)=824, Enc(7)=2369, forged=(Enc(6)*Enc(7)) mod n=2557, decrypted=42
[PASS] C-T9: 824 * 2369 mod 3233 = 2557; decrypt = 42

===== PART 3: RSA-OAEP SHA-256 / MGF1 =====
RSA=2048 bits, k=256 bytes, message="attack at dawn"
Manual OAEP encoded (hex): 0057f3ca4209488793f98d712be75b33... (256 bytes)
[PASS] Custom OAEP encrypt -> OpenSSL decrypt
OpenSSL OAEP encoded (hex): 004ab7bedc50ec59ee2c4bb6ca42603b... (256 bytes)
[PASS] OpenSSL encrypt -> Custom OAEP decode
[PASS] OAEP is randomized (two ciphertexts differ)
[PASS] OAEP maximum 190-byte message round-trip
[PASS] OAEP rejects 191-byte message
[PASS] OAEP rejects tampered ciphertext (1 bit flipped)
[PASS] OAEP rejects multiplicatively modified ciphertext
```

Hình 41. Kết quả kiểm thử chuyển đổi OS2IP/I2OSP, các tính chất của Textbook RSA và mã hóa RSA-OAEP với khóa 2048 bit.

```
[PASS] OAEP rejects tampered ciphertext (1 bit flipped)
[PASS] OAEP rejects multiplicatively modified ciphertext

===== C-T7: RANDOM 32-BYTE LEADING ZERO =====
Input:     00606999b3e14e39f3e38cf56031c003e54b4250021631774cd29c6840fc73f6
Recovered: 00606999b3e14e39f3e38cf56031c003e54b4250021631774cd29c6840fc73f6
[PASS] C-T7 exact 32-byte round-trip including leading zero

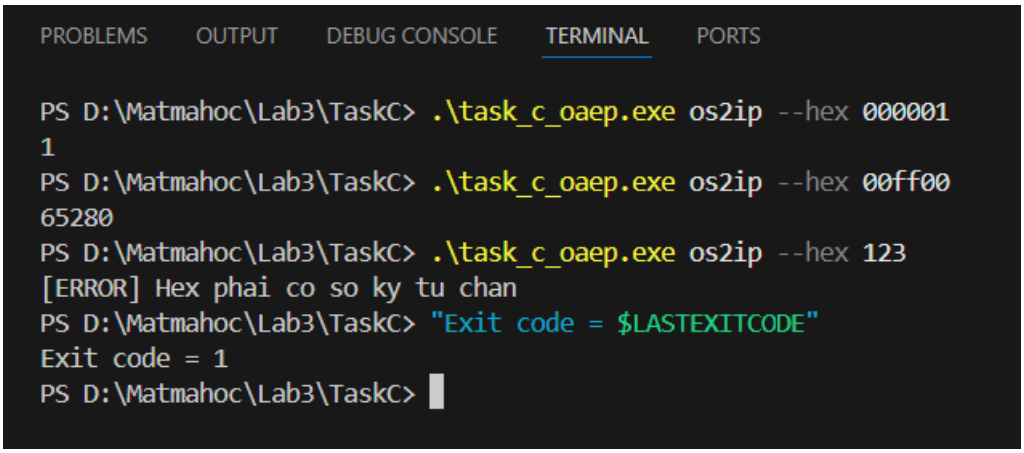
EDUCATIONAL - INSECURE: TEXTBOOK RSA, NEVER USE FOR REAL DATA
===== C-T10: RAW RSA (512 bits) =====
Generated with BN_generate_prime_ex2 + Task B formulas: 512-bit modulus, e=65537
Round-trip X=000001 -> 000001 [OK]
Round-trip X=0100 -> 0100 [OK]
Round-trip X=00ff00 -> 00ff00 [OK]
Round-trip X= ->  [OK]
Round-trip X=01 -> 01 [OK]
Round-trip X=000000 -> 000000 [OK]
Round-trip X=0038e3fc7811f25a0b85454fd46a0fed699dec5e2a0e8f6d2dfc1b226cebe97c -> 0038e3fc7811f25a0b85454fd46a0fed699dec5e2a0e8f6d2dfc1b226cebe97c [OK]
[PASS] C-T10 all fixed-width RSA round-trips
[PASS] C-T10 range check rejects x >= n
[PASS] C-T10 determinism (same plaintext -> same ciphertext)
[PASS] C-T10 malleability: decrypt(Enc(6)*Enc(7))=42

===== C-T11: RSA-3072 OAEP-SHA256/MGF1 =====
RSA=3072 bits, k=384 bytes, maximum=318 bytes
[PASS] C-T11 randomized: two ciphertexts differ
[PASS] C-T11 both decrypt to original
[PASS] C-T11 318-byte message accepted
[PASS] C-T11 319-byte message rejected by EVP
[PASS] C-T11 flipped ciphertext bit rejected

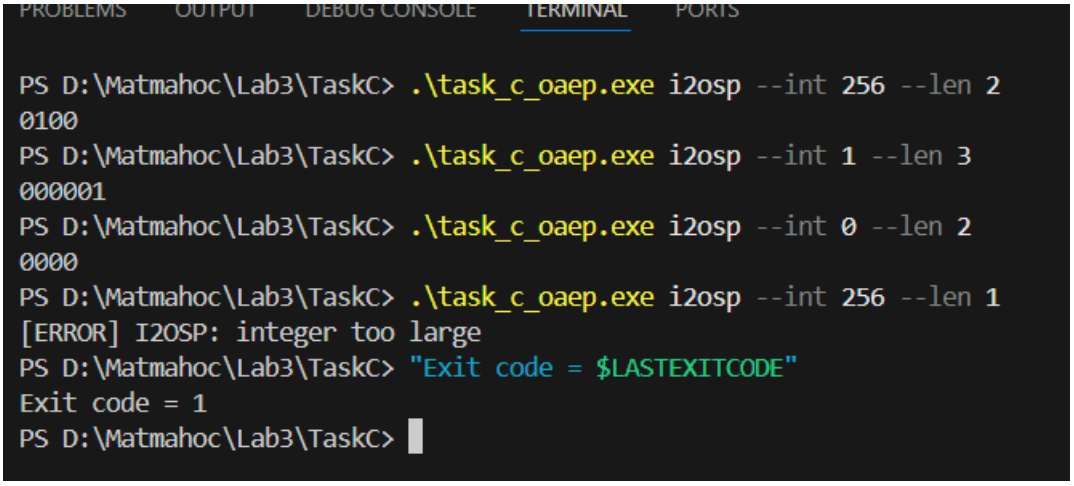
===== SUMMARY =====
Checks: 26, passed: 26, failed: 0
ALL CHECKS PASSED
PS D:\Matmahoc\Lab3\TaskC>
```

Hình 42. Kết quả kiểm thử C-T7, RSA học thuật 512 bit và RSA-OAEP 3072 bit; toàn bộ 26 kiểm tra đều thành công.

Chuyển đổi OS2IP và I2OSP

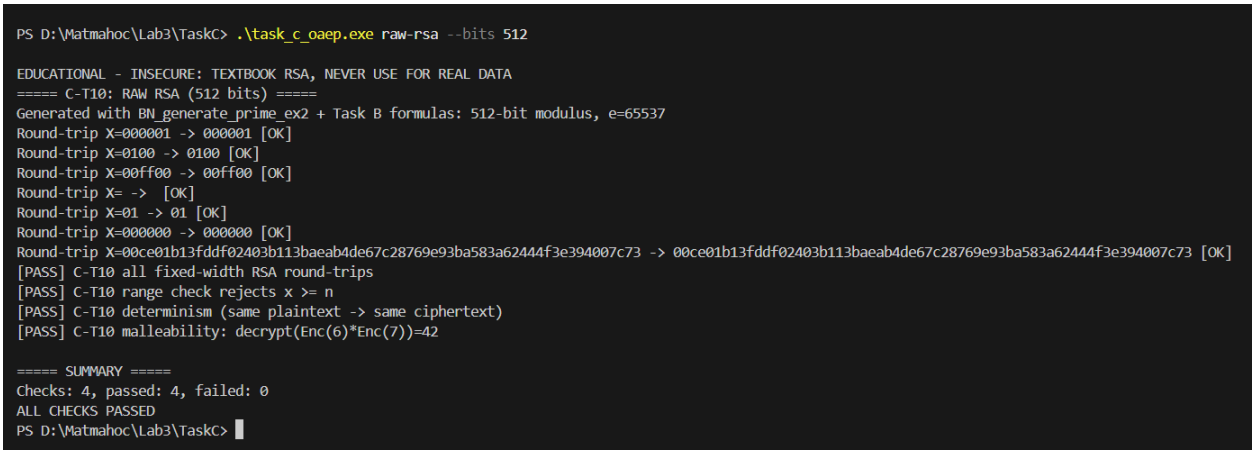


Hình 43. Kiểm thử chức năng OS2IP qua giao diện dòng lệnh với hai dữ liệu hợp lệ và một dữ liệu không hợp lệ. Chương trình chuyển đổi chính xác các giá trị và trả mã lỗi 1 khi đầu vào không đúng định dạng.



Hình 44. Kết quả kiểm thử chức năng I2OSP với các giá trị và độ dài biểu diễn khác nhau. Chương trình tạo chuỗi byte đúng kích thước và trả mã lỗi 1 khi giá trị vượt quá phạm vi biểu diễn.

RSA học thuật và RSA-OAEP



Hình 45. Kiểm thử chức năng RSA học thuật với khóa 512 bit, bao gồm mã hóa và giải mã, kiểm tra phạm vi thông điệp, tính xác định và khả năng biến đổi bản mã. Cả bốn kiểm tra đều thành công.

```
PS D:\Matmahoc\Lab3\TaskC> Get-ChildItem -Path .. -Recurse -File | Where-Object { $_.Name -in @('public.pem', 'private.pem', 'public.der', 'private.der') } |
Select-Object FullName

FullName
--------
D:\Matmahoc\Lab3\TaskB\private.der
D:\Matmahoc\Lab3\TaskB\private.pem
D:\Matmahoc\Lab3\TaskB\public.der
D:\Matmahoc\Lab3\TaskB\public.pem
D:\Matmahoc\Lab3\TaskB\mismatch_test\private.der
D:\Matmahoc\Lab3\TaskB\mismatch_test\private.pem
D:\Matmahoc\Lab3\TaskB\mismatch_test\public.der
D:\Matmahoc\Lab3\TaskB\mismatch_test\public.pem


PS D:\Matmahoc\Lab3\TaskC> .\task_c_oaep.exe oaep-demo --pub ..\TaskB\public.pem --priv ..\TaskB\private.pem

===== OAEP DEMO WITH SAVED KEYS (C-F4) =====
Key bits=3072, k=384, maximum OAEP message=318 bytes
C-T11 RSA-3072 vector: 318 accepted / 319 rejected
[PASS] C-T11 twice encrypted -> different ciphertexts
[PASS] C-T11 both decrypt to original
[PASS] C-T11 max message accepted
[PASS] C-T11 max+1 message rejected
[PASS] C-T11 one bit flipped is rejected

===== SUMMARY =====
Checks: 5, passed: 5, failed: 0
ALL CHECKS PASSED
PS D:\Matmahoc\Lab3\TaskC>
```

Hình 46. Kiểm thử RSA-OAEP với cặp khóa RSA 3072 bit định dạng PEM được tạo ở Task B. Kết quả cho thấy cả năm phép kiểm thử đều thành công, bao gồm tính ngẫu nhiên, giải mã, giới hạn độ dài thông điệp và phát hiện bản mã bị thay đổi.

```
PS D:\Matmahoc\Lab3\TaskC> .\task_c_oaep.exe oaep-demo --pub ..\TaskB\public.der --priv ..\TaskB\private.der

===== OAEP DEMO WITH SAVED KEYS (C-F4) =====
Key bits=3072, k=384, maximum OAEP message=318 bytes
C-T11 RSA-3072 vector: 318 accepted / 319 rejected
[PASS] C-T11 twice encrypted -> different ciphertexts
[PASS] C-T11 both decrypt to original
[PASS] C-T11 max message accepted
[PASS] C-T11 max+1 message rejected
[PASS] C-T11 one bit flipped is rejected

===== SUMMARY =====
Checks: 5, passed: 5, failed: 0
ALL CHECKS PASSED
PS D:\Matmahoc\Lab3\TaskC>
```

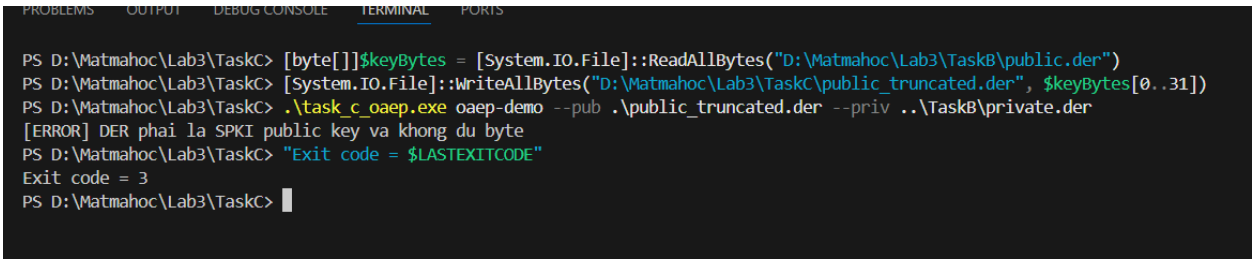
Hình 47. Kiểm thử chức năng RSA-OAEP với cặp khóa RSA 3072 bit định dạng DER được tạo ở Task B. Chương trình hoàn thành cả năm phép kiểm thử, bao gồm tính ngẫu nhiên, khả năng giải mã, giới hạn độ dài thông điệp và phát hiện thay đổi bản mã.

Kiểm thử khóa và dữ liệu không hợp lệ

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\Matmahoc\Lab3\TaskC> .\task_c_oaep.exe oaep-demo --pub ..\TaskB\public.pem --priv ..\TaskB\mismatch_test\private.pem
[ERROR] Khoa cong khai/bi mat khong khop
PS D:\Matmahoc\Lab3\TaskC> "Exit code = $LASTEXITCODE"
Exit code = 4
PS D:\Matmahoc\Lab3\TaskC>
```

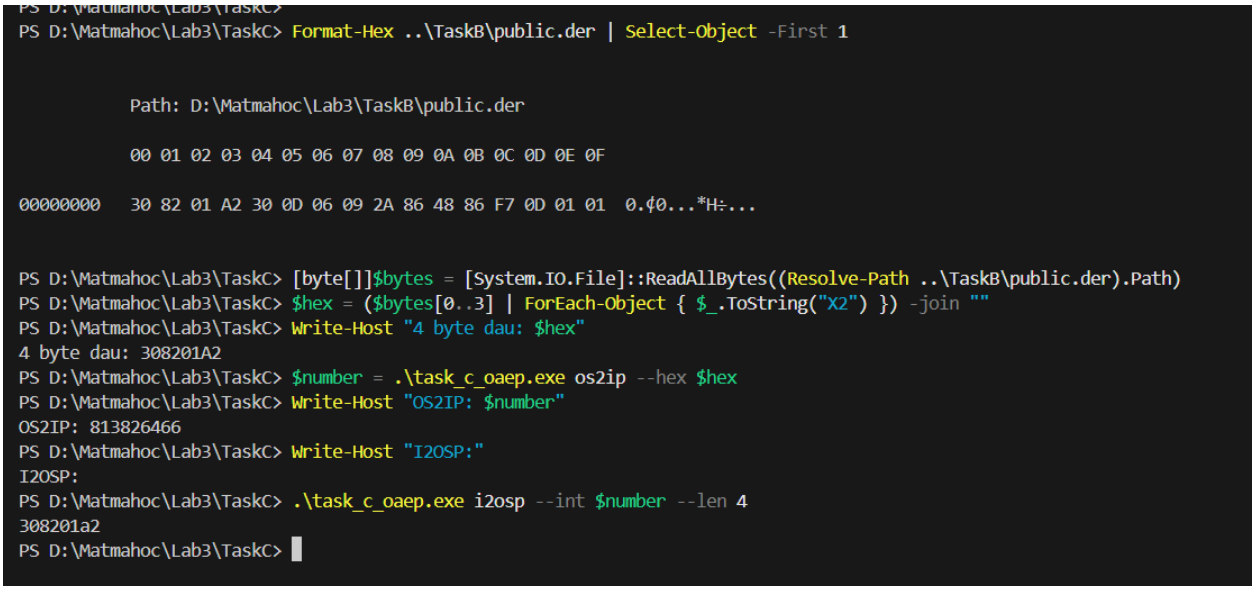
Hình 48. Kiểm thử chức năng RSA-OAEP với cặp khóa không tương ứng. Chương trình phát hiện sự không khớp giữa khóa công khai và khóa bí mật, từ chối thực hiện thao tác và trả về mã lỗi 4.



Hình 49. Kiểm thử khả năng xử lý khóa DER không hợp lệ bằng cách tạo bản sao chỉ chứa 32 byte đầu của khóa công khai. Chương trình từ chối dữ liệu bị cắt ngắn và trả về mã lỗi 3.

Một số ảnh trong mục này dùng lệnh OpenSSL CLI 3.1.4 để đối chiếu kết quả. Các file C++ của bài Lab 3 được biên dịch và liên kết với OpenSSL 4.1.0-beta1; đây là hai phép kiểm tra riêng biệt.

Phân tích bản mã và tính ngẫu nhiên



Hình 50. Phân tích bốn byte đầu của khóa công khai RSA định dạng DER và kiểm chứng phép chuyển đổi OS2IP/I2OSP. Chuỗi 30 82 01 A2 được chuyển thành số nguyên 813826466 và khôi phục chính xác về biểu diễn ban đầu với độ dài 4 byte.

```
PS D:\Matmahoc\Lab3\TaskC> Get-Item .\oaep_plain.bin, .\oaep_from4.bin, .\oaep_decrypted3.bin |
>> Select-Object Name, Length

Name                Length
----                ------
oaep_plain.bin          14
oaep_from4.bin         384
oaep_decrypted3.bin     14


PS D:\Matmahoc\Lab3\TaskC> Format-Hex .\oaep_from4.bin | Select-Object -First 6


           Path: D:\Matmahoc\Lab3\TaskC\oaep_from4.bin

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000000   2A 44 E8 EE C2 45 F8 E7 2D 58 87 A0 7D EA E8 2A  *DèîÂEøç-X }êè*
00000010   13 F3 27 C9 83 0D ED 55 8F FD 40 5B 66 85 5D 7B  .ó'É.íUý@[f]{
00000020   93 9D 7C 13 46 1B 62 56 13 BC 8F 52 D8 7C 28 9D  |.F.bV.¼RØ|(
00000030   F7 A1 CF AD 96 6B 1E 9A 68 D4 AB E3 99 DF 26 CC  ÷¡Ï k.hÔ«ãß&Ì
00000040   7B AC AC F1 BA C9 64 51 8A 7F 7C CD B8 FB E1 AA  {¬¬ñºÉdQ|Í¸ûáª
00000050   D1 08 E1 F5 6E F5 EA 43 58 F5 D0 06 13 21 EE B8  Ñ.áõnõêCXõÐ..!î¸


PS D:\Matmahoc\Lab3\TaskC>
```

Hình 51. Quan sát kích thước và dữ liệu nhị phân của bản mã RSA-OAEP sử dụng khóa RSA 3072 bit. Bản rõ có độ dài 14 byte, trong khi bản mã có độ dài 384 byte, tương ứng với kích thước modulus RSA.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\Matmahoc\Lab3\TaskC> $original = [IO.File]::ReadAllBytes(
>> (Resolve-Path .\oaep_from4.bin).Path
>> )
PS D:\Matmahoc\Lab3\TaskC> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskC> $modified[0x50] = $modified[0x50] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskC> [IO.File]::WriteAllBytes(
>> (Join-Path (Get-Location).Path "oaep_tampered.bin"),
>> $modified
>> )
PS D:\Matmahoc\Lab3\TaskC> Write-Host ("Original byte (0x50): {0:X2}" -f $original[0x50])
Original byte (0x50): D1
PS D:\Matmahoc\Lab3\TaskC> Write-Host ("Modified byte (0x50): {0:X2}" -f $modified[0x50])
Modified byte (0x50): D0
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Original SHA256:"
Original SHA256:
PS D:\Matmahoc\Lab3\TaskC> (Get-FileHash .\oaep_from4.bin).Hash
604466447CEB17B63EB375DE7B3726B681A29B728083462719BFC2A863FAA397
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Modified SHA256:"
Modified SHA256:
PS D:\Matmahoc\Lab3\TaskC> (Get-FileHash .\oaep_tampered.bin).Hash
DB1D2FF64DDAC4A4EB92DD311E574F9028FCA0835A24D94178F92947B79B4FF3
PS D:\Matmahoc\Lab3\TaskC>
```

Hình 52. Thực nghiệm thay đổi một bit tại offset 0x50 của bản mã RSA-OAEP. Giá trị byte thay đổi từ D1 thành D0 thông qua phép XOR với 01. Giá trị băm SHA-256 của hai tệp khác nhau, xác nhận dữ liệu bản mã đã bị thay đổi.

```
PS D:\Matmahoc\Lab3\TaskC> $ossl3 = "C:\Program Files\OpenSSL\bin\openssl.exe"
PS D:\Matmahoc\Lab3\TaskC> & $ossl3 pkeyutl -decrypt `
>> -inkey ..\TaskB\private.pem `
>> -in .\oaep_tampered.bin `
>> -out .\oaep_tampered_dec.bin `
>> -pkeyopt rsa_padding_mode:oaep `
>> -pkeyopt rsa_oaep_md:sha256 `
>> -pkeyopt rsa_mgf1_md:sha256
Public Key operation error
088A0000:error:02000079:rsa routines:RSA_padding_check_PKCS1_OAEP_mgf1:oaep decoding error:crypto\rsa\rsa_oaep.c:314:
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 1
PS D:\Matmahoc\Lab3\TaskC>
```

Hình 53. Kiểm tra giải mã bản mã RSA-OAEP sau khi thay đổi một bit. OpenSSL 3.1.4 trả về thông báo "oaep decoding error" và mã thoát 1, cho thấy bản mã không vượt qua bước kiểm tra định dạng OAEP.

```
PS D:\Matmahoc\Lab3\TaskC> Write-Host "=== ORIGINAL ==="
=== ORIGINAL ===
PS D:\Matmahoc\Lab3\TaskC> Format-Hex .\oaep_from4.bin |
>> Select-Object -Skip 5 -First 1


           Path: D:\Matmahoc\Lab3\TaskC\oaep_from4.bin

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000050   D1 08 E1 F5 6E F5 EA 43 58 F5 D0 06 13 21 EE B8  Ñ.áõnõêCXõÐ..!î¸


PS D:\Matmahoc\Lab3\TaskC> Write-Host "=== TAMPERED ==="
=== TAMPERED ===
PS D:\Matmahoc\Lab3\TaskC> Format-Hex .\oaep_tampered.bin |
>> Select-Object -Skip 5 -First 1


           Path: D:\Matmahoc\Lab3\TaskC\oaep_tampered.bin

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000050   D0 08 E1 F5 6E F5 EA 43 58 F5 D0 06 13 21 EE B8  Ð.áõnõêCXõÐ..!î¸


PS D:\Matmahoc\Lab3\TaskC>
```

Hình 54. Đối chiếu dữ liệu nhị phân của bản mã RSA-OAEP trước và sau khi thay đổi một bit. Tại offset 0x50, giá trị byte chuyển từ D1 thành D0, trong khi các byte còn lại trên cùng dòng không thay đổi.

```
PS D:\Matmahoc\Lab3\TaskC> $ossl4 = "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe"
PS D:\Matmahoc\Lab3\TaskC> & $ossl4 pkeyutl -encrypt `
>>     -pubin `
>>     -inkey ..\TaskB\public.pem `
>>     -in .\oaep_plain.bin `
>>     -out .\oaep_second.bin `
>>     -pkeyopt rsa_padding_mode:oaep `
>>     -pkeyopt rsa_oaep_md:sha256 `
>>     -pkeyopt rsa_mgf1_md:sha256
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskC>
```

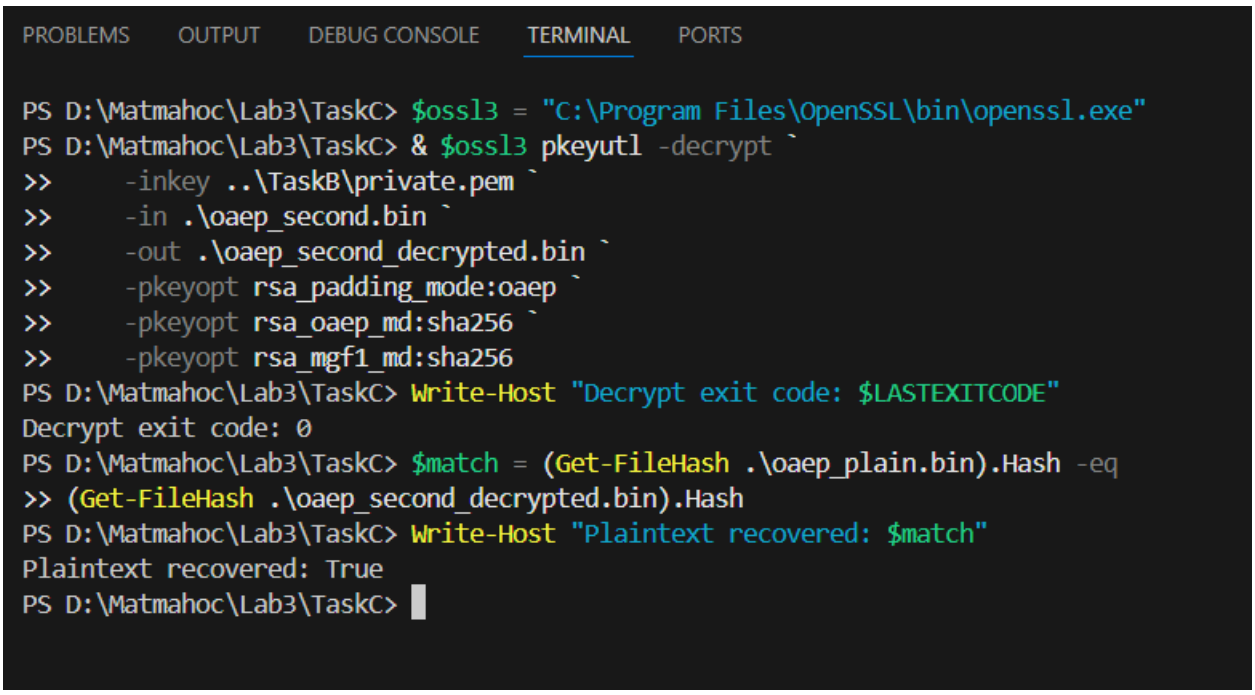
Hình 55. Thực hiện mã hóa lại bản rõ bằng RSA-OAEP với SHA-256 và MGF1-SHA256 trên OpenSSL 4.1.0-beta1. Quá trình mã hóa hoàn tất với mã thoát 0.

```
PS D:\Matmahoc\Lab3\TaskC> Get-Item .\oaep_from4.bin, .\oaep_second.bin |
>> Select-Object Name, Length

Name            Length
----            ------
oaep_from4.bin     384
oaep_second.bin    384


PS D:\Matmahoc\Lab3\TaskC> $hash1 = (Get-FileHash .\oaep_from4.bin).Hash
PS D:\Matmahoc\Lab3\TaskC> $hash2 = (Get-FileHash .\oaep_second.bin).Hash
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Ciphertext 1 SHA256: $hash1"
Ciphertext 1 SHA256: 604466447CEB17B63EB375DE7B3726B681A29B728083462719BFC2A863FAA397
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Ciphertext 2 SHA256: $hash2"
Ciphertext 2 SHA256: A8F10E137EC791C7C87F8859F367BA40A173A850353A7C5F6D5CD5873986B965
PS D:\Matmahoc\Lab3\TaskC> Write-Host "Ciphertexts identical: $($hash1 -eq $hash2)"
Ciphertexts identical: False
PS D:\Matmahoc\Lab3\TaskC>
```

Hình 56. So sánh hai bản mã RSA-OAEP được tạo từ cùng bản rõ và cùng khóa công khai. Cả hai bản mã đều dài 384 byte nhưng có giá trị băm SHA-256 khác nhau, cho thấy kết quả mã hóa không cố định giữa các lần thực hiện.



Hình 57. Giải mã bản mã thứ hai bằng OpenSSL 3.1.4 với khóa bí mật RSA tương ứng. Quá trình giải mã thành công với mã thoát 0 và kết quả đối chiếu bản rõ trả về True.

Task D

1. Mã nguồn

Mã nguồn: src/task_d.cpp (phiên bản tích hợp CLI trong thư mục nộp bài).

```
#include "key_io.hpp"
#include "task_cli.hpp"
#include "ossl_raii.hpp"
// Lab 3 - Task D: RSA-OAEP-SHA256 + AES-256-GCM.
// Một tệp nguồn cho phần thực hành; các lệnh encrypt/decrypt dùng lại khi tích hợp lab03.
// Sử dụng C++17 và thư viện libcrypto.
// Không in khóa AES, khóa bí mật RSA hoặc dữ liệu giải mã chưa xác thực.


#include <openssl/crypto.h>
#include <openssl/err.h>
#include <openssl/evp.h>
#include <openssl/pem.h>
#include <openssl/rand.h>
#include <openssl/rsa.h>
#include <openssl/x509.h>


#include <algorithm>
#include <array>
#include <cstdint>
#include <cstring>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>


namespace task_d {


namespace fs = std::filesystem;
using Bytes = std::vector<unsigned char>;


namespace {
constexpr std::size_t BLOCK = 64 * 1024;
constexpr std::uint64_t MAX_FILE = std::uint64_t{1} << 30;  // 1 GiB.
constexpr std::size_t AES_KEY_LEN = 32;
constexpr std::size_t NONCE_LEN = 12;
constexpr std::size_t TAG_LEN = 16;
constexpr unsigned char VERSION = 0x01;
constexpr unsigned char MODE_OAEP = 0x01;


using LabError = lab::Error;


[[noreturn]] void fail(int code, const std::string& detail) { throw LabError(code, detail); }


std::string opensslError() {
    const unsigned long e = ERR_get_error();
    if (!e) return "OpenSSL returned failure";
    char buf[256]{};
    ERR_error_string_n(e, buf, sizeof(buf));
    while (ERR_get_error() != 0) {}  // Xóa các lỗi còn lại.
    return buf;
}


void sslCheck(int result, const std::string& operation) {
    if (result <= 0) fail(5, operation + ": " + opensslError());
}


using PKey = std::unique_ptr<EVP_PKEY, decltype(&EVP_PKEY_free)>;
using PKeyCtx = std::unique_ptr<EVP_PKEY_CTX, decltype(&EVP_PKEY_CTX_free)>;
using CipherCtx = std::unique_ptr<EVP_CIPHER_CTX, decltype(&EVP_CIPHER_CTX_free)>;
using Bio = std::unique_ptr<BIO, decltype(&BIO_free)>;
```

```
struct SecureKey {
    std::array<unsigned char, AES_KEY_LEN> data{};
    ~SecureKey() { OPENSSL_cleanse(data.data(), data.size()); }
    SecureKey() = default;
    SecureKey(const SecureKey&) = delete;
    SecureKey& operator=(const SecureKey&) = delete;
};


struct SecureChunk {
    std::array<unsigned char, BLOCK + 16> data{};
    ~SecureChunk() { OPENSSL_cleanse(data.data(), data.size()); }
};


std::uint64_t lengthOf(const fs::path& path) {
    std::error_code ec;
    const auto n = fs::file_size(path, ec);
    if (ec) fail(2, "cannot read input file size: " + path.string());
    return static_cast<std::uint64_t>(n);
}


void readExact(std::istream& f, unsigned char* dest, std::size_t n) {
    if (n == 0) return;
    if (!f.read(reinterpret_cast<char*>(dest), static_cast<std::streamsize>(n)))
        fail(2, "cannot read input file");
}


void writeExact(std::ostream& f, const unsigned char* data, std::size_t n) {
    if (n == 0) return;
    f.write(reinterpret_cast<const char*>(data), static_cast<std::streamsize>(n));
    if (!f) fail(2, "cannot write output file");
}


Bytes readSmallFile(const fs::path& path, std::size_t limit) {
    const auto size = lengthOf(path);
    if (size > limit) fail(3, "file is too large");
    std::ifstream f(path, std::ios::binary);
    if (!f) fail(2, "cannot open input file: " + path.string());
    Bytes bytes(static_cast<std::size_t>(size));
    readExact(f, bytes.data(), bytes.size());
    return bytes;
}


void writeSmallFile(const fs::path& path, const Bytes& bytes) {
    std::ofstream f(path, std::ios::binary | std::ios::trunc);
    if (!f) fail(2, "cannot open output file: " + path.string());
    writeExact(f, bytes.data(), bytes.size());
    f.close();
    if (!f) fail(2, "cannot close output file");
}


// Hỗ trợ PEM và DER, kiểm tra cả phần dữ liệu thừa sau khóa DER/PEM.
PKey loadKey(const fs::path& path, bool privatePart) {
    auto key=task_b::loadKey(path,privatePart);
    if(privatePart) task_b::validatePrivate(key.get()); else task_b::validatePublic(key.get());
    return key;
}


void checkRsaKey(EVP_PKEY* key) {
    if (!key || EVP_PKEY_is_a(key, "RSA") != 1 || EVP_PKEY_get_bits(key) < 2048)
        fail(3, "RSA key must have at least 2048 bits");
}


void configureOaep(EVP_PKEY_CTX* ctx) {
    sslCheck(EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_OAEP_PADDING), "OAEP padding");
    sslCheck(EVP_PKEY_CTX_set_rsa_oaep_md(ctx, EVP_sha256()), "OAEP SHA-256");
    sslCheck(EVP_PKEY_CTX_set_rsa_mgf1_md(ctx, EVP_sha256()), "MGF1 SHA-256");
}


Bytes wrapKey(EVP_PKEY* pub, const SecureKey& aes) {
    PKeyCtx ctx(EVP_PKEY_CTX_new_from_pkey(nullptr, pub, nullptr), EVP_PKEY_CTX_free);
    if (!ctx) fail(5, "cannot create RSA context: " + opensslError());
    sslCheck(EVP_PKEY_encrypt_init(ctx.get()), "RSA encrypt init");
```

```
    configureOaep(ctx.get());
    std::size_t n = 0;
    sslCheck(EVP_PKEY_encrypt(ctx.get(), nullptr, &n, aes.data.data(), aes.data.size()),
             "RSA wrapped key length");
    Bytes wrapped(n);
    sslCheck(EVP_PKEY_encrypt(ctx.get(), wrapped.data(), &n, aes.data.data(), aes.data.size()),
             "RSA wrapped key");
    wrapped.resize(n);
    if (wrapped.size() != static_cast<std::size_t>(EVP_PKEY_get_size(pub)))
        fail(5, "unexpected wrapped key size");
    return wrapped;
}

// Mọi lỗi giải mã OAEP đều trả về cùng thông báo, không để lộ lỗi padding.
void unwrapKey(EVP_PKEY* priv, const Bytes& wrapped, SecureKey& aes) {
    PKeyCtx ctx(EVP_PKEY_CTX_new_from_pkey(nullptr, priv, nullptr), EVP_PKEY_CTX_free);
    if (!ctx) fail(5, "cannot create RSA context: " + opensslError());
    sslCheck(EVP_PKEY_decrypt_init(ctx.get()), "RSA decrypt init");
    configureOaep(ctx.get());
    std::size_t n = 0;
    if (EVP_PKEY_decrypt(ctx.get(), nullptr, &n, wrapped.data(), wrapped.size()) <= 0) {
        ERR_clear_error();
        fail(4, "decryption failed");
    }
    Bytes recovered(n);
    if (EVP_PKEY_decrypt(ctx.get(), recovered.data(), &n, wrapped.data(), wrapped.size()) <= 0 ||
        n != AES_KEY_LEN) {
        if (!recovered.empty()) OPENSSL_cleanse(recovered.data(), recovered.size());
        ERR_clear_error();
        fail(4, "decryption failed");
    }
    std::copy_n(recovered.begin(), AES_KEY_LEN, aes.data.begin());
    OPENSSL_cleanse(recovered.data(), recovered.size());
}

void putU16(Bytes& b, std::uint16_t v) {
    b.push_back(static_cast<unsigned char>(v >> 8));
    b.push_back(static_cast<unsigned char>(v));
}
void putU64(Bytes& b, std::uint64_t v) {
    for (int shift = 56; shift >= 0; shift -= 8)
        b.push_back(static_cast<unsigned char>(v >> shift));
}
std::uint16_t getU16(const unsigned char* p) {
    return static_cast<std::uint16_t>((std::uint16_t(p[0]) << 8) | p[1]);
}
std::uint64_t getU64(const unsigned char* p) {
    std::uint64_t v = 0;
    for (int i = 0; i < 8; ++i) v = (v << 8) | p[i];
    return v;
}

// Ghi vào <out>.part; chỉ đổi sang tên chính thức khi công việc thành công.
class PendingOutput {
public:
    fs::path finalPath;
    fs::path partPath;
    bool done = false;
    bool force = false;

    PendingOutput(const fs::path& requested, bool allowOverwrite)
        : finalPath(requested), partPath(fs::path(requested.string() + ".part")),
          force(allowOverwrite) {
        std::error_code ec;
        if (fs::exists(finalPath, ec) && !force) fail(2, "output exists (use --force)");
        if (fs::exists(partPath, ec)) fail(2, "temporary .part file already exists");
    }

    ~PendingOutput() {
        if (!done) {
            std::error_code ec;
            fs::remove(partPath, ec);
        }
    }

    void commit() {
        std::error_code ec;
```

```
        if (force && fs::exists(finalPath, ec)) {
            fs::remove(finalPath, ec);  // Chỉ xóa kết quả cũ sau khi hoàn tất xác thực.
            if (ec) fail(2, "cannot replace existing output");
        }
        fs::rename(partPath, finalPath, ec);
        if (ec) fail(2, "cannot finalize output: " + ec.message());
        done = true;
    }
};

void rejectSameFile(const fs::path& input, const fs::path& output) {
    std::error_code ec;
    if (fs::exists(output, ec) && fs::equivalent(input, output, ec) && !ec)
        fail(1, "input and output paths must be different");
    if (fs::absolute(input).lexically_normal() == fs::absolute(output).lexically_normal())
        fail(1, "input and output paths must be different");
}

// Header v1: L2PK | version | mode | reserved | klen | wrapped_key | nonce | ct_len.
// Toàn bộ header được xác thực bằng AES-GCM AAD.
std::size_t encryptFile(EVP_PKEY* pub, const fs::path& input,
                        const fs::path& output, bool force) {
    checkRsaKey(pub);
    rejectSameFile(input, output);
    const std::uint64_t n = lengthOf(input);
    if (n > MAX_FILE) fail(3, "input exceeds 1 GiB limit");
    std::ifstream in(input, std::ios::binary);
    if (!in) fail(2, "cannot open input file");
    PendingOutput target(output, force);
    rejectSameFile(input, target.partPath);


    SecureKey aes;
    std::array<unsigned char, NONCE_LEN> nonce{};
    sslCheck(RAND_bytes(aes.data.data(), static_cast<int>(aes.data.size())), "AES key RNG");
    sslCheck(RAND_bytes(nonce.data(), static_cast<int>(nonce.size())), "nonce RNG");
    const Bytes wrapped = wrapKey(pub, aes);


    Bytes header{'L', '2', 'P', 'K', VERSION, MODE_OAEP, 0x00, 0x00};
    if (wrapped.size() > std::numeric_limits<std::uint16_t>::max())
        fail(5, "RSA key is too large for package");
    putU16(header, static_cast<std::uint16_t>(wrapped.size()));
    header.insert(header.end(), wrapped.begin(), wrapped.end());
    header.insert(header.end(), nonce.begin(), nonce.end());
    putU64(header, n);


    std::ofstream out(target.partPath, std::ios::binary | std::ios::trunc);
    if (!out) fail(2, "cannot create output .part");
    writeExact(out, header.data(), header.size());


    CipherCtx ctx(EVP_CIPHER_CTX_new(), EVP_CIPHER_CTX_free);
    if (!ctx) fail(5, "cannot create AES-GCM context");
    sslCheck(EVP_EncryptInit_ex2(ctx.get(), EVP_aes_256_gcm(),
                                 aes.data.data(), nonce.data(), nullptr), "GCM encrypt init");
    int produced = 0;
    sslCheck(EVP_EncryptUpdate(ctx.get(), nullptr, &produced, header.data(),
                               static_cast<int>(header.size())), "GCM header AAD");
    std::array<unsigned char, BLOCK> inBuf{};
    std::array<unsigned char, BLOCK + 16> outBuf{};
    std::uint64_t remaining = n;
    while (remaining > 0) {
        const auto chunk = static_cast<std::size_t>(std::min<std::uint64_t>(remaining, BLOCK));
        readExact(in, inBuf.data(), chunk);
        sslCheck(EVP_EncryptUpdate(ctx.get(), outBuf.data(), &produced,
                                   inBuf.data(), static_cast<int>(chunk)), "GCM encrypt update");
        writeExact(out, outBuf.data(), static_cast<std::size_t>(produced));
        remaining -= chunk;
    }
    sslCheck(EVP_EncryptFinal_ex(ctx.get(), outBuf.data(), &produced), "GCM encrypt final");
    writeExact(out, outBuf.data(), static_cast<std::size_t>(produced));
    std::array<unsigned char, TAG_LEN> tag{};
    sslCheck(EVP_CIPHER_CTX_ctrl(ctx.get(), EVP_CTRL_AEAD_GET_TAG,
                                 static_cast<int>(tag.size()), tag.data()), "GCM get tag");
    writeExact(out, tag.data(), tag.size());
    out.close();
    if (!out) fail(2, "cannot close encrypted file");
    target.commit();
    return header.size();
```

```
}

struct PackageInfo {
    Bytes header;
    Bytes wrapped;
    std::array<unsigned char, NONCE_LEN> nonce{};
    std::uint64_t ctLen = 0;
    std::size_t k = 0;
};

// Kiểm tra đầy đủ định dạng trước khi giải mã hoặc cấp phát theo ct_len.
PackageInfo parsePackage(std::ifstream& in, std::uint64_t fileSize,
                         EVP_PKEY* priv, bool strictReserved = true) {
    if (fileSize < 10 + NONCE_LEN + 8 + TAG_LEN) fail(3, "truncated package");
    std::array<unsigned char, 10> first{};
    readExact(in, first.data(), first.size());
    if (std::memcmp(first.data(), "L2PK", 4) != 0) fail(3, "bad package magic");
    if (first[4] != VERSION) fail(3, "unsupported package version");
    if (first[5] != MODE_OAEP) fail(3, "unexpected package mode");
    if (strictReserved && (first[6] != 0 || first[7] != 0))
        fail(3, "reserved bytes must be zero");


    const auto k = static_cast<std::size_t>(EVP_PKEY_get_size(priv));
    const auto klen = static_cast<std::size_t>(getU16(first.data() + 8));
    if (klen != k) fail(3, "klen does not match RSA modulus size");
    const auto headerSize = 30 + klen;
    if (fileSize < static_cast<std::uint64_t>(headerSize + TAG_LEN))
        fail(3, "truncated package");


    PackageInfo info;
    info.k = k;
    info.header.assign(first.begin(), first.end());
    info.header.resize(headerSize);
    readExact(in, info.header.data() + 10, headerSize - 10);
    const auto* p = info.header.data();
    info.ctLen = getU64(p + 22 + klen);
    if (info.ctLen > MAX_FILE) fail(3, "ciphertext length exceeds 1 GiB limit");
    if (fileSize != static_cast<std::uint64_t>(headerSize) + info.ctLen + TAG_LEN)
        fail(3, "package total length mismatch");
    info.wrapped.assign(p + 10, p + 10 + klen);
    std::copy_n(p + 10 + klen, NONCE_LEN, info.nonce.begin());
    return info;
}

std::size_t decryptFile(EVP_PKEY* priv, const fs::path& input,
                        const fs::path& output, bool force, bool strictReserved = true) {
    checkRsaKey(priv);
    rejectSameFile(input, output);
    const auto fileSize = lengthOf(input);
    std::ifstream in(input, std::ios::binary);
    if (!in) fail(2, "cannot open package");
    const PackageInfo info = parsePackage(in, fileSize, priv, strictReserved);
    PendingOutput target(output, force);
    rejectSameFile(input, target.partPath);
    SecureKey aes;
    unwrapKey(priv, info.wrapped, aes);


    CipherCtx ctx(EVP_CIPHER_CTX_new(), EVP_CIPHER_CTX_free);
    if (!ctx) fail(5, "cannot create AES-GCM context");
    sslCheck(EVP_DecryptInit_ex2(ctx.get(), EVP_aes_256_gcm(),
                                 aes.data.data(), info.nonce.data(), nullptr), "GCM decrypt init");
    int produced = 0;
    sslCheck(EVP_DecryptUpdate(ctx.get(), nullptr, &produced, info.header.data(),
                               static_cast<int>(info.header.size())), "GCM header AAD");
    std::ofstream out(target.partPath, std::ios::binary | std::ios::trunc);
    if (!out) fail(2, "cannot create output .part");
    std::array<unsigned char, BLOCK> inBuf{};
    SecureChunk ptBuf;
    std::uint64_t remaining = info.ctLen;
    while (remaining > 0) {
        const auto chunk = static_cast<std::size_t>(std::min<std::uint64_t>(remaining, BLOCK));
        readExact(in, inBuf.data(), chunk);
        sslCheck(EVP_DecryptUpdate(ctx.get(), ptBuf.data.data(), &produced,
                                   inBuf.data(), static_cast<int>(chunk)), "GCM decrypt update");
        writeExact(out, ptBuf.data.data(), static_cast<std::size_t>(produced));
        remaining -= chunk;
    }
```

```
    std::array<unsigned char, TAG_LEN> tag{};
    readExact(in, tag.data(), tag.size());
    sslCheck(EVP_CIPHER_CTX_ctrl(ctx.get(), EVP_CTRL_AEAD_SET_TAG,
                                 static_cast<int>(tag.size()), tag.data()), "GCM set tag");
    if (EVP_DecryptFinal_ex(ctx.get(), ptBuf.data.data(), &produced) != 1) {
        ERR_clear_error();
        out.close();
        fail(4, "decryption failed");
    }
    writeExact(out, ptBuf.data.data(), static_cast<std::size_t>(produced));
    out.close();
    if (!out) fail(2, "cannot close decrypted file");
    target.commit();
    return info.header.size();
}


std::string hexHead(const unsigned char* p, std::size_t n) {
    static const char* table = "0123456789ABCDEF";
    std::string str;
    for (std::size_t i = 0; i < n; ++i) {
        if (i) str += ' ';
        str += table[p[i] >> 4];
        str += table[p[i] & 15];
    }
    return str;
}


void reportPackage(const fs::path& package, std::size_t headerSize) {
    const Bytes small = readSmallFile(package, 8192 + 4096);
    const std::size_t total = small.size();
    if (total < headerSize + TAG_LEN) fail(5, "demo package truncated");
    std::cout << "  magic/version/mode: " << hexHead(small.data(), 8) << "\n";
    std::cout << "  header (first 16): " << hexHead(small.data(), std::min<std::size_t>(16, headerSize)) <<
"\n";
    std::cout << "  ciphertext (first 16): "
              << hexHead(small.data() + headerSize,
                         std::min<std::size_t>(16, total - headerSize - TAG_LEN)) << "\n";
    std::cout << "  tag: " << hexHead(small.data() + total - TAG_LEN, TAG_LEN) << "\n";
    std::cout << "  wrote " << package.string() << " (" << total << " = " << headerSize
              << " header + " << total - headerSize - TAG_LEN << " ciphertext + 16 tag)\n";
}


// Phần demo giữ đúng ý tưởng PDF: round trip, ngẫu nhiên và thay đổi từng trường.
// Các bài kiểm thử dự án lớn (50 MiB, PEM/DER, D-T11) sẽ kiểm tra riêng qua CLI.
int selfTest(EVP_PKEY* pub, EVP_PKEY* priv, const fs::path* userInput) {
    const fs::path input = userInput ? *userInput : fs::path("demo_input.bin");
    if (!userInput) {
        const std::string message = "Meet at the library at 10:00. Bring the USB key.";
        writeSmallFile(input, Bytes(message.begin(), message.end()));
    }
    const auto bytes = lengthOf(input);
    if (bytes > 8192) fail(1, "in-class demo accepts up to 8192 bytes; use encrypt for larger files");
    const fs::path package("demo.l2pk"), decoded("demo_decoded.bin");
    const fs::path second("demo_second.l2pk"), tampered("demo_tampered.l2pk");
    const fs::path shouldNotExist("demo_failed_plain.bin");


    int passed = 0, total = 0;
    const auto check = [&](const std::string& name, bool condition) {
        ++total;
        if (condition) ++passed;
        std::cout << (condition ? "[PASS] " : "[FAIL] ") << name << "\n";
    };


    std::cout << "== Task D: RSA-OAEP + AES-256-GCM, " << bytes << "-byte message ==\n";
    const std::size_t headerSize = encryptFile(pub, input, package, true);
    reportPackage(package, headerSize);
    decryptFile(priv, package, decoded, true);
    check("round trip recovers the message", readSmallFile(input, 8192) == readSmallFile(decoded, 8192));
    encryptFile(pub, input, second, true);
    check("sealing twice gives different packages", readSmallFile(package, 8192 + 4096) !=
readSmallFile(second, 8192 + 4096));


    const Bytes original = readSmallFile(package, 8192 + 4096);
    const auto expectFailure = [&](const char* label, std::size_t offset,
                                   int expectedCode, bool strictReserved = true) {
        Bytes changed = original;
        if (offset >= changed.size()) fail(5, "selftest offset out of range");
```

```
        changed[offset] ^= 0x01;
        writeSmallFile(tampered, changed);
        std::error_code ec;
        fs::remove(shouldNotExist, ec);
        bool failedCorrectly = false;
        std::string why;
        try { decryptFile(priv, tampered, shouldNotExist, false, strictReserved); }
        catch (const LabError& e) { why = e.what(); failedCorrectly = e.exitCode == expectedCode; }
        const bool noPlaintext = !fs::exists(shouldNotExist) && !fs::exists(fs::path(shouldNotExist.string() +
".part"));
        std::cout << "  flip " << label << " -> " << why << "\n";
        check(label, failedCorrectly && noPlaintext);
    };
    std::cout << "\n-- tampering (each must fail, no plaintext) --\n";
    if (bytes != 0) expectFailure("first ciphertext byte", headerSize, 4);
    expectFailure("last tag byte", original.size() - 1, 4);
    expectFailure("first nonce byte", 10 + EVP_PKEY_get_size(priv), 4);
    expectFailure("a byte of the wrapped key", 15, 4);
    expectFailure("version byte", 4, 3);
    expectFailure("reserved byte (strict parser)", 7, 3);
    // Chỉ dùng chế độ bỏ qua kiểm tra reserved trong selftest để chứng minh AAD.
    // Lệnh decrypt thông thường LUÔN bật strictReserved.
    expectFailure("reserved byte (lax parser, AAD)", 7, 4, false);
    expectFailure("magic byte", 0, 3);


    Bytes truncated = original;
    truncated.pop_back();
    writeSmallFile(tampered, truncated);
    bool rejectsTruncation = false;
    try { decryptFile(priv, tampered, shouldNotExist, false); }
    catch (const LabError& e) { rejectsTruncation = (e.exitCode == 3); }
    check("truncated package rejected", rejectsTruncation && !fs::exists(shouldNotExist));


    PKey other(EVP_RSA_gen(EVP_PKEY_get_bits(priv)), EVP_PKEY_free);
    if (!other) fail(5, "cannot generate second RSA key: " + opensslError());
    bool rejectsWrongKey = false;
    try { decryptFile(other.get(), package, shouldNotExist, false); }
    catch (const LabError& e) { rejectsWrongKey = e.exitCode == 4; }
    check("wrong private key cannot open", rejectsWrongKey && !fs::exists(shouldNotExist));


    std::error_code ec;
    fs::remove(tampered, ec);
    fs::remove(second, ec);
    std::cout << "\n" << passed << "/" << total << " PASS\n";
    return passed == total ? 0 : 5;
}


void usage() {
    std::cout
        << "Lab 3 Task D - RSA-OAEP-SHA256 + AES-256-GCM\n"
        << "  task_d_hybrid.exe                                  (demo, fresh RSA key)\n"
        << "  task_d_hybrid.exe <public.pem> <private.pem> <file> (demo with Task B keys)\n"
        << "  task_d_hybrid.exe --all                             (in-class selftest)\n"
        << "  task_d_hybrid.exe encrypt --pub <key> --in <file> --out <package> [--force]\n"
        << "  task_d_hybrid.exe decrypt --priv <key> --in <package> --out <file> [--force]\n"
        << "  task_d_hybrid.exe --help\n";
}


struct CliArgs { fs::path key; fs::path input; fs::path output; bool force = false; };
CliArgs parseCli(int argc, char** argv, bool isEncrypt) {
    CliArgs a;
    bool hasKey = false, hasIn = false, hasOut = false;
    for (int i = 2; i < argc; ++i) {
        const std::string arg = argv[i];
        if (arg == "--force") { if (a.force) fail(1, "duplicate --force"); a.force = true; continue; }
        if (arg == (isEncrypt ? "--pub" : "--priv")) {
            if (hasKey || i + 1 >= argc) fail(1, "invalid key option");
            a.key = argv[++i]; hasKey = true;
        } else if (arg == "--in") {
            if (hasIn || i + 1 >= argc) fail(1, "invalid --in option");
            a.input = argv[++i]; hasIn = true;
        } else if (arg == "--out") {
            if (hasOut || i + 1 >= argc) fail(1, "invalid --out option");
            a.output = argv[++i]; hasOut = true;
        } else fail(1, "unknown or unsupported argument: " + arg);
    }
    if (!hasKey || !hasIn || !hasOut) fail(1, "missing --pub/--priv, --in, or --out");
    return a;
```

```
}
}  // namespace


int entry(int argc, char** argv) {
    try {
        if (argc == 1 || (argc == 2 && std::string(argv[1]) == "--all")) {
            PKey key(EVP_RSA_gen(3072), EVP_PKEY_free);
            if (!key) fail(5, "cannot generate demo RSA key: " + opensslError());
            return selfTest(key.get(), key.get(), nullptr);
        }
        if (argc == 2 && (std::string(argv[1]) == "--help" || std::string(argv[1]) == "help")) {
            usage(); return 0;
        }
        if (argc == 4 && argv[1][0] != '-') {
            PKey pub = loadKey(argv[1], false);
            PKey priv = loadKey(argv[2], true);
            const fs::path input(argv[3]);
            return selfTest(pub.get(), priv.get(), &input);
        }
        const std::string command = argv[1];
        if (command == "encrypt" || command == "decrypt") {
            const bool encrypt = command == "encrypt";
            const CliArgs args = parseCli(argc, argv, encrypt);
            PKey key = loadKey(args.key, !encrypt);
            const std::size_t headerSize = encrypt ? encryptFile(key.get(), args.input, args.output,
args.force)
                                                   : decryptFile(key.get(), args.input, args.output,
args.force);
            std::cout << (encrypt ? "ENCRYPT OK" : "DECRYPT OK")
                      << " | header=" << headerSize << " bytes | output=" << args.output.string() << "\n";
            return 0;
        }
        usage();
        return 1;
    } catch (const LabError& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return e.exitCode;
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return 5;
    }
}

} // namespace task_d
```

2. Hình ảnh thực hiện và kết quả kiểm thử

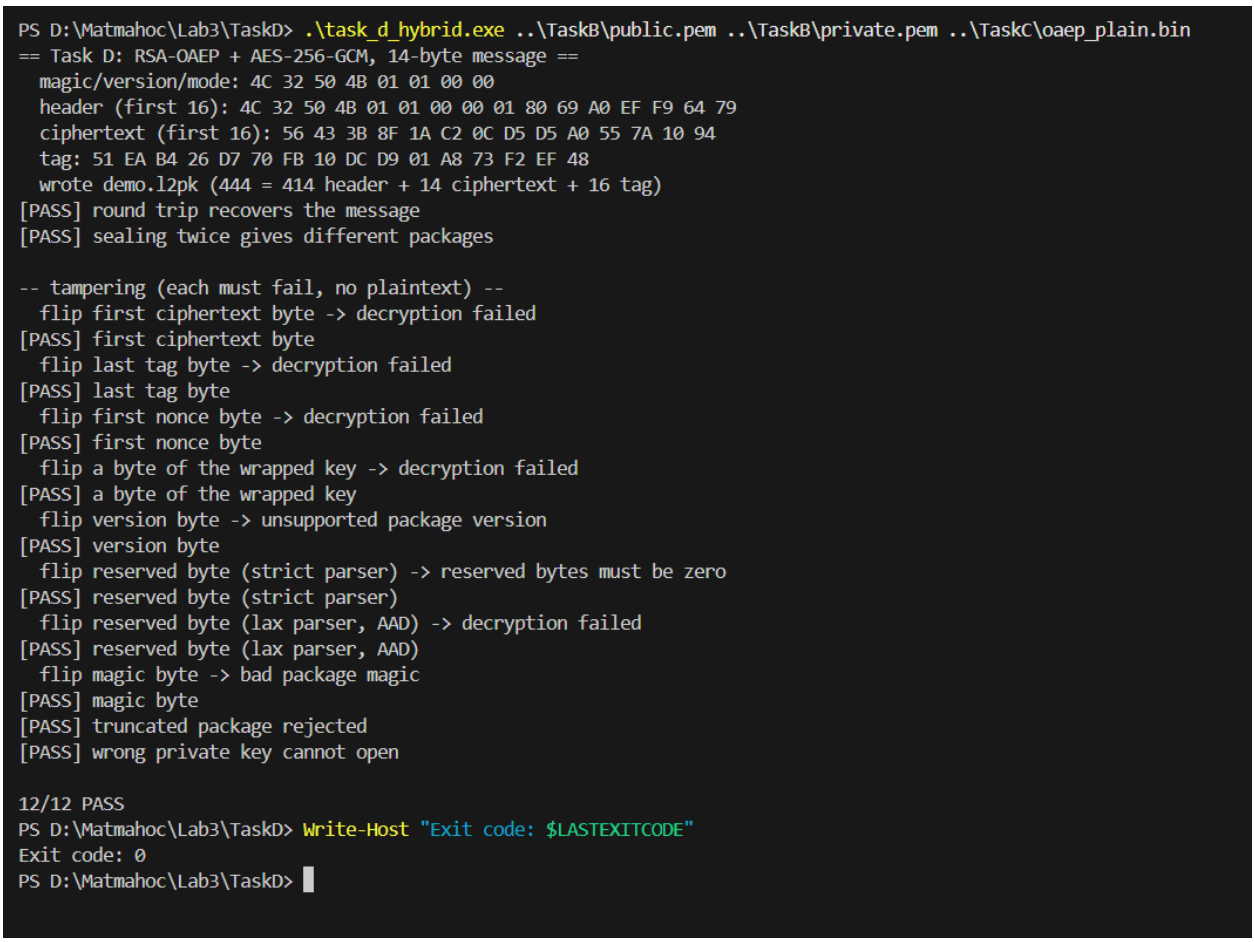
Kiểm thử tổng hợp Task D

```
PS D:\Matmahoc\Lab3> $env:Path = "C:\msys64\mingw64\bin;$env:Path"
PS D:\Matmahoc\Lab3> cd TaskD
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe --all
== Task D: RSA-OAEP + AES-256-GCM, 48-byte message ==
  magic/version/mode: 4C 32 50 4B 01 01 00 00
  header (first 16): 4C 32 50 4B 01 01 00 00 01 80 B4 8D CF 2E B9 44
  ciphertext (first 16): 79 7E 3B CE 6E EE FB C0 70 14 F4 E0 67 9B 79 1
  tag: 4F 2E BE D6 B6 0A E3 0B 1B C2 93 D0 47 31 62 1B
  wrote demo.l2pk (478 = 414 header + 48 ciphertext + 16 tag)
[PASS] round trip recovers the message
[PASS] sealing twice gives different packages

-- tampering (each must fail, no plaintext) --
  flip first ciphertext byte -> decryption failed
[PASS] first ciphertext byte
  flip last tag byte -> decryption failed
[PASS] last tag byte
  flip first nonce byte -> decryption failed
[PASS] first nonce byte
  flip a byte of the wrapped key -> decryption failed
[PASS] a byte of the wrapped key
  flip version byte -> unsupported package version
[PASS] version byte
  flip reserved byte (strict parser) -> reserved bytes must be zero
[PASS] reserved byte (strict parser)
  flip reserved byte (lax parser, AAD) -> decryption failed
[PASS] reserved byte (lax parser, AAD)
  flip magic byte -> bad package magic
[PASS] magic byte
[PASS] truncated package rejected
[PASS] wrong private key cannot open

12/12 PASS
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Exit code: $LASTEXITCODE"
Exit code: 0
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 58. Kiểm thử cơ chế mã hóa lai RSA-OAEP-SHA256 kết hợp AES-256-GCM. Chương trình hoàn thành 12/12 phép thử, bao gồm kiểm tra tính toàn vẹn, tính ngẫu nhiên và khả năng từ chối dữ liệu không hợp lệ.



Hình 59. Kiểm thử cơ chế mã hóa lai RSA-OAEP và AES-256-GCM bằng cặp khóa RSA được tạo từ Task B. Chương trình hoàn thành 12/12 phép kiểm thử, khôi phục chính xác bản rõ và trả về mã thoát 0.

Phân tích định dạng L2PK

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== PACKAGE SIZE ==="
=== PACKAGE SIZE ===
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\demo.l2pk | Select-Object Name, Length

Name      Length
----      ------
demo.l2pk    444


PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== PACKAGE HEADER ==="
=== PACKAGE HEADER ===
PS D:\Matmahoc\Lab3\TaskD> Format-Hex .\demo.l2pk | Select-Object -First 4


           Path: D:\Matmahoc\Lab3\TaskD\demo.l2pk

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000000   4C 32 50 4B 01 01 00 00 01 80 69 A0 EF F9 64 79  L2PK.....i ïùdy
00000010   E1 92 76 44 3B 44 52 E8 0D 62 89 2A E1 BC 40 AF  ávD;DRè.b*á¼@¯
00000020   CF 4F A0 10 C4 5C 4C 84 C9 E9 C7 B9 A0 BB 7F 4C  ÏO .Ä\LÉéÇ¹ »L
00000030   29 78 53 55 31 02 F1 51 A5 FC 64 D6 53 33 3F 18  )xSU1.ñQ¥üdÖS3?.


PS D:\Matmahoc\Lab3\TaskD>
```

Hình 60. Quan sát kích thước và cấu trúc phần đầu của gói dữ liệu demo.l2pk. Mười byte đầu thể hiện mã nhận dạng L2PK, phiên bản 1, chế độ RSA-OAEP, trường reserved bằng 0 và độ dài wrapped key là 384 byte.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== NONCE, CT_LEN, CIPHERTEXT, TAG ==="
=== NONCE, CT_LEN, CIPHERTEXT, TAG ===
PS D:\Matmahoc\Lab3\TaskD> Format-Hex .\demo.l2pk |
>> Select-Object -Skip 24 -First 4


           Path: D:\Matmahoc\Lab3\TaskD\demo.l2pk

           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000180   AF 65 52 C3 A2 95 C3 1A 8F 10 57 4D 82 44 26 53  ¯eRÃ¢Ã..WMD&S
00000190   2A 0E A8 23 19 5B 00 00 00 00 00 00 00 0E 56 43  *.¨#.[........VC
000001A0   3B 8F 1A C2 0C D5 D5 A0 55 7A 10 94 51 EA B4 26  ;.Â.ÕÕ Uz.Qê´&
000001B0   D7 70 FB 10 DC D9 01 A8 73 F2 EF 48              xpû.ÜÙ.¨sòïH


PS D:\Matmahoc\Lab3\TaskD>
```

Hình 61. Phân tích dữ liệu nhị phân của gói RSA-OAEP kết hợp AES-256-GCM tại các offset cuối. Kết quả xác định nonce dài 12 byte, trường ct_len biểu diễn giá trị 14, ciphertext dài 14 byte và authentication tag dài 16 byte, phù hợp với định dạng L2PK phiên bản 1.

```
PS D:\Matmahoc\Lab3\TaskD> $b = [System.IO.File]::ReadAllBytes(
>> (Resolve-Path .\demo.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> function Show-Hex([byte[]]$data) {
>> ($data | ForEach-Object { $_.ToString("X2") }) -join " "
>> }
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== PACKAGE FIELD ANALYSIS ==="
=== PACKAGE FIELD ANALYSIS ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Nonce (12 bytes):"
Nonce (12 bytes):
PS D:\Matmahoc\Lab3\TaskD> Show-Hex ($b[394..405])
57 4D 82 44 26 53 2A 0E A8 23 19 5B
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Ciphertext length (8 bytes):"
Ciphertext length (8 bytes):
PS D:\Matmahoc\Lab3\TaskD> Show-Hex ($b[406..413])
00 00 00 00 00 00 00 0E
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Ciphertext (14 bytes):"
Ciphertext (14 bytes):
PS D:\Matmahoc\Lab3\TaskD> Show-Hex ($b[414..427])
56 43 3B 8F 1A C2 0C D5 D5 A0 55 7A 10 94
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Authentication tag (16 bytes):"
Authentication tag (16 bytes):
PS D:\Matmahoc\Lab3\TaskD> Show-Hex ($b[428..443])
51 EA B4 26 D7 70 FB 10 DC D9 01 A8 73 F2 EF 48
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Total package bytes: $($b.Length)"
Total package bytes: 444
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 62. Trích xuất và kiểm chứng các trường nonce, ciphertext length, ciphertext và authentication tag trong gói dữ liệu L2PK. Các trường được đọc theo offset và độ dài quy định, với tổng kích thước gói là 444 byte.

D-T1. Mã hóa và giải mã các loại tệp

```
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>> (Join-Path (Get-Location).Path "D-T1_empty.bin"),
>> [byte[]]@()
>> )
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T1_empty.bin | Select-Object Name, Length

Name           Length
----           ------
D-T1_empty.bin      0


PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\D-T1_empty.bin `
>>     --out .\D-T1_empty.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T1_empty.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T1_empty.l2pk `
>>     --out .\D-T1_empty_recovered.bin
DECRYPT OK | header=414 bytes | output=.\D-T1_empty_recovered.bin
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T1_empty.bin, .\D-T1_empty.l2pk, .\D-T1_empty_recovered.bin |
>> Select-Object Name, Length

Name                   Length
----                   ------
D-T1_empty.bin              0
D-T1_empty.l2pk           430
D-T1_empty_recovered.bin    0


PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\D-T1_empty.bin -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T1_empty_recovered.bin -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Plaintext identical: $($h1 -eq $h2)"
Plaintext identical: True
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 63. Kiểm thử mã hóa và giải mã tệp rỗng bằng RSA-OAEP kết hợp AES-256-GCM. Gói dữ liệu có kích thước 430 byte, quá trình mã hóa và giải mã đều trả về mã thoát 0. Tệp khôi phục có kích thước 0 byte và giá trị băm SHA-256 trùng với tệp ban đầu.

```
PS D:\Matmahoc\Lab3\TaskD> $message = "Thử nghiệm Task D: mã hóa và giải mã tệp văn bản UTF-8."
PS D:\Matmahoc\Lab3\TaskD> [System.IO.File]::WriteAllText(
>>     (Join-Path (Get-Location).Path "D-T1_text.txt"),
>>     $message,
>>     [System.Text.UTF8Encoding]::new($false)
>> )
PS D:\Matmahoc\Lab3\TaskD> Get-Content .\D-T1_text.txt -Encoding UTF8
Thử nghiệm Task D: mã hóa và giải mã tệp văn bản UTF-8.
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T1_text.txt | Select-Object Name, Length

Name          Length
----          ------
D-T1_text.txt     70


PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\D-T1_text.txt `
>>     --out .\D-T1_text.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T1_text.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T1_text.l2pk `
>>     --out .\D-T1_text_recovered.txt
DECRYPT OK | header=414 bytes | output=.\D-T1_text_recovered.txt
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== RECOVERED TEXT ==="
=== RECOVERED TEXT ===
PS D:\Matmahoc\Lab3\TaskD> Get-Content .\D-T1_text_recovered.txt -Encoding UTF8
Thử nghiệm Task D: mã hóa và giải mã tệp văn bản UTF-8.
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== SHA256 VERIFICATION ==="
=== SHA256 VERIFICATION ===
PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\D-T1_text.txt -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T1_text_recovered.txt -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original SHA256: $h1"
Original SHA256: 37BD550AFDCC12F04357E9B1B7138EECEF767E49D38C1B91B17CE025B33F1D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Recovered SHA256: $h2"
Recovered SHA256: 37BD550AFDCC12F04357E9B1B7138EECEF767E49D38C1B91B17CE025B33F1D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Plaintext identical: $($h1 -eq $h2)"
Plaintext identical: True
```

Hình 64. Kiểm thử mã hóa và giải mã tệp văn bản UTF-8 có dấu tiếng Việt bằng RSA-OAEP kết hợp AES-256-GCM. Hai thao tác đều trả về mã thoát 0; nội dung khôi phục chính xác và giá trị SHA-256 trùng khớp với tệp gốc.

```
PS D:\Matmahoc\Lab3\TaskD> Get-ChildItem D:\Matmahoc\Lab3 -Recurse -File -Filter *.pdf |
>> Select-Object FullName, Length

FullName                                    Length
--------                                    ------
D:\Matmahoc\Lab3\25521679_BaoCaoDoAnGame.pdf 1138881


PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in ..\25521679_BaoCaoDoAnGame.pdf `
>>     --out .\D-T1_pdf.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T1_pdf.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T1_pdf.l2pk `
>>     --out .\D-T1_pdf_recovered.pdf
DECRYPT OK | header=414 bytes | output=.\D-T1_pdf_recovered.pdf
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Get-Item ..\25521679_BaoCaoDoAnGame.pdf, .\D-T1_pdf.l2pk, .\D-T1_pdf_recovered.pdf |
>> Select-Object Name, Length

Name                          Length
----                          ------
25521679_BaoCaoDoAnGame.pdf 1138881
D-T1_pdf.l2pk               1139311
D-T1_pdf_recovered.pdf      1138881


PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash ..\25521679_BaoCaoDoAnGame.pdf -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T1_pdf_recovered.pdf -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original SHA256: $h1"
Original SHA256: 5A7F3AF88F0E0716E991C354E996B913B55465382228FCD672621005BBEA3187
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Recovered SHA256: $h2"
Recovered SHA256: 5A7F3AF88F0E0716E991C354E996B913B55465382228FCD672621005BBEA3187
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Plaintext identical: $($h1 -eq $h2)"
Plaintext identical: True
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 65. Kiểm thử mã hóa và giải mã tệp PDF có kích thước 1.138.881 byte bằng RSA-OAEP kết hợp AES-256-GCM. Hai thao tác đều thành công với mã thoát 0; tệp khôi phục có kích thước và giá trị SHA-256 trùng khớp với tệp gốc.

```
PS D:\Matmahoc\Lab3\TaskD> Compress-Archive `
>>     -LiteralPath .\D-T1_text.txt, ..\25521679_BaoCaoDoAnGame.pdf `
>>     -DestinationPath .\D-T1_sample.zip `
>>     -Force
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T1_sample.zip |
>>     Select-Object Name, Length

Name             Length
----             ------
D-T1_sample.zip 1047567
```

Hình 66. Tạo tệp ZIP dùng cho kiểm thử mã hóa và giải mã.

```
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\D-T1_sample.zip `
>>     --out .\D-T1_zip.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T1_zip.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T1_zip.l2pk `
>>     --out .\D-T1_zip_recovered.zip
DECRYPT OK | header=414 bytes | output=.\D-T1_zip_recovered.zip
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== FILE SIZES ==="
=== FILE SIZES ===
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T1_sample.zip, .\D-T1_zip.l2pk, .\D-T1_zip_recovered.zip |
>>     Select-Object Name, Length

Name                  Length
----                  ------
D-T1_sample.zip      1047567
D-T1_zip.l2pk        1047997
D-T1_zip_recovered.zip 1047567


PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== SHA256 VERIFICATION ==="
=== SHA256 VERIFICATION ===
PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\D-T1_sample.zip -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T1_zip_recovered.zip -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original SHA256: $h1"
Original SHA256: E11498213D57EB2C0E55874DE315C002E8595E289F8257465918C566C8100D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Recovered SHA256: $h2"
Recovered SHA256: E11498213D57EB2C0E55874DE315C002E8595E289F8257465918C566C8100D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "ZIP identical: $($h1 -eq $h2)"
ZIP identical: True
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 67. Kiểm thử mã hóa và giải mã tệp ZIP bằng RSA-OAEP kết hợp AES-256-GCM. Gói dữ liệu có kích thước lớn hơn tệp gốc 430 byte; hai thao tác đều trả về mã thoát 0 và giá trị SHA-256 của tệp khôi phục trùng khớp với tệp ban đầu.

```
PS D:\Matmahoc\Lab3\TaskD> Get-ChildItem D:\Matmahoc\Lab3 -Recurse -File |
>>     Where-Object { $_.Extension -in ".png", ".jpg", ".jpeg" } |
>>     Select-Object FullName, Length

FullName                          Length
--------                          ------
D:\Matmahoc\Lab3\TaskD\Picture.png 432951
```

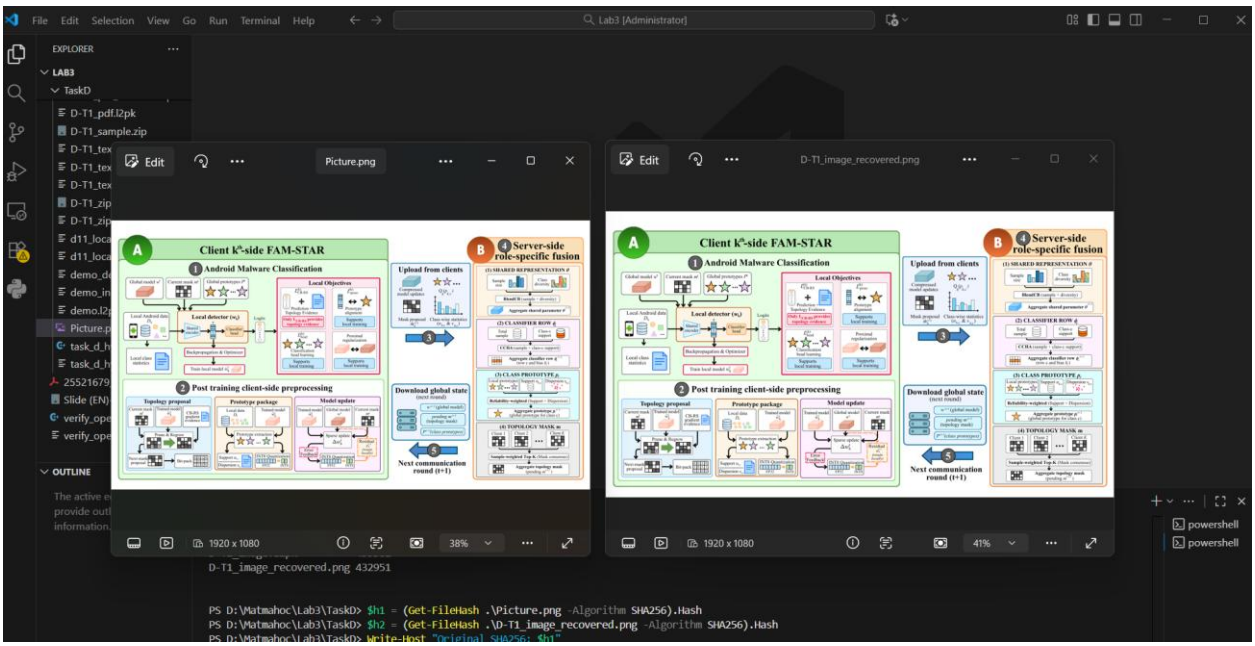
Hình 68. Xác định tệp ảnh PNG và kích thước dữ liệu đầu vào.

```
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\Picture.png `
>>     --out .\D-T1_image.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T1_image.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T1_image.l2pk `
>>     --out .\D-T1_image_recovered.png
DECRYPT OK | header=414 bytes | output=.\D-T1_image_recovered.png
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== FILE SIZES ==="
=== FILE SIZES ===
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\Picture.png, .\D-T1_image.l2pk, .\D-T1_image_recovered.png |
>>     Select-Object Name, Length

Name                     Length
----                     ------
Picture.png              432951
D-T1_image.l2pk          433381
D-T1_image_recovered.png 432951


PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\Picture.png -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T1_image_recovered.png -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original SHA256: $h1"
Original SHA256: 652CBFE7CF66EAEE7579240DF405B52C773C9766E21671746CADD5255D56C211
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Recovered SHA256: $h2"
Recovered SHA256: 652CBFE7CF66EAEE7579240DF405B52C773C9766E21671746CADD5255D56C211
PS D:\Matmahoc\Lab3\TaskD> Write-Host "PNG identical: $($h1 -eq $h2)"
PNG identical: True
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 69. Kiểm thử mã hóa và giải mã tệp ảnh PNG bằng RSA-OAEP kết hợp AES-256-GCM. Hai thao tác thành công với mã thoát 0; ảnh khôi phục có kích thước và giá trị SHA-256 trùng khớp hoàn toàn với ảnh gốc.



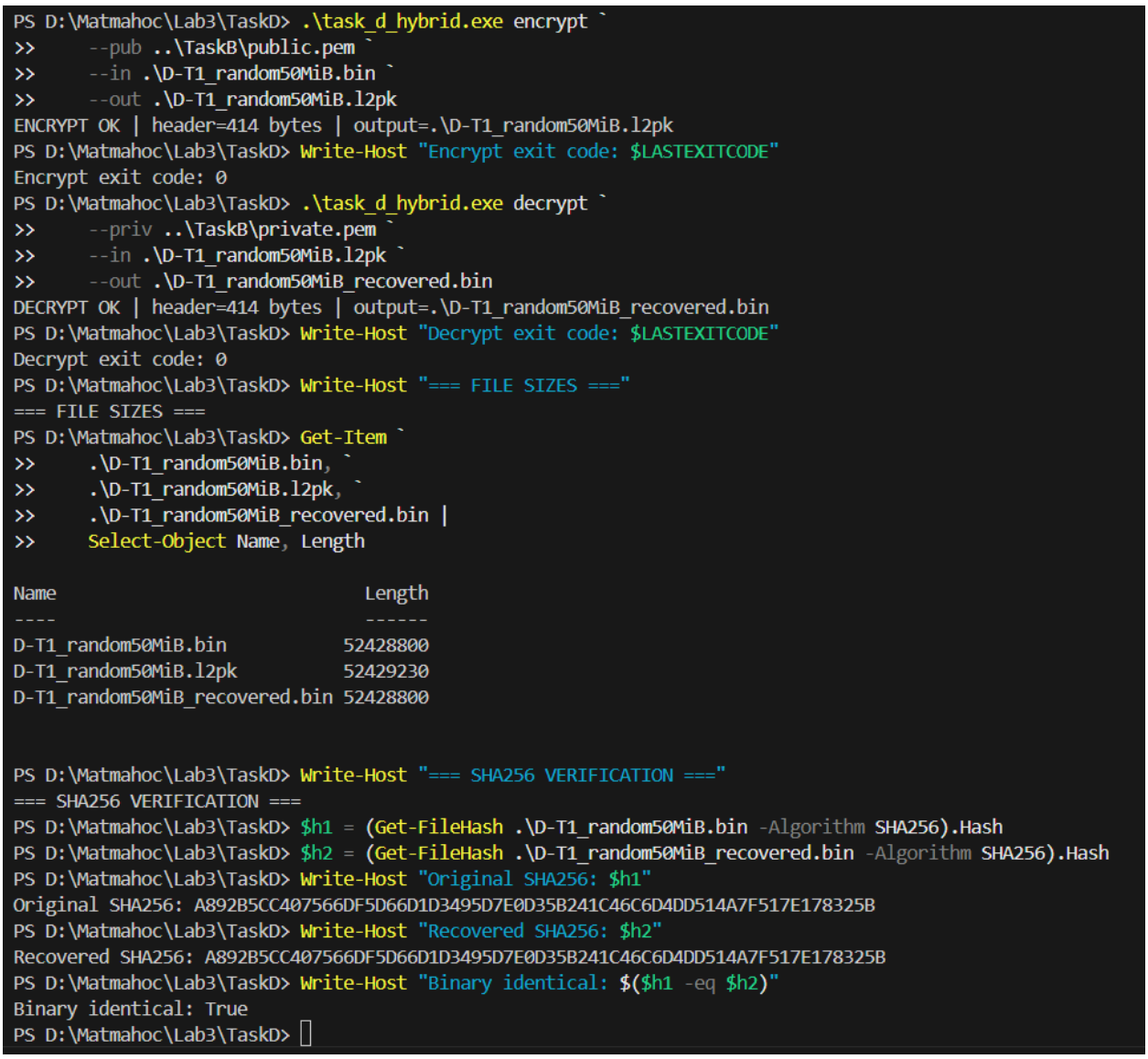
Hình 70. Đối chiếu trực quan ảnh PNG gốc và ảnh được khôi phục sau khi giải mã bằng RSA-OAEP kết hợp AES-256-GCM. Hai ảnh hiển thị cùng nội dung; kết quả so sánh SHA-256 xác nhận dữ liệu được bảo toàn chính xác từng byte.

```
PS D:\Matmahoc\Lab3\TaskD> $path = Join-Path (Get-Location).Path "D-T1_random50MiB.bin"
PS D:\Matmahoc\Lab3\TaskD> $rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
PS D:\Matmahoc\Lab3\TaskD> $stream = $null
PS D:\Matmahoc\Lab3\TaskD> try {
>>     $stream = [System.IO.File]::Create($path)
>>     $buffer = New-Object byte[] 65536
>>
>>     for ($i = 0; $i -lt 800; $i++) {
>>         $rng.GetBytes($buffer)
>>         $stream.Write($buffer, 0, $buffer.Length)
>>     }
>> }
>> finally {
>>     if ($null -ne $stream) {
>>         $stream.Dispose()
>>         $stream = $null
>>     }
>>
>>     $rng.Dispose()
>>     $rng = $null
>> }
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T1_random50MiB.bin |
>>     Select-Object Name, Length

Name                 Length
----                 ------
D-T1_random50MiB.bin 52428800

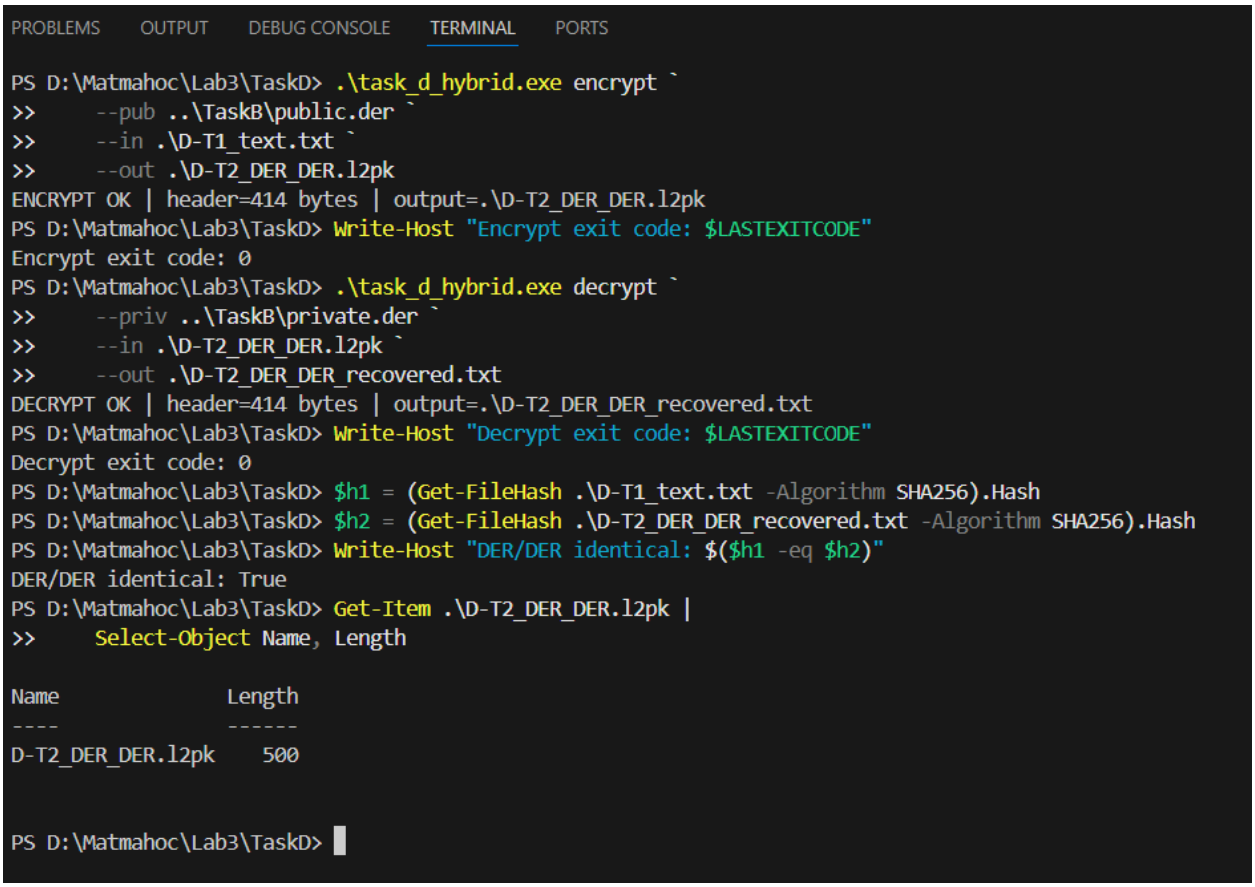

PS D:\Matmahoc\Lab3\TaskD>
```

Hình 71. Tạo tệp nhị phân ngẫu nhiên dung lượng 50 MiB.



Hình 72. Kiểm thử mã hóa và giải mã tệp nhị phân ngẫu nhiên dung lượng 50 MiB bằng RSA-OAEP kết hợp AES-256-GCM. Chương trình trả về mã thoát 0 ở cả hai thao tác; tệp khôi phục có kích thước 52.428.800 byte và giá trị SHA-256 trùng khớp với tệp gốc.

D-T2. Ma trận khóa PEM và DER



Hình 73. Kiểm thử mã hóa và giải mã bằng cặp khóa RSA định dạng DER. Chương trình hoàn thành hai thao tác với mã thoát 0, tạo package 500 byte và khôi phục dữ liệu có giá trị SHA-256 trùng khớp với bản rõ ban đầu.

```
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\D-T1_text.txt `
>>     --out .\D-T2_PEM_DER.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T2_PEM_DER.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.der `
>>     --in .\D-T2_PEM_DER.l2pk `
>>     --out .\D-T2_PEM_DER_recovered.txt
DECRYPT OK | header=414 bytes | output=.\D-T2_PEM_DER_recovered.txt
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\D-T1_text.txt -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T2_PEM_DER_recovered.txt -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original SHA256: $h1"
Original SHA256: 37BD550AFDCC12F04357E9B1B7138EECEF767E49D38C1B91B17CE025B33F1D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Recovered SHA256: $h2"
Recovered SHA256: 37BD550AFDCC12F04357E9B1B7138EECEF767E49D38C1B91B17CE025B33F1D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "PEM/DER identical: $($h1 -eq $h2)"
PEM/DER identical: True
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T2_PEM_DER.l2pk |
>>     Select-Object Name, Length

Name              Length
----              ------
D-T2_PEM_DER.l2pk    500


PS D:\Matmahoc\Lab3\TaskD>
```

Hình 74. Kiểm thử mã hóa bằng khóa công khai RSA định dạng PEM và giải mã bằng khóa bí mật định dạng DER. Chương trình trả về mã thoát 0 cho cả hai thao tác, khôi phục bản rõ có giá trị SHA-256 trùng khớp với dữ liệu gốc.

```
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.der `
>>     --in .\D-T1_text.txt `
>>     --out .\D-T2_DER_PEM.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T2_DER_PEM.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T2_DER_PEM.l2pk `
>>     --out .\D-T2_DER_PEM_recovered.txt
DECRYPT OK | header=414 bytes | output=.\D-T2_DER_PEM_recovered.txt
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\D-T1_text.txt -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T2_DER_PEM_recovered.txt -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original SHA256: $h1"
Original SHA256: 37BD550AFDCC12F04357E9B1B7138EECEF767E49D38C1B91B17CE025B33F1D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Recovered SHA256: $h2"
Recovered SHA256: 37BD550AFDCC12F04357E9B1B7138EECEF767E49D38C1B91B17CE025B33F1D1D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "DER/PEM identical: $($h1 -eq $h2)"
DER/PEM identical: True
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T2_DER_PEM.l2pk |
>>     Select-Object Name, Length

Name              Length
----              ------
D-T2_DER_PEM.l2pk    500


PS D:\Matmahoc\Lab3\TaskD>
```

Hình 75. Kiểm thử mã hóa bằng khóa công khai RSA định dạng DER và giải mã bằng khóa bí mật định dạng PEM. Chương trình thực hiện thành công hai thao tác với mã thoát 0, đồng thời khôi phục dữ liệu có giá trị SHA-256 trùng khớp với bản rõ.

D-T3. Tính ngẫu nhiên của gói dữ liệu

```
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\D-T1_text.txt `
>>     --out .\D-T3_first.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T3_first.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt 1 exit code: $LASTEXITCODE"
Encrypt 1 exit code: 0
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\D-T1_text.txt `
>>     --out .\D-T3_second.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T3_second.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt 2 exit code: $LASTEXITCODE"
Encrypt 2 exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== PACKAGE SIZES ==="
=== PACKAGE SIZES ===
PS D:\Matmahoc\Lab3\TaskD> Get-Item .\D-T3_first.l2pk, .\D-T3_second.l2pk |
>>     Select-Object Name, Length

Name             Length
----             ------
D-T3_first.l2pk     500
D-T3_second.l2pk    500


PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== SHA256 COMPARISON ==="
=== SHA256 COMPARISON ===
PS D:\Matmahoc\Lab3\TaskD> $h1 = (Get-FileHash .\D-T3_first.l2pk -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> $h2 = (Get-FileHash .\D-T3_second.l2pk -Algorithm SHA256).Hash
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package 1 SHA256: $h1"
Package 1 SHA256: 922433E943EFB9521009FFE6DD36764466E732EBD2360506148FAF692F84E5C2
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package 2 SHA256: $h2"
Package 2 SHA256: ED69A79945F162904C317983D8E65458F695BA13A1DAD3ECC9D8C254BC649E15
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Packages identical: $($h1 -eq $h2)"
Packages identical: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 76. Mã hóa cùng một tệp hai lần và đối chiếu kích thước, SHA-256 của hai gói dữ liệu.

```
PS D:\Matmahoc\Lab3\TaskD> $b1 = [IO.File]::ReadAllBytes((Resolve-Path .\D-T3_first.l2pk).Path)
PS D:\Matmahoc\Lab3\TaskD> $b2 = [IO.File]::ReadAllBytes((Resolve-Path .\D-T3_second.l2pk).Path)
PS D:\Matmahoc\Lab3\TaskD> $n1 = [BitConverter]::ToString($b1, 394, 12)
PS D:\Matmahoc\Lab3\TaskD> $n2 = [BitConverter]::ToString($b2, 394, 12)
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== GCM NONCE COMPARISON ==="
=== GCM NONCE COMPARISON ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Nonce 1: $n1"
Nonce 1: 67-37-47-09-19-17-F8-05-C2-AA-2F-5C
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Nonce 2: $n2"
Nonce 2: 9A-96-1C-6C-A6-2A-14-BD-97-E1-32-23
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Nonces identical: $($n1 -eq $n2)"
Nonces identical: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 77. Kiểm chứng tính ngẫu nhiên của cơ chế mã hóa lai RSA-OAEP kết hợp AES-256-GCM. Hai lần mã hóa cùng một tệp bằng cùng khóa công khai tạo ra các package dài 500 byte nhưng có giá trị SHA-256 và nonce khác nhau.

D-T4. Thay đổi ciphertext

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 414
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = $modified[$offset] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T4_tampered.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T4 CIPHERTEXT TAMPERING ==="
=== D-T4 CIPHERTEXT TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 414 (0x19E)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
Original byte: 1B
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
Modified byte: 1A
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 78. Thay đổi một bit trong ciphertext

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Test-Path .\D-T4_recovered.bin
False
PS D:\Matmahoc\Lab3\TaskD> Test-Path .\D-T4_recovered.bin.part
False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T4_tampered.l2pk `
>>     --out .\D-T4_recovered.bin
Error: decryption failed
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 4
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists:"
Output exists:
PS D:\Matmahoc\Lab3\TaskD> Test-Path .\D-T4_recovered.bin
False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary file exists:"
Temporary file exists:
PS D:\Matmahoc\Lab3\TaskD> Test-Path .\D-T4_recovered.bin.part
False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 79. Kiểm thử thay đổi một bit trong ciphertext của gói L2PK. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp đầu ra hoặc để lại tệp tạm, bảo đảm dữ liệu chưa được xác thực không bị công bố.

D-T5

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = $modified.Length - 1
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = $modified[$offset] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T5_tampered_tag.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T5 AUTHENTICATION TAG TAMPERING ==="
=== D-T5 AUTHENTICATION TAG TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 499 (0x1F3)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
Original byte: 86
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
Modified byte: 87
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 80. Thay đổi một bit trong authentication tag của gói L2PK.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T5_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T5_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T5_tampered_tag.l2pk `
>>     --out .\D-T5_recovered.bin
Error: decryption failed
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 4
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T5_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T5_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 81. Kiểm thử thay đổi một bit trong authentication tag của gói dữ liệu L2PK. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp bản rõ hoặc để lại tệp tạm, bảo đảm dữ liệu chưa được xác thực không bị công bố.

D-T6

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 394
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = $modified[$offset] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T6_tampered_nonce.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T6.1 NONCE TAMPERING ==="
=== D-T6.1 NONCE TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 394 (0x18A)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
Original byte: 67
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
Modified byte: 66
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 82. Thay đổi một bit trong nonce AES-GCM của gói L2PK.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T6_nonce_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T6_nonce_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T6_tampered_nonce.l2pk `
>>     --out .\D-T6_nonce_recovered.bin
Error: decryption failed
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 4
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T6_nonce_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T6_nonce_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 83. Kiểm thử thay đổi một bit trong nonce AES-GCM của gói L2PK. Chương trình từ chối giải mã với mã thoát 4, đồng thời không tạo tệp bản rõ hoặc để lại tệp tạm.

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 15
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = $modified[$offset] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T6_tampered_wrapped_key.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T6.2 WRAPPED KEY TAMPERING ==="
=== D-T6.2 WRAPPED KEY TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 15 (0xF)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
Original byte: 2C
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
Modified byte: 2D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 84. Thay đổi một bit trong trường wrapped key RSA-OAEP.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T6_wrapped_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T6_wrapped_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T6_tampered_wrapped_key.l2pk `
>>     --out .\D-T6_wrapped_recovered.bin
Error: decryption failed
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 4
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T6_wrapped_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T6_wrapped_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 85. Kiểm thử thay đổi một bit tại offset 15 trong trường wrapped key RSA-OAEP. Chương trình từ chối giải mã với mã thoát 4, không tạo tệp bản rõ hoặc để lại tệp tạm, đồng thời sử dụng thông báo lỗi thống nhất với trường hợp xác thực GCM thất bại.

D-T7

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 7
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = $modified[$offset] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T7_bad_reserved.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T7.1 RESERVED FIELD TAMPERING ==="
=== D-T7.1 RESERVED FIELD TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 7 (0x7)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
Original byte: 00
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
Modified byte: 01
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD>
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T7_reserved_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T7_reserved_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T7_bad_reserved.l2pk `
>>     --out .\D-T7_reserved_recovered.bin
Error: reserved bytes must be zero
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T7_reserved_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T7_reserved_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 86. Kiểm thử trường reserved không hợp lệ trong header L2PK. Chương trình phát hiện giá trị reserved khác 0, từ chối giải mã với mã thoát 3 và không tạo tệp đầu ra hoặc để lại tệp tạm.

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 4
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = 0x02
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T7_bad_version.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T7.2 VERSION TAMPERING ==="
=== D-T7.2 VERSION TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 4 (0x4)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original version: {0:X2}" -f $original[$offset])
Original version: 01
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified version: {0:X2}" -f $modified[$offset])
Modified version: 02
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD> ^C
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T7_version_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T7_version_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T7_bad_version.l2pk `
>>     --out .\D-T7_version_recovered.bin
Error: unsupported package version
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T7_version_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T7_version_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 87. Kiểm thử trường version không hợp lệ trong header L2PK. Chương trình phát hiện phiên bản 0x02 không được hỗ trợ, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm.

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 0
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = $modified[$offset] -bxor 0x01
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T7_bad_magic.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T7.3 MAGIC TAMPERING ==="
=== D-T7.3 MAGIC TAMPERING ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
Offset: 0 (0x0)
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
Original byte: 4C
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
Modified byte: 4D
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original magic: $([Text.Encoding]::ASCII.GetString($original, 0, 4))"
Original magic: L2PK
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Modified magic: $([Text.Encoding]::ASCII.GetString($modified, 0, 4))"
Modified magic: M2PK
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Package size: $($modified.Length) bytes"
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T7_magic_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T7_magic_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T7_bad_magic.l2pk `
>>     --out .\D-T7_magic_recovered.bin
Error: bad package magic
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T7_magic_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T7_magic_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 88. Kiểm thử mã nhận dạng Magic không hợp lệ trong header L2PK. Sau khi thay đổi Magic từ L2PK thành M2PK, chương trình từ chối giải mã với mã thoát 3 và không tạo tệp đầu ra hoặc để lại tệp tạm.

D-T8

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $truncated = [byte[]]$original[0..($original.Length - 2)]
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T8_truncated.l2pk"),
>>     $truncated
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T8.1 TRUNCATED PACKAGE ==="
=== D-T8.1 TRUNCATED PACKAGE ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Original size: $($original.Length) bytes"
Original size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Truncated size: $($truncated.Length) bytes"
Truncated size: 499 bytes
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Bytes removed: $($original.Length - $truncated.Length)"
Bytes removed: 1
PS D:\Matmahoc\Lab3\TaskD> Write-Host ("Last original byte: {0:X2}" -f $original[-1])
Last original byte: 86
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
=== BEFORE DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T8_truncated_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T8_truncated_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T8_truncated.l2pk `
>>     --out .\D-T8_truncated_recovered.bin
Error: package total length mismatch
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
=== AFTER DECRYPTION ===
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Output exists: $(Test-Path .\D-T8_truncated_recovered.bin)"
Output exists: False
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Temporary exists: $(Test-Path .\D-T8_truncated_recovered.bin.part)"
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 89. Kiểm thử gói L2PK bị cắt bỏ một byte cuối của authentication tag. Chương trình phát hiện kích thước không hợp lệ, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm.

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $extended = [byte[]]::new($original.Length + 1)
PS D:\Matmahoc\Lab3\TaskD> [Array]::Copy($original, $extended, $original.Length)
PS D:\Matmahoc\Lab3\TaskD> $extended[$original.Length] = 0xA5
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T8_extra_byte.l2pk"),
>>     $extended
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T8.2 EXTRA BYTE TEST ==="
>> Write-Host "Original size: $($original.Length) bytes"
>> Write-Host "Modified size: $($extended.Length) bytes"
>> Write-Host "Bytes added: $($extended.Length - $original.Length)"
>> Write-Host ("Added byte: {0:X2}" -f $extended[-1])
=== D-T8.2 EXTRA BYTE TEST ===
Original size: 500 bytes
Modified size: 501 bytes
Bytes added: 1
Added byte: A5
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 90. Thêm byte 0xA5 vào cuối gói dữ liệu L2PK.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T8_extra_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T8_extra_recovered.bin.part)"
=== BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T8_extra_byte.l2pk `
>>     --out .\D-T8_extra_recovered.bin
Error: package total length mismatch
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T8_extra_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T8_extra_recovered.bin.part)"
=== AFTER DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 91. Kiểm thử gói L2PK có một byte dư ở cuối. Sau khi thêm byte 0xA5 khiến kích thước tăng từ 500 lên 501 byte, chương trình phát hiện lỗi không khớp tổng độ dài package, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm.

D-T8 (tiếp theo)

```
PS D:\Matmahoc\Lab3\TaskD> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\D-T3_first.l2pk).Path
>> )
PS D:\Matmahoc\Lab3\TaskD> $modified = [byte[]]$original.Clone()
PS D:\Matmahoc\Lab3\TaskD> $offset = 406
PS D:\Matmahoc\Lab3\TaskD> $modified[$offset] = 0x80
PS D:\Matmahoc\Lab3\TaskD> for ($i = 1; $i -lt 8; $i++) {
>>     $modified[$offset + $i] = 0x00
>> }
>>
PS D:\Matmahoc\Lab3\TaskD> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "D-T8_huge_ctlen.l2pk"),
>>     $modified
>> )
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T8.3 HUGE CT_LEN TEST ==="
>>
>> Write-Host "Original ct_len:"
>> Write-Host ([BitConverter]::ToString($original, 406, 8))
>>
>> Write-Host "Modified ct_len:"
>> Write-Host ([BitConverter]::ToString($modified, 406, 8))
>>
>> Write-Host "Package size: $($modified.Length) bytes"
=== D-T8.3 HUGE CT_LEN TEST ===
Original ct_len:
00-00-00-00-00-00-00-46
Modified ct_len:
80-00-00-00-00-00-00-00
Package size: 500 bytes
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 92. Đặt trường ct_len thành giá trị 2^63 trong gói L2PK.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T8_huge_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T8_huge_recovered.bin.part)"
=== BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T8_huge_ctlen.l2pk `
>>     --out .\D-T8_huge_recovered.bin
Error: ciphertext length exceeds 1 GiB limit
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T8_huge_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T8_huge_recovered.bin.part)"
=== AFTER DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 93. Kiểm thử trường ct_len có giá trị 2^63 trong header L2PK. Chương trình phát hiện độ dài ciphertext không phù hợp với kích thước package thực tế, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm.

D-T9

```
PS D:\Matmahoc\Lab3\TaskD> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" genpkey `
>>     -algorithm RSA `
>>     -pkeyopt rsa_keygen_bits:3072 `
>>     -out D-T9_wrong_private.pem
PS D:\Matmahoc\Lab3\TaskD> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" pkey `
>>     -in D-T9_wrong_private.pem `
>>     -check `
>>     -noout
Key is valid
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T9 WRONG PRIVATE KEY ==="
>>
>> Write-Host "Wrong private key exists: $(Test-Path .\D-T9_wrong_private.pem)"
>>
>> Write-Host "Wrong private key size: $((Get-Item .\D-T9_wrong_private.pem).Length) bytes"
=== D-T9 WRONG PRIVATE KEY ===
Wrong private key exists: True
Wrong private key size: 2524 bytes
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 94. Tạo khóa bí mật RSA 3072 bit không tương ứng để kiểm thử giải mã.

```
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T9_wrong_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T9_wrong_recovered.bin.part)"
=== BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv .\D-T9_wrong_private.pem `
>>     --in .\D-T3_first.l2pk `
>>     --out .\D-T9_wrong_recovered.bin
Error: decryption failed
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 4
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T9_wrong_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T9_wrong_recovered.bin.part)"
=== AFTER DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 95. Kiểm thử giải mã gói L2PK bằng khóa RSA Private không tương ứng với khóa Public mã hóa. Chương trình từ chối giải mã với mã thoát 4, không tạo tệp bản rõ hoặc để lại tệp tạm.

D-T10

```
PS D:\Matmahoc\Lab3\TaskD> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" genpkey `
>>      -algorithm RSA `
>>      -pkeyopt rsa_keygen_bits:2048 `
>>      -out D-T10_private_2048.pem
PS D:\Matmahoc\Lab3\TaskD> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" pkey `
>>      -in D-T10_private_2048.pem `
>>      -check `
>>      -noout
Key is valid
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== D-T10 RSA KEY SIZE MISMATCH ==="
>> Write-Host "Private key exists: $(Test-Path .\D-T10_private_2048.pem)"
=== D-T10 RSA KEY SIZE MISMATCH ===
Private key exists: True
PS D:\Matmahoc\Lab3\TaskD> & "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin\openssl.exe" pkey `
>>      -in D-T10_private_2048.pem `
>>      -text `
>>      -noout |
>>      Select-String "Private-Key:"

Private-Key: (2048 bit, 2 primes)


PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== BEFORE DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T10_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T10_recovered.bin.part)"
=== BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>      --priv .\D-T10_private_2048.pem `
>>      --in .\D-T3_first.l2pk `
>>      --out .\D-T10_recovered.bin
Error: klen does not match RSA modulus size
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 3
PS D:\Matmahoc\Lab3\TaskD> Write-Host "=== AFTER DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\D-T10_recovered.bin)"
>> Write-Host "Temporary exists: $(Test-Path .\D-T10_recovered.bin.part)"
=== AFTER DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 96. Kiểm thử giải mã package L2PK bằng khóa RSA-2048 trong khi trường klen khai báo 384 byte tương ứng RSA-3072. Chương trình phát hiện sự không khớp kích thước khóa, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm.

D-T11

```
● zhacutis1tg@fedora /run/media/zhacutis1tg/New Volume/Lab3-MMH ❯ ./task_d_hybrid --seal Thang_public.pem Ha_messages.txt from_Ha.l2pk
  [OK] Sealed 17 bytes into package from_Ha.l2pk
```

Hình 97. Hà thực hiện mã hóa tệp Ha_messages.txt bằng khóa công khai RSA của Thắng trên hệ điều hành Fedora. Chương trình tạo thành công gói tin from_Ha.l2pk từ dữ liệu bản rõ 17 byte.

```
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\D-T11\from_Ha.l2pk `
>>     --out .\D-T11\Ha_message_decrypted.txt
DECRYPT OK | header=414 bytes | output=.\D-T11\Ha_message_decrypted.txt
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Decrypt exit code: $LASTEXITCODE"
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD> Get-Content .\D-T11\Ha_message_decrypted.txt -Encoding UTF8
zha dep gai nhat
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 98. Giải mã thành công gói dữ liệu do nhóm Hà tạo bằng chương trình Task D của nhóm nhận. Quá trình sử dụng khóa bí mật RSA tương ứng, trả về mã thoát 0 và khôi phục nội dung bản rõ.

```
PS D:\Matmahoc\Lab3\TaskD> $message = "Toan Thang da nhan - ky ten: Meow Silver"
PS D:\Matmahoc\Lab3\TaskD> [System.IO.File]::WriteAllText(
>>     (Join-Path (Get-Location).Path "D-T11\from_Thang.txt"),
>>     $message,
>>     [System.Text.UTF8Encoding]::new($false)
>> )
PS D:\Matmahoc\Lab3\TaskD> Get-Content .\D-T11\from_Thang.txt
Toan Thang da nhan - ky ten: Meow Silver
PS D:\Matmahoc\Lab3\TaskD> .\task_d_hybrid.exe encrypt `
>>     --pub .\D-T11\Ha_public.pem `
>>     --in .\D-T11\from_Thang.txt `
>>     --out .\D-T11\from_Thang.l2pk
ENCRYPT OK | header=414 bytes | output=.\D-T11\from_Thang.l2pk
PS D:\Matmahoc\Lab3\TaskD> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskD>
```

Hình 99. Mã hóa thông điệp phản hồi bằng chương trình Task D và khóa công khai RSA của nhóm Hà. Chương trình tạo thành công gói from_Thang.l2pk theo định dạng L2PK phiên bản 1 và trả về mã thoát 0.

```
zhacutis1tg@fedora /run/media/zhacutis1tg/New Volume/Lab3-MMH ❯ ./task_d_hybrid --open private.pem from_Thang.l2pk Thang_messages.txt
[PASS] Package opened successfully! Extracted 40 bytes into Thang_messages.txt
zhacutis1tg@fedora /run/media/zhacutis1tg/New Volume/Lab3-MMH ❯ cat Thang_messages.txt; echo ""
Toan Thang da nhan - ky ten: Meow Silver
```

Hình 100. Hà thực hiện giải mã gói tin from_Thang.l2pk do Thắng tạo. Chương trình thông báo giải mã thành công, khôi phục 40 byte dữ liệu và hiển thị chính xác nội dung bản rõ ban đầu, chứng minh khả năng tương thích giữa hai chương trình.

Task E

1. Mã nguồn

Mã nguồn: src/task_e.cpp (phiên bản tích hợp CLI trong thư mục nộp bài).

```
#include "rsa_kem.hpp"
#include "key_io.hpp"
#include "task_cli.hpp"
#include "ossl_raii.hpp"
// Task E
#include <openssl/bn.h>
#include <openssl/core_names.h>
#include <openssl/crypto.h>
#include <openssl/err.h>
#include <openssl/evp.h>
#include <openssl/kdf.h>
#include <openssl/params.h>
#include <openssl/pem.h>
#include <openssl/rand.h>
#include <openssl/rsa.h>
#include <openssl/x509.h>


#include <algorithm>
#include <array>
#include <cstdint>
#include <cstring>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <memory>
#include <set>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>


namespace task_e {


namespace fs = std::filesystem;
using Bytes = std::vector<unsigned char>;
using PKey = std::unique_ptr<EVP_PKEY, decltype(&EVP_PKEY_free)>;
using PKeyCtx = std::unique_ptr<EVP_PKEY_CTX, decltype(&EVP_PKEY_CTX_free)>;
using CipherCtx = std::unique_ptr<EVP_CIPHER_CTX, decltype(&EVP_CIPHER_CTX_free)>;
using Bio = std::unique_ptr<BIO, decltype(&BIO_free)>;
using BN = std::unique_ptr<BIGNUM, decltype(&BN_clear_free)>;
using BNCtx = std::unique_ptr<BN_CTX, decltype(&BN_CTX_free)>;
using Kdf = std::unique_ptr<EVP_KDF, decltype(&EVP_KDF_free)>;
using KdfCtx = std::unique_ptr<EVP_KDF_CTX, decltype(&EVP_KDF_CTX_free)>;
using Pkcs8 = std::unique_ptr<PKCS8_PRIV_KEY_INFO, decltype(&PKCS8_PRIV_KEY_INFO_free)>;


constexpr std::size_t CHUNK = 64 * 1024;
constexpr std::uint64_t MAX_FILE = std::uint64_t{1} << 30;
constexpr std::size_t KEY_ID_LEN = 32;
constexpr std::size_t AES_KEY_LEN = 32;
constexpr std::size_t NONCE_LEN = 12;
constexpr std::size_t TAG_LEN = 16;
constexpr unsigned char VERSION = 1;
constexpr unsigned char MODE_KEM = 2;
constexpr char LABEL[] = "LAB02-RSASVE-HKDF-SHA256-AES256GCM-v1";
static_assert(sizeof(LABEL) - 1 == 37, "KDF label must be 37 ASCII bytes");


using LabError = lab::Error;
[[noreturn]] void fail(int code, const std::string& why) { throw LabError(code, why); }
std::string sslError() {
    unsigned long e = ERR_get_error();
    if (!e) return "OpenSSL operation failed";
    char text[256]{};
    ERR_error_string_n(e, text, sizeof(text));
    ERR_clear_error();
    return text;
}
void sslCheck(int ok, const char* operation) {
```

```
    if (ok <= 0) fail(5, std::string(operation) + ": " + sslError());
}


// Lưu dữ liệu bí mật và xóa khỏi bộ nhớ khi không còn sử dụng.


struct PlaintextBlock {
    std::array<unsigned char, CHUNK + 16> data{};
    ~PlaintextBlock() { OPENSSL_cleanse(data.data(), data.size()); }
};
bool equalSecret(const SecureBytes& a, const SecureBytes& b) {
    return a.value.size() == b.value.size() &&
           CRYPTO_memcmp(a.value.data(), b.value.data(), a.value.size()) == 0;
}
void require(bool ok, const std::string& why) { if (!ok) fail(5, why); }
// Chỉ nhận khóa RSA từ 2048 bit và có độ dài phù hợp với trường klen.
void checkRsa(EVP_PKEY* k) {
    if (!k || EVP_PKEY_is_a(k, "RSA") != 1 || EVP_PKEY_get_bits(k) < 2048 ||
        EVP_PKEY_get_size(k) <= 0 || EVP_PKEY_get_size(k) > 65535)
        fail(3, "RSA key must be at least 2048 bits and fit the L2PK length field");
}
std::uint64_t fileSize(const fs::path& path) {
    std::error_code ec;
    const auto n = fs::file_size(path, ec);
    if (ec) fail(2, "cannot read file size: " + path.string());
    return static_cast<std::uint64_t>(n);
}
void readExact(std::istream& in, unsigned char* p, std::size_t n) {
    if (n && !in.read(reinterpret_cast<char*>(p), static_cast<std::streamsize>(n)))
        fail(2, "cannot read file");
}
void writeExact(std::ostream& out, const unsigned char* p, std::size_t n) {
    if (n) out.write(reinterpret_cast<const char*>(p), static_cast<std::streamsize>(n));
    if (!out) fail(2, "cannot write file");
}
Bytes readSmall(const fs::path& path, std::size_t limit = 1024 * 1024) {
    auto n = fileSize(path);
    if (n > limit) fail(3, "file exceeds read limit");
    Bytes result(static_cast<std::size_t>(n));
    std::ifstream f(path, std::ios::binary);
    if (!f) fail(2, "cannot open file: " + path.string());
    readExact(f, result.data(), result.size());
    return result;
}
void writeSmall(const fs::path& path, const Bytes& bytes) {
    std::ofstream f(path, std::ios::binary | std::ios::trunc);
    if (!f) fail(2, "cannot create file: " + path.string());
    writeExact(f, bytes.data(), bytes.size());
    f.close();
    if (!f) fail(2, "cannot close file");
}


// Đọc khóa PEM hoặc DER, đồng thời kiểm tra dữ liệu thừa ở cuối tệp.
PKey loadKey(const fs::path& path, bool privatePart) {
    auto key=task_b::loadKey(path,privatePart);
    if(privatePart) task_b::validatePrivate(key.get()); else task_b::validatePublic(key.get());
    return key;
}
BN rsaParam(EVP_PKEY* key, const char* param) {
    BIGNUM* number = nullptr;
    if (EVP_PKEY_get_bn_param(key, param, &number) != 1 || !number)
        fail(5, std::string("cannot obtain RSA parameter: ") + param);
    return BN(number, BN_clear_free);
}
void putU16(Bytes& out, std::size_t n) {
    if (n > 65535) fail(3, "klen does not fit two bytes");
    out.push_back(static_cast<unsigned char>(n >> 8));
    out.push_back(static_cast<unsigned char>(n));
}
void putU64(Bytes& out, std::uint64_t n) {
    for (int s = 56; s >= 0; s -= 8) out.push_back(static_cast<unsigned char>(n >> s));
}
std::size_t getU16(const unsigned char* p) {
    return (std::size_t(p[0]) << 8) | p[1];
}
std::uint64_t getU64(const unsigned char* p) {
    std::uint64_t r = 0;
    for (int i = 0; i < 8; ++i) r = (r << 8) | p[i];
    return r;
}
```

```
// E-F1, E-F2: Đóng gói và khôi phục bí mật chung bằng RSASVE qua EVP.


Encapsulation encapsulate(EVP_PKEY* recipientPublicKey) {
    checkRsa(recipientPublicKey);
    PKeyCtx ctx(EVP_PKEY_CTX_new_from_pkey(nullptr, recipientPublicKey, nullptr), EVP_PKEY_CTX_free);
    if (!ctx) fail(5, "encapsulation context allocation: " + sslError());
    sslCheck(EVP_PKEY_encapsulate_init(ctx.get(), nullptr), "encapsulate init");
    sslCheck(EVP_PKEY_CTX_set_kem_op(ctx.get(), "RSASVE"), "RSASVE operation");
    std::size_t ctSize = 0, zSize = 0;
    sslCheck(EVP_PKEY_encapsulate(ctx.get(), nullptr, &ctSize, nullptr, &zSize), "query encapsulation
lengths");
    const auto k = static_cast<std::size_t>(EVP_PKEY_get_size(recipientPublicKey));
    if (ctSize != k || zSize != k) fail(5, "unexpected EVP RSASVE sizes");
    Encapsulation result{Bytes(ctSize), SecureBytes(zSize)};
    sslCheck(EVP_PKEY_encapsulate(ctx.get(), result.kem_ct.data(), &ctSize,
                                  result.shared_secret.value.data(), &zSize), "encapsulate");
    if (ctSize != k || zSize != k) fail(5, "incorrect actual EVP RSASVE sizes");
    return result;
}
// Ckem phải đúng số byte của modulus và thỏa 0 < c < n.
void validateKemCt(EVP_PKEY* priv, const Bytes& ct) {
    checkRsa(priv);
    const auto k = static_cast<std::size_t>(EVP_PKEY_get_size(priv));
    if (ct.size() != k) fail(3, "invalid KEM ciphertext length");
    BN n = rsaParam(priv, OSSL_PKEY_PARAM_RSA_N);
    BN c(BN_bin2bn(ct.data(), static_cast<int>(ct.size()), nullptr), BN_clear_free);
    if (!c) fail(5, "cannot parse KEM ciphertext integer");
    if (BN_is_negative(c.get()) || BN_is_zero(c.get()) || BN_cmp(c.get(), n.get()) >= 0)
        fail(3, "KEM ciphertext integer out of range");
}
SecureBytes decapsulate(EVP_PKEY* recipientPrivateKey, const Bytes& kem_ct) {
    validateKemCt(recipientPrivateKey, kem_ct);  // Kiểm tra giá trị Ckem trước khi giải mã và dẫn xuất khóa.
    PKeyCtx ctx(EVP_PKEY_CTX_new_from_pkey(nullptr, recipientPrivateKey, nullptr), EVP_PKEY_CTX_free);
    if (!ctx) fail(5, "decapsulation context allocation: " + sslError());
    sslCheck(EVP_PKEY_decapsulate_init(ctx.get(), nullptr), "decapsulate init");
    sslCheck(EVP_PKEY_CTX_set_kem_op(ctx.get(), "RSASVE"), "RSASVE decapsulation op");
    std::size_t zSize = 0;
    if (EVP_PKEY_decapsulate(ctx.get(), nullptr, &zSize, kem_ct.data(), kem_ct.size()) <= 0) {
        ERR_clear_error(); fail(4, "decryption failed");
    }
    const auto k = static_cast<std::size_t>(EVP_PKEY_get_size(recipientPrivateKey));
    if (zSize != k) fail(5, "unexpected RSASVE shared secret size");
    SecureBytes secret(zSize);
    if (EVP_PKEY_decapsulate(ctx.get(), secret.value.data(), &zSize, kem_ct.data(), kem_ct.size()) <= 0) {
        ERR_clear_error(); fail(4, "decryption failed");
    }
    if (zSize != k) fail(4, "decryption failed");
    return secret;
}


// E-F3: Cách tính RSASVE bằng BIGNUM để đối chiếu với EVP trong kiểm thử.
namespace edu {
Encapsulation rsasveEncapsulate(EVP_PKEY* recipientPublicKey) {
    checkRsa(recipientPublicKey);
    BN n = rsaParam(recipientPublicKey, OSSL_PKEY_PARAM_RSA_N);
    BN e = rsaParam(recipientPublicKey, OSSL_PKEY_PARAM_RSA_E);
    BN bound(BN_dup(n.get()), BN_clear_free);
    BN z(BN_new(), BN_clear_free);
    BN c(BN_new(), BN_clear_free);
    BNCtx ctx(BN_CTX_new(), BN_CTX_free);
    if (!bound || !z || !c || !ctx) fail(5, "BIGNUM allocation failed");
    sslCheck(BN_sub_word(bound.get(), 3), "n minus 3");
    sslCheck(BN_rand_range(z.get(), bound.get()), "BN_rand_range");
    sslCheck(BN_add_word(z.get(), 2), "z plus 2");
    sslCheck(BN_mod_exp(c.get(), z.get(), e.get(), n.get(), ctx.get()), "educational modular exp");
    const int k = EVP_PKEY_get_size(recipientPublicKey);
    Encapsulation enc{Bytes(static_cast<std::size_t>(k)), SecureBytes(static_cast<std::size_t>(k))};
    if (BN_bn2binpad(c.get(), enc.kem_ct.data(), k) != k ||
        BN_bn2binpad(z.get(), enc.shared_secret.value.data(), k) != k)
        fail(5, "educational I2OSP failed");
    return enc;
}
SecureBytes rsasveDecapsulate(EVP_PKEY* recipientPrivateKey, const Bytes& kem_ct) {
    validateKemCt(recipientPrivateKey, kem_ct);
    BN n = rsaParam(recipientPrivateKey, OSSL_PKEY_PARAM_RSA_N);
    BN d = rsaParam(recipientPrivateKey, OSSL_PKEY_PARAM_RSA_D);
    BN c(BN_bin2bn(kem_ct.data(), static_cast<int>(kem_ct.size()), nullptr), BN_clear_free);
    BN z(BN_new(), BN_clear_free);
```

```
    BNCtx ctx(BN_CTX_new(), BN_CTX_free);
    if (!n || !d || !c || !z || !ctx) fail(5, "BIGNUM allocation failed");
    sslCheck(BN_mod_exp(z.get(), c.get(), d.get(), n.get(), ctx.get()), "educational decrypt mod exp");
    const int k = EVP_PKEY_get_size(recipientPrivateKey);
    SecureBytes result(static_cast<std::size_t>(k));
    if (BN_bn2binpad(z.get(), result.value.data(), k) != k) fail(5, "educational I2OSP failed");
    return result;
}
}  // Kết thúc phần tính toán bằng BIGNUM


// Tính key_id bằng SHA-256 của khóa công khai ở dạng SPKI DER.
Bytes keyId(EVP_PKEY* key) {
    checkRsa(key);
    const int n = i2d_PUBKEY(key, nullptr);
    if (n <= 0) fail(5, "cannot encode SPKI DER");
    Bytes spki(static_cast<std::size_t>(n));
    unsigned char* p = spki.data();
    if (i2d_PUBKEY(key, &p) != n) fail(5, "SPKI DER output error");
    Bytes id(KEY_ID_LEN);
    unsigned int digestSize = 0;
    sslCheck(EVP_Digest(spki.data(), spki.size(), id.data(), &digestSize, EVP_sha256(), nullptr),
             "SHA256(SPKI DER)");
    if (digestSize != KEY_ID_LEN) fail(5, "unexpected SHA256 digest length");
    return id;
}


// E-F4: Dẫn xuất khóa bằng HKDF-SHA256. Salt rỗng được xử lý theo RFC 5869.
SecureBytes hkdf(const Bytes& ikm, const Bytes& salt, const Bytes& info, std::size_t n) {
    Kdf algorithm(EVP_KDF_fetch(nullptr, "HKDF", nullptr), EVP_KDF_free);
    if (!algorithm) fail(5, "HKDF algorithm not found");
    KdfCtx ctx(EVP_KDF_CTX_new(algorithm.get()), EVP_KDF_CTX_free);
    if (!ctx) fail(5, "HKDF context allocation failed");
    char digest[] = "SHA256";
    int mode = EVP_KDF_HKDF_MODE_EXTRACT_AND_EXPAND;
    const unsigned char empty = 0;
    OSSL_PARAM parameters[] = {
        OSSL_PARAM_construct_utf8_string(OSSL_KDF_PARAM_DIGEST, digest, 0),
        OSSL_PARAM_construct_int(OSSL_KDF_PARAM_MODE, &mode),
        OSSL_PARAM_construct_octet_string(OSSL_KDF_PARAM_KEY,
            const_cast<unsigned char*>(ikm.empty() ? &empty : ikm.data()), ikm.size()),
        OSSL_PARAM_construct_octet_string(OSSL_KDF_PARAM_SALT,
            const_cast<unsigned char*>(salt.empty() ? &empty : salt.data()), salt.size()),
        OSSL_PARAM_construct_octet_string(OSSL_KDF_PARAM_INFO,
            const_cast<unsigned char*>(info.empty() ? &empty : info.data()), info.size()),
        OSSL_PARAM_construct_end()
    };
    SecureBytes output(n);
    sslCheck(EVP_KDF_derive(ctx.get(), output.value.data(), output.value.size(), parameters),
             "HKDF derive");
    return output;
}
// Ghép nhãn, key_id, độ dài Ckem và Ckem vào trường info của HKDF.
SecureBytes deriveWithLabel(const SecureBytes& Z, const Bytes& keyIdBytes,
                            const Bytes& kemCt, const std::string& label) {
    if (keyIdBytes.size() != KEY_ID_LEN || Z.value.empty() || Z.value.size() != kemCt.size())
        fail(3, "invalid HKDF context lengths");
    if (kemCt.size() > 65535) fail(3, "KEM ciphertext too long for HKDF context");
    Bytes info(label.begin(), label.end());
    info.push_back(0x00);
    info.insert(info.end(), keyIdBytes.begin(), keyIdBytes.end());
    putU16(info, kemCt.size());
    info.insert(info.end(), kemCt.begin(), kemCt.end());
    return hkdf(Z.value, {}, info, AES_KEY_LEN);
}
SecureBytes deriveAesKey(const SecureBytes& Z, const Bytes& keyIdBytes, const Bytes& kemCt) {
    return deriveWithLabel(Z, keyIdBytes, kemCt, LABEL);
}


// Gói L2PK chế độ 0x02 gồm phần đầu, Ckem, key_id, nonce, độ dài, bản mã và tag.
// Ghi kết quả vào tệp .part; chỉ đổi tên sau khi xử lý thành công.
class PendingOutput {
    fs::path dest_, part_;
    bool committed_ = false;
    bool force_;
public:
    PendingOutput(const fs::path& path, bool force) : dest_(path), part_(path.string() + ".part"),
force_(force) {
        std::error_code ec;
        if (fs::exists(dest_, ec) && !force_) fail(2, "output exists (use --force)");
```

```
        if (fs::exists(part_, ec)) fail(2, "temporary .part already exists");
    }
    const fs::path& part() const { return part_; }
    void commit() {
        std::error_code ec;
        if (force_ && fs::exists(dest_, ec)) {
            fs::remove(dest_, ec);
            if (ec) fail(2, "cannot replace existing output");
        }
        fs::rename(part_, dest_, ec);
        if (ec) fail(2, "cannot finalize output: " + ec.message());
        committed_ = true;
    }
    ~PendingOutput() {
        if (!committed_) { std::error_code ignored; fs::remove(part_, ignored); }
    }
};
void rejectSameFile(const fs::path& a, const fs::path& b) {
    std::error_code ec;
    if (fs::exists(b, ec) && fs::equivalent(a, b, ec) && !ec)
        fail(1, "input and output must differ");
    if (fs::absolute(a).lexically_normal() == fs::absolute(b).lexically_normal())
        fail(1, "input and output must differ");
}

// Đóng gói Ckem và dùng khóa dẫn xuất từ Z để mã hóa dữ liệu bằng AES-GCM.
std::size_t kemEncrypt(EVP_PKEY* pub, const fs::path& input,
                       const fs::path& output, bool force = false) {
    checkRsa(pub);
    rejectSameFile(input, output);
    const std::uint64_t plainLen = fileSize(input);
    if (plainLen > MAX_FILE) fail(3, "input exceeds 1 GiB limit");
    std::ifstream in(input, std::ios::binary);
    if (!in) fail(2, "cannot open input");
    PendingOutput target(output, force);
    rejectSameFile(input, target.part());


    Encapsulation enc = encapsulate(pub);
    Bytes id = keyId(pub);
    SecureBytes aes = deriveAesKey(enc.shared_secret, id, enc.kem_ct);
    std::array<unsigned char, NONCE_LEN> nonce{};
    sslCheck(RAND_bytes(nonce.data(), static_cast<int>(nonce.size())), "nonce RNG");
    Bytes header{'L', '2', 'P', 'K', VERSION, MODE_KEM, 0, 0};
    putU16(header, enc.kem_ct.size());
    header.insert(header.end(), enc.kem_ct.begin(), enc.kem_ct.end());
    header.insert(header.end(), id.begin(), id.end());
    header.insert(header.end(), nonce.begin(), nonce.end());
    putU64(header, plainLen);
    if (header.size() != 62 + enc.kem_ct.size()) fail(5, "invalid header construction");


    std::ofstream out(target.part(), std::ios::binary | std::ios::trunc);
    if (!out) fail(2, "cannot create output .part");
    writeExact(out, header.data(), header.size());
    CipherCtx ctx(EVP_CIPHER_CTX_new(), EVP_CIPHER_CTX_free);
    if (!ctx) fail(5, "cannot create AES-GCM cipher context");
    sslCheck(EVP_EncryptInit_ex2(ctx.get(), EVP_aes_256_gcm(),
                                 aes.value.data(), nonce.data(), nullptr), "AES-GCM encrypt init");
    int produced = 0;
    sslCheck(EVP_EncryptUpdate(ctx.get(), nullptr, &produced, header.data(),
                               static_cast<int>(header.size())), "AES-GCM header AAD");
    std::array<unsigned char, CHUNK> src{};
    std::array<unsigned char, CHUNK + 16> dst{};
    for (std::uint64_t remain = plainLen; remain > 0;) {
        auto block = static_cast<std::size_t>(std::min<std::uint64_t>(CHUNK, remain));
        readExact(in, src.data(), block);
        sslCheck(EVP_EncryptUpdate(ctx.get(), dst.data(), &produced, src.data(), static_cast<int>(block)),
                 "AES-GCM encrypt chunk");
        writeExact(out, dst.data(), static_cast<std::size_t>(produced));
        remain -= block;
    }
    sslCheck(EVP_EncryptFinal_ex(ctx.get(), dst.data(), &produced), "AES-GCM encrypt final");
    writeExact(out, dst.data(), static_cast<std::size_t>(produced));
    std::array<unsigned char, TAG_LEN> tag{};
    sslCheck(EVP_CIPHER_CTX_ctrl(ctx.get(), EVP_CTRL_AEAD_GET_TAG,
                                 static_cast<int>(TAG_LEN), tag.data()), "AES-GCM get tag");
    writeExact(out, tag.data(), tag.size());
    out.close();
    if (!out) fail(2, "cannot close encrypted file");
    target.commit();
```

```
    return header.size();
}


struct Package {
    Bytes header;
    Bytes kemCt;
    Bytes id;
    std::array<unsigned char, NONCE_LEN> nonce{};
    std::uint64_t ctLen = 0;
};
// Kiểm tra cấu trúc gói tin và các trường độ dài trước khi giải mã.
Package parsePackage(std::ifstream& in, std::uint64_t size, EVP_PKEY* priv) {
    checkRsa(priv);
    if (size < 62 + TAG_LEN) fail(3, "truncated package");
    std::array<unsigned char, 10> first{};
    readExact(in, first.data(), first.size());
    if (std::memcmp(first.data(), "L2PK", 4) != 0) fail(3, "bad package magic");
    if (first[4] != VERSION) fail(3, "unsupported package version");
    if (first[5] != MODE_KEM) fail(3, "unexpected package mode (expected KEM 0x02)");
    if (first[6] != 0 || first[7] != 0) fail(3, "reserved bytes must be zero");
    const std::size_t k = static_cast<std::size_t>(EVP_PKEY_get_size(priv));
    const std::size_t klen = getU16(first.data() + 8);
    if (klen != k) fail(3, "klen does not match RSA modulus size");
    const std::size_t headerSize = 62 + k;
    if (size < static_cast<std::uint64_t>(headerSize + TAG_LEN)) fail(3, "truncated package");
    Package p;
    p.header.assign(first.begin(), first.end());
    p.header.resize(headerSize);
    readExact(in, p.header.data() + first.size(), headerSize - first.size());
    const auto* ptr = p.header.data();
    p.ctLen = getU64(ptr + 54 + k);
    if (p.ctLen > MAX_FILE) fail(3, "ciphertext length exceeds 1 GiB limit");
    if (size != static_cast<std::uint64_t>(headerSize) + p.ctLen + TAG_LEN)
        fail(3, "package total length mismatch");
    p.kemCt.assign(ptr + 10, ptr + 10 + k);
    p.id.assign(ptr + 10 + k, ptr + 42 + k);
    std::copy_n(ptr + 42 + k, NONCE_LEN, p.nonce.begin());
    // Nếu key_id không khớp, trả lỗi 4 để không tiết lộ thông tin về khóa nhận.
    Bytes expectedId = keyId(priv);
    if (CRYPTO_memcmp(p.id.data(), expectedId.data(), KEY_ID_LEN) != 0)
        fail(4, "decryption failed");
    validateKemCt(priv, p.kemCt);  // Kiểm tra Ckem trước HKDF; dữ liệu sai trả mã 3.
    return p;
}
// Chỉ công bố bản rõ sau khi kiểm tra GCM tag thành công.
std::size_t kemDecrypt(EVP_PKEY* priv, const fs::path& input,
                       const fs::path& output, bool force = false) {
    checkRsa(priv);
    rejectSameFile(input, output);
    const auto size = fileSize(input);
    std::ifstream in(input, std::ios::binary);
    if (!in) fail(2, "cannot open package");
    Package package = parsePackage(in, size, priv);
    PendingOutput target(output, force);
    rejectSameFile(input, target.part());
    SecureBytes Z = decapsulate(priv, package.kemCt);
    SecureBytes aes = deriveAesKey(Z, package.id, package.kemCt);
    CipherCtx ctx(EVP_CIPHER_CTX_new(), EVP_CIPHER_CTX_free);
    if (!ctx) fail(5, "cannot create AES-GCM decrypt context");
    sslCheck(EVP_DecryptInit_ex2(ctx.get(), EVP_aes_256_gcm(),
                                 aes.value.data(), package.nonce.data(), nullptr), "AES-GCM decrypt init");
    int produced = 0;
    sslCheck(EVP_DecryptUpdate(ctx.get(), nullptr, &produced,
                               package.header.data(), static_cast<int>(package.header.size())),
             "AES-GCM header AAD");
    std::ofstream out(target.part(), std::ios::binary | std::ios::trunc);
    if (!out) fail(2, "cannot create output .part");
    std::array<unsigned char, CHUNK> src{};
    PlaintextBlock dst;
    for (std::uint64_t remain = package.ctLen; remain > 0;) {
        auto block = static_cast<std::size_t>(std::min<std::uint64_t>(CHUNK, remain));
        readExact(in, src.data(), block);
        sslCheck(EVP_DecryptUpdate(ctx.get(), dst.data.data(), &produced, src.data(), static_cast<int>(block)),
                 "AES-GCM decrypt chunk");
        writeExact(out, dst.data.data(), static_cast<std::size_t>(produced));
        remain -= block;
    }
    std::array<unsigned char, TAG_LEN> tag{};
    readExact(in, tag.data(), tag.size());
    sslCheck(EVP_CIPHER_CTX_ctrl(ctx.get(), EVP_CTRL_AEAD_SET_TAG,
                                 static_cast<int>(TAG_LEN), tag.data()), "AES-GCM set tag");
```

```
    if (EVP_DecryptFinal_ex(ctx.get(), dst.data.data(), &produced) != 1) {
        ERR_clear_error();
        out.close();
        fail(4, "decryption failed");
    }
    writeExact(out, dst.data.data(), static_cast<std::size_t>(produced));
    out.close();
    if (!out) fail(2, "cannot close decrypted file");
    target.commit();
    return package.header.size();
}


Bytes fromHex(const char* text) {
    std::string s(text);
    if (s.size() % 2) fail(5, "invalid test hex");
    Bytes out;
    for (std::size_t i = 0; i < s.size(); i += 2) {
        const std::string two = s.substr(i, 2);
        out.push_back(static_cast<unsigned char>(std::stoul(two, nullptr, 16)));
    }
    return out;
}
Bytes hexRange(unsigned char start, std::size_t n) {
    Bytes b(n);
    for (std::size_t i = 0; i < n; ++i) b[i] = static_cast<unsigned char>(start + i);
    return b;
}


// Tạo thư mục tạm cho các bài kiểm thử và tự xóa khi kết thúc.
class TestDir {
    fs::path dir_;
public:
    TestDir() {
        auto base = fs::temp_directory_path();
        for (int attempts = 0; attempts < 16; ++attempts) {
            std::array<unsigned char, 12> r{};
            sslCheck(RAND_bytes(r.data(), static_cast<int>(r.size())), "test dir RNG");
            const char* hex = "0123456789abcdef";
            std::string suffix;
            for (auto v : r) { suffix.push_back(hex[v >> 4]); suffix.push_back(hex[v & 15]); }
            dir_ = base / ("lab03_task_e_" + suffix);
            std::error_code ec;
            if (fs::create_directory(dir_, ec)) return;
            if (ec && ec != std::errc::file_exists) fail(2, "cannot create temporary test directory");
        }
        fail(2, "cannot obtain unique temporary test directory");
    }
    fs::path file(const std::string& filename) const { return dir_ / filename; }
    ~TestDir() { std::error_code ec; fs::remove_all(dir_, ec); }
};
// Xuất khóa PEM/DER tạm thời để thử các định dạng đầu vào.
void saveDerPublic(const fs::path& path, EVP_PKEY* key) {
    int len = i2d_PUBKEY(key, nullptr);
    if (len <= 0) fail(5, "cannot DER encode public key");
    Bytes raw(static_cast<std::size_t>(len));
    unsigned char* p = raw.data();
    if (i2d_PUBKEY(key, &p) != len) fail(5, "DER public key encoding failed");
    writeSmall(path, raw);
}
void saveDerPrivate(const fs::path& path, EVP_PKEY* key) {
    Pkcs8 p8(EVP_PKEY2PKCS8(key), PKCS8_PRIV_KEY_INFO_free);
    if (!p8) fail(5, "cannot PKCS#8 encode private key");
    const int len = i2d_PKCS8_PRIV_KEY_INFO(p8.get(), nullptr);
    if (len <= 0) fail(5, "cannot DER encode PKCS#8");
    Bytes raw(static_cast<std::size_t>(len));
    struct Clean { Bytes& b; ~Clean() { if (!b.empty()) OPENSSL_cleanse(b.data(), b.size()); } } clean{raw};
    unsigned char* p = raw.data();
    if (i2d_PKCS8_PRIV_KEY_INFO(p8.get(), &p) != len) fail(5, "DER PKCS#8 encoding failed");
    writeSmall(path, raw);
}
void savePemPublic(const fs::path& path, EVP_PKEY* key) {
    Bio bio(BIO_new_file(path.string().c_str(), "wb"), BIO_free);
    if (!bio || PEM_write_bio_PUBKEY(bio.get(), key) != 1) fail(5, "cannot write test public PEM");
}
void savePemPrivate(const fs::path& path, EVP_PKEY* key) {
    Bio bio(BIO_new_file(path.string().c_str(), "wb"), BIO_free);
    if (!bio || PEM_write_bio_PrivateKey(bio.get(), key, nullptr, nullptr, 0, nullptr, nullptr) != 1)
        fail(5, "cannot write test private PEM");
}
```

```
// Kiểm tra mã lỗi và bảo đảm không còn tệp kết quả hoặc tệp .part.
bool expectedDecryptFailure(EVP_PKEY* priv, const fs::path& pkg,
                            const fs::path& output, int code) {
    std::error_code ec;
    fs::remove(output, ec);
    fs::remove(fs::path(output.string() + ".part"), ec);
    bool matched = false;
    try { (void)kemDecrypt(priv, pkg, output); }
    catch (const LabError& e) {
        matched = e.code == code && (code != 4 || std::string(e.what()) == "decryption failed");
    }
    return matched && !fs::exists(output) && !fs::exists(fs::path(output.string() + ".part"));
}

// E-F6: Chạy lần lượt các kiểm thử E-T1 đến E-T9.
int selfTest(EVP_PKEY* pub, EVP_PKEY* priv) {
    checkRsa(pub); checkRsa(priv);
    if (EVP_PKEY_get_size(pub) != EVP_PKEY_get_size(priv) || keyId(pub) != keyId(priv))
        fail(3, "selftest public and private keys do not match");
    std::cout << "=== TASK E COMPLETE SELFTEST ===\n";
    std::cout << "Linked OpenSSL: " << OpenSSL_version(OPENSSL_VERSION) << '\n';
    std::cout << "RSA modulus: " << EVP_PKEY_get_bits(pub) << " bits\n";
    int passed = 0;
    const auto check = [&](const char* label, bool ok) {
        std::cout << label << ": " << (ok ? "PASS" : "FAIL") << '\n';
        if (ok) ++passed;
    };

    // E-T1: Thực hiện 100 lần đóng gói, kiểm tra bí mật chung và Ckem không trùng.
    std::set<Bytes> distinct;
    bool all100 = true;
    for (int i = 0; i < 100; ++i) {
        Encapsulation e = encapsulate(pub);
        SecureBytes z = decapsulate(priv, e.kem_ct);
        all100 = all100 && equalSecret(e.shared_secret, z);
        distinct.insert(e.kem_ct);
    }
    check("E-T1 EVP 100 rounds / 100 distinct Ckem", all100 && distinct.size() == 100);

    Encapsulation eduEnc = edu::rsasveEncapsulate(pub);
    SecureBytes byEvp = decapsulate(priv, eduEnc.kem_ct);
    Encapsulation evpEnc = encapsulate(pub);
    SecureBytes byEdu = edu::rsasveDecapsulate(priv, evpEnc.kem_ct);
    check("E-T2 EDU->EVP and EVP->EDU", equalSecret(eduEnc.shared_secret, byEvp) &&
                                        equalSecret(evpEnc.shared_secret, byEdu));

    // E-T3: Đối chiếu kết quả HKDF với bộ dữ liệu mẫu số 1 của RFC 5869.
    Bytes ikm(22, 0x0b);
    Bytes salt = hexRange(0x00, 13);
    Bytes info = hexRange(0xf0, 10);
    const Bytes kat = fromHex("3cb25f25faacd57a90434f64d0362f2a"
                              "2d2d0a90cf1a5a4c5db02d56ecc4c5bf"
                              "34007208d5b887185865");
    SecureBytes actual = hkdf(ikm, salt, info, 42);
    check("E-T3 RFC5869 SHA256 KAT", actual.value.size() == kat.size() &&
          CRYPTO_memcmp(actual.value.data(), kat.data(), kat.size()) == 0);

    Bytes id = keyId(pub);
    SecureBytes k1 = deriveAesKey(evpEnc.shared_secret, id, evpEnc.kem_ct);
    std::string alteredLabel(LABEL);
    alteredLabel[0] ^= 0x01;
    SecureBytes k2 = deriveWithLabel(evpEnc.shared_secret, id, evpEnc.kem_ct, alteredLabel);
    check("E-T4 one-bit KDF label change", !equalSecret(k1, k2));

    TestDir t;
    const fs::path plain = t.file("sample.bin");
    const Bytes sample = Bytes{'L','a','b','0','3',0,0xff,'E','-','T','5','\n'};
    writeSmall(plain, sample);
    const fs::path pemPub = t.file("public.pem"), pemPriv = t.file("private.pem");
    const fs::path derPub = t.file("public.der"), derPriv = t.file("private.der");
    savePemPublic(pemPub, pub); savePemPrivate(pemPriv, priv);
    saveDerPublic(derPub, pub); saveDerPrivate(derPriv, priv);
    bool matrixOk = true;
    const std::array<fs::path, 4> pubs{pemPub, derPub, pemPub, derPub};
    const std::array<fs::path, 4> privs{pemPriv, derPriv, derPriv, pemPriv};
```

```
for (std::size_t i = 0; i < 4; ++i) {
    PKey p = loadKey(pubs[i], false);
    PKey s = loadKey(privs[i], true);
    auto package = t.file("test_package_" + std::to_string(i) + ".l2pk");
    auto recovered = t.file("test_recovered_" + std::to_string(i) + ".bin");
    kemEncrypt(p.get(), plain, package);
    kemDecrypt(s.get(), package, recovered);
    matrixOk = matrixOk && readSmall(recovered) == sample;
}
check("E-T5 PEM/DER 4-way file round trip", matrixOk);


const fs::path good = t.file("good.l2pk"), broken = t.file("broken.l2pk"),
               rejected = t.file("rejected.bin");
kemEncrypt(pub, plain, good);
Bytes raw = readSmall(good);
const std::size_t k = static_cast<std::size_t>(EVP_PKEY_get_size(pub));
const auto testModified = [&](const Bytes& changed, int expectedCode) {
    writeSmall(broken, changed);
    return expectedDecryptFailure(priv, broken, rejected, expectedCode);
};
// E-T6: Kiểm tra klen sai, Ckem bằng n hoặc bằng 0; đều phải trả mã 3.
Bytes wrongLen = raw;
wrongLen[9] ^= 1;
bool t6 = testModified(wrongLen, 3);
BN n = rsaParam(pub, OSSL_PKEY_PARAM_RSA_N);
Bytes modulus(k);
require(BN_bn2binpad(n.get(), modulus.data(), static_cast<int>(k)) == static_cast<int>(k),
        "cannot encode RSA modulus for E-T6");
Bytes tooLarge = raw;
std::copy(modulus.begin(), modulus.end(), tooLarge.begin() + 10);
t6 = t6 && testModified(tooLarge, 3);
Bytes zero = raw;
std::fill(zero.begin() + 10, zero.begin() + 10 + static_cast<std::ptrdiff_t>(k), 0);
t6 = t6 && testModified(zero, 3);
check("E-T6 invalid kem_ct length / >=n / zero", t6);


// E-T7: Đảo một bit của Ckem nhưng vẫn giữ giá trị trong khoảng hợp lệ.
// Khi Z thay đổi, bước xác thực GCM phải thất bại và không tạo bản rõ.
Bytes altered = raw;
altered[10 + k - 1] ^= 1;
Bytes alteredCt(altered.begin() + 10, altered.begin() + 10 + static_cast<std::ptrdiff_t>(k));
bool stillInRange = true;
try { validateKemCt(priv, alteredCt); }
catch (const LabError&) { stillInRange = false; }
bool t7 = false;
if (stillInRange) {
    Bytes goodCt(raw.begin() + 10, raw.begin() + 10 + static_cast<std::ptrdiff_t>(k));
    SecureBytes zGood = decapsulate(priv, goodCt);
    SecureBytes zTampered = decapsulate(priv, alteredCt);
    t7 = !equalSecret(zGood, zTampered) && testModified(altered, 4);
}
check("E-T7 modified KEM changes Z; GCM rejects", t7);


// E-T8: Lần lượt đảo một bit ở key_id, nonce, ciphertext và tag.
bool t8 = true;
for (auto offset : {10 + k, 42 + k, 62 + k, raw.size() - 1}) {
    Bytes changed = raw;
    changed[offset] ^= 1;
    // Sai key_id trả mã 4; các trường còn lại bị GCM phát hiện.
    t8 = t8 && testModified(changed, 4);
}
check("E-T8 tampered key_id / nonce / ciphertext / tag", t8);


// E-T9: Thử sai khóa RSA, sai mode và phiên bản không được hỗ trợ.
PKey other(EVP_RSA_gen(EVP_PKEY_get_bits(priv)), EVP_PKEY_free);
if (!other) fail(5, "cannot make second RSA key");
bool t9 = expectedDecryptFailure(other.get(), good, rejected, 4);
Bytes wrongMode = raw;
wrongMode[5] = 1;
t9 = t9 && testModified(wrongMode, 3);
Bytes wrongVersion = raw;
wrongVersion[4] = 2;
t9 = t9 && testModified(wrongVersion, 3);
check("E-T9 wrong key / mode 0x01 / version 0x02", t9);


std::cout << "Task E total: " << passed << "/9 PASS\n";
std::cout << "No shared secrets or AES keys were printed.\n";
return passed == 9 ? 0 : 5;
```

```
}


struct Cli {
    fs::path pub, priv, input, output;
    bool force = false;
};
Cli parseCli(int argc, char** argv, bool test, bool encrypt) {
    Cli args;
    bool hasPub = false, hasPriv = false, hasIn = false, hasOut = false;
    for (int i = 2; i < argc; ++i) {
        std::string option(argv[i]);
        if (option == "--force") {
            if (test || args.force) fail(1, "invalid/duplicate --force");
            args.force = true;
        } else if (option == "--pub" && (test || encrypt)) {
            if (hasPub || i + 1 >= argc) fail(1, "invalid --pub");
            args.pub = argv[++i]; hasPub = true;
        } else if (option == "--priv" && (test || !encrypt)) {
            if (hasPriv || i + 1 >= argc) fail(1, "invalid --priv");
            args.priv = argv[++i]; hasPriv = true;
        } else if (!test && option == "--in") {
            if (hasIn || i + 1 >= argc) fail(1, "invalid --in");
            args.input = argv[++i]; hasIn = true;
        } else if (!test && option == "--out") {
            if (hasOut || i + 1 >= argc) fail(1, "invalid --out");
            args.output = argv[++i]; hasOut = true;
        } else fail(1, "unknown or unsupported argument: " + option);
    }
    if (test) { if (!hasPub || !hasPriv) fail(1, "kem-selftest requires --pub and --priv"); }
    else if (!(hasIn && hasOut && (encrypt ? hasPub : hasPriv)))
        fail(1, "missing --pub/--priv, --in or --out");
    return args;
}
void usage() {
    std::cout << "Task E - RSASVE/HKDF/AES-256-GCM (lab profile, not RFC 9690)\n"
              << "  task_e_kem.exe kem-selftest --pub <key> --priv <key>\n"
              << "  task_e_kem.exe kem-encrypt --pub <key> --in <file> --out <l2pk> [--force]\n"
              << "  task_e_kem.exe kem-decrypt --priv <key> --in <l2pk> --out <file> [--force]\n"
              << "  task_e_kem.exe <public.pem> <private.pem>  (shortcut for selftest)\n";
}



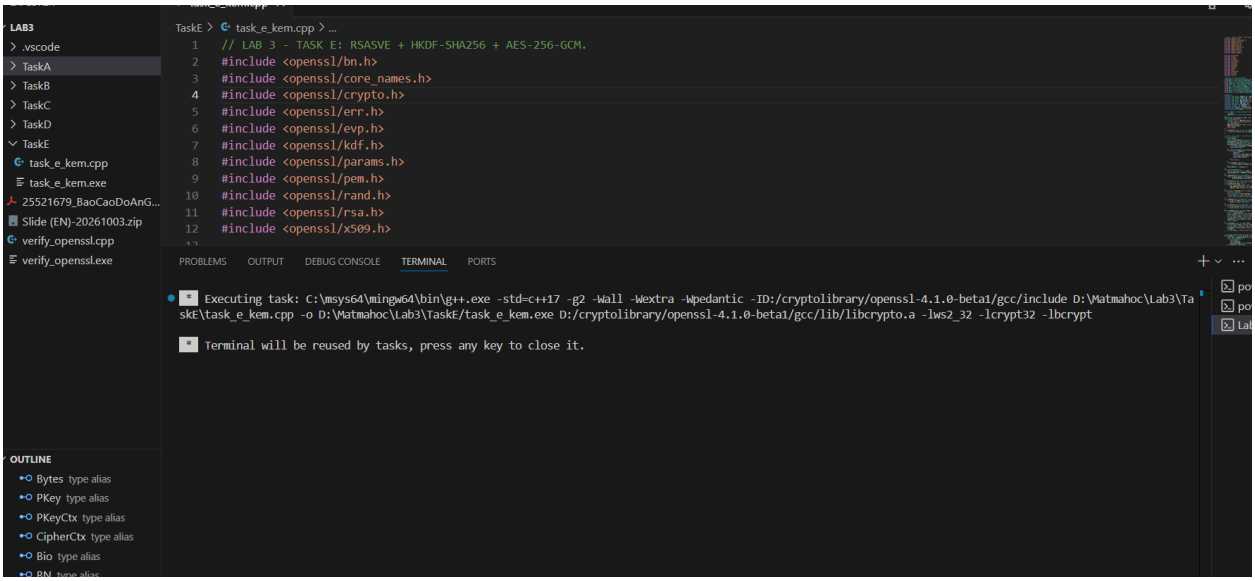
int entry(int argc, char** argv) {
    try {
        if (argc == 2 && (std::string(argv[1]) == "--help" || std::string(argv[1]) == "help")) {
            usage(); return 0;
        }
        if (argc == 3 && argv[1][0] != '-') {
            PKey pub = loadKey(argv[1], false);
            PKey priv = loadKey(argv[2], true);
            return selfTest(pub.get(), priv.get());
        }
        if (argc < 2) { usage(); return 1; }
        std::string sub(argv[1]);
        if (sub == "kem-selftest") {
            Cli a = parseCli(argc, argv, true, false);
            PKey pub = loadKey(a.pub, false), priv = loadKey(a.priv, true);
            return selfTest(pub.get(), priv.get());
        }
        if (sub == "kem-encrypt") {
            Cli a = parseCli(argc, argv, false, true);
            PKey pub = loadKey(a.pub, false);
            auto header = kemEncrypt(pub.get(), a.input, a.output, a.force);
            std::cout << "ENCRYPT OK | mode=0x02 | header=" << header << " bytes\n";
            return 0;
        }
        if (sub == "kem-decrypt") {
            Cli a = parseCli(argc, argv, false, false);
            PKey priv = loadKey(a.priv, true);
            auto header = kemDecrypt(priv.get(), a.input, a.output, a.force);
            std::cout << "DECRYPT OK | mode=0x02 | header=" << header << " bytes\n";
            return 0;
        }
        usage(); return 1;
    } catch (const LabError& e) {
        std::cerr << "Error: " << e.what() << '\n';
        return e.code;
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << '\n';
```

```
        return 5;
    }
}

} // namespace task_e
```

2. Hình ảnh thực hiện và kết quả kiểm thử

Biên dịch và kiểm thử tổng hợp Task E



Hình 101. Biên dịch chương trình Task E bằng GCC MinGW64 và thư viện OpenSSL.

```
PS D:\Matmahoc\Lab3\TaskE> $env:Path = "C:\msys64\mingw64\bin;$env:Path"
PS D:\Matmahoc\Lab3\TaskE> .\task_e_kem.exe kem-selftest `
>>      --pub ..\TaskB\public.pem `
>>      --priv ..\TaskB\private.pem
=== TASK E COMPLETE SELFTEST ===
Linked OpenSSL: OpenSSL 4.1.0-beta1 23 Sep 2026
RSA modulus: 3072 bits
E-T1 EVP 100 rounds / 100 distinct Ckem: PASS
E-T2 EDU->EVP and EVP->EDU: PASS
E-T3 RFC5869 SHA256 KAT: PASS
E-T4 one-bit KDF label change: PASS
E-T5 PEM/DER 4-way file round trip: PASS
E-T6 invalid kem_ct length / >=n / zero: PASS
E-T7 modified KEM changes Z; GCM rejects: PASS
E-T8 tampered key_id / nonce / ciphertext / tag: PASS
E-T9 wrong key / mode 0x01 / version 0x02: PASS
Task E total: 9/9 PASS
No shared secrets or AES keys were printed.
PS D:\Matmahoc\Lab3\TaskE> Write-Host "Run exit code: $LASTEXITCODE"
Run exit code: 0
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 102. Kiểm thử tổng hợp Task E với RSASVE, HKDF-SHA256 và AES-256-GCM; cả 9 nhóm kiểm thử đều thành công.

E-T5

```
PS D:\Matmahoc\Lab3\TaskE> Set-Content -Path .\E-T5_message.txt `
>>     -Value "Kiem thu Task E - RSASVE va AES-256-GCM." `
>>     -Encoding UTF8
PS D:\Matmahoc\Lab3\TaskE> .\task_e_kem.exe kem-encrypt `
>>     --pub ..\TaskB\public.pem `
>>     --in .\E-T5_message.txt `
>>     --out .\E-T5_message.l2pk
ENCRYPT OK | mode=0x02 | header=446 bytes
PS D:\Matmahoc\Lab3\TaskE> Write-Host "Encrypt exit code: $LASTEXITCODE"
Encrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskE> Get-Item .\E-T5_message.txt, .\E-T5_message.l2pk |
>>     Select-Object Name, Length

Name              Length
----              ------
E-T5_message.txt      45
E-T5_message.l2pk    507


PS D:\Matmahoc\Lab3\TaskE>
```

Hình 103. Mã hóa tệp văn bản bằng RSASVE-HKDF-AES-256-GCM và kiểm tra kích thước gói L2PK.

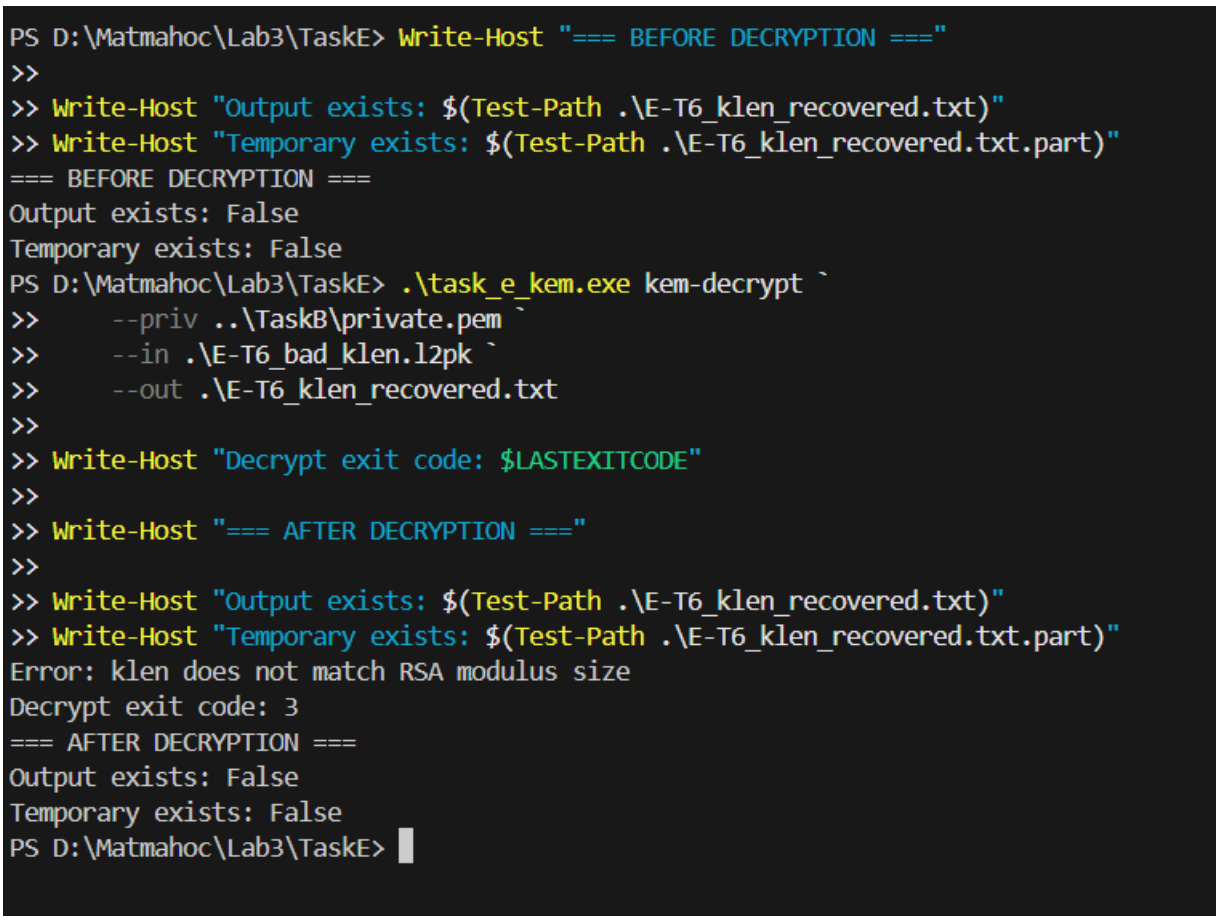
```
PS D:\Matmahoc\Lab3\TaskE> Write-Host "=== BEFORE DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\E-T5_recovered_new.txt)"
>> Write-Host "Temporary exists: $(Test-Path .\E-T5_recovered_new.txt.part)"
=== BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskE> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T5_message.l2pk `
>>     --out .\E-T5_recovered_new.txt
>>
>> Write-Host "Decrypt exit code: $LASTEXITCODE"
DECRYPT OK | mode=0x02 | header=446 bytes
Decrypt exit code: 0
PS D:\Matmahoc\Lab3\TaskE> $hashOriginal = (Get-FileHash .\E-T5_message.txt -Algorithm SHA256).Hash
>>
>> $hashRecovered = (Get-FileHash .\E-T5_recovered_new.txt -Algorithm SHA256).Hash
>>
>> Write-Host "=== SHA-256 COMPARISON ==="
>>
>> Write-Host "Original SHA256: $hashOriginal"
>> Write-Host "Recovered SHA256: $hashRecovered"
>> Write-Host "Hashes identical: $($hashOriginal -eq $hashRecovered)"
>>
>> Write-Host "=== RECOVERED MESSAGE ==="
>> Get-Content .\E-T5_recovered_new.txt
=== SHA-256 COMPARISON ===
Original SHA256: C2EE6ED4C93ECB09C03596FF1F2B5ACB9CC78810E279EC9F1860893925CB75D6
Recovered SHA256: C2EE6ED4C93ECB09C03596FF1F2B5ACB9CC78810E279EC9F1860893925CB75D6
Hashes identical: True
=== RECOVERED MESSAGE ===
Kiem thu Task E - RSASVE va AES-256-GCM.
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 104. Kiểm thử giải mã gói L2PK chế độ RSASVE-HKDF-AES-256-GCM. Chương trình khôi phục thành công tệp bản rõ và xác nhận giá trị SHA-256 của tệp gốc và tệp sau giải mã trùng khớp.

E-T6

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>>
>> $modified = [byte[]]$original.Clone()
>>
>> # Đảo một bit trong trường klen
>> $offset = 9
>>
>> $modified[$offset] = $modified[$offset] -bxor 0x01
>>
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T6_bad_klen.l2pk"),
>>     $modified
>> )
>>
>> Write-Host "=== E-T6.1 INVALID KLEN ==="
>>
>> Write-Host ("Original klen: {0}" -f [BitConverter]::ToString($original, 8, 2))
>> Write-Host ("Modified klen: {0}" -f [BitConverter]::ToString($modified, 8, 2))
>>
>> Write-Host "Package size: $($modified.Length) bytes"
=== E-T6.1 INVALID KLEN ===
Original klen: 01-80
Modified klen: 01-81
Package size: 507 bytes
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 105. Thay đổi trường klen từ 384 thành 385 byte để tạo gói kiểm thử không hợp lệ.



Hình 106. Kiểm thử trường klen không hợp lệ trong gói L2PK chế độ RSASVE. Sau khi thay đổi klen từ 384 thành 385 byte, chương trình từ chối giải mã với mã thoát 3, không tạo tệp bản rõ hoặc để lại tệp tạm.

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>>
>> $modified = [byte[]]$original.Clone()
>>
>> # Đọc độ dài kem_ct từ header
>> $klen = [int]$original[8] * 256 + [int]$original[9]
>>
>> if ($klen -ne 384) {
>>     throw "Kích thước kem_ct không đúng 384 byte"
>> }
>>
>> # Thay toàn bộ kem_ct bằng FF
>> for ($i = 10; $i -lt (10 + $klen); $i++) {
>>     $modified[$i] = 0xFF
>> }
>>
>> # Lưu package kiểm thử
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T6_kem_ct_ge_n.l2pk"),
>>     $modified
>> )
>>
>> Write-Host "=== E-T6.2 INVALID KEM CIPHERTEXT ==="
>> Write-Host "KEM ciphertext length: $klen bytes"
>>
>> Write-Host "Original kem_ct (first 16 bytes):"
>> Write-Host ([BitConverter]::ToString($original, 10, 16))
>>
>> Write-Host "Modified kem_ct (first 16 bytes):"
>> Write-Host ([BitConverter]::ToString($modified, 10, 16))
>>
>> Write-Host "Package size: $($modified.Length) bytes"
=== E-T6.2 INVALID KEM CIPHERTEXT ===
KEM ciphertext length: 384 bytes
Original kem_ct (first 16 bytes):
56-8C-5F-B0-E5-9C-B2-DF-70-BB-6C-3D-EC-0A-2F-FB
Modified kem_ct (first 16 bytes):
FF-FF-FF-FF-FF-FF-FF-FF-FF-FF-FF-FF-FF-FF-FF-FF
Package size: 507 bytes
PS D:\Matmahoc\Lab3\TaskE>
```

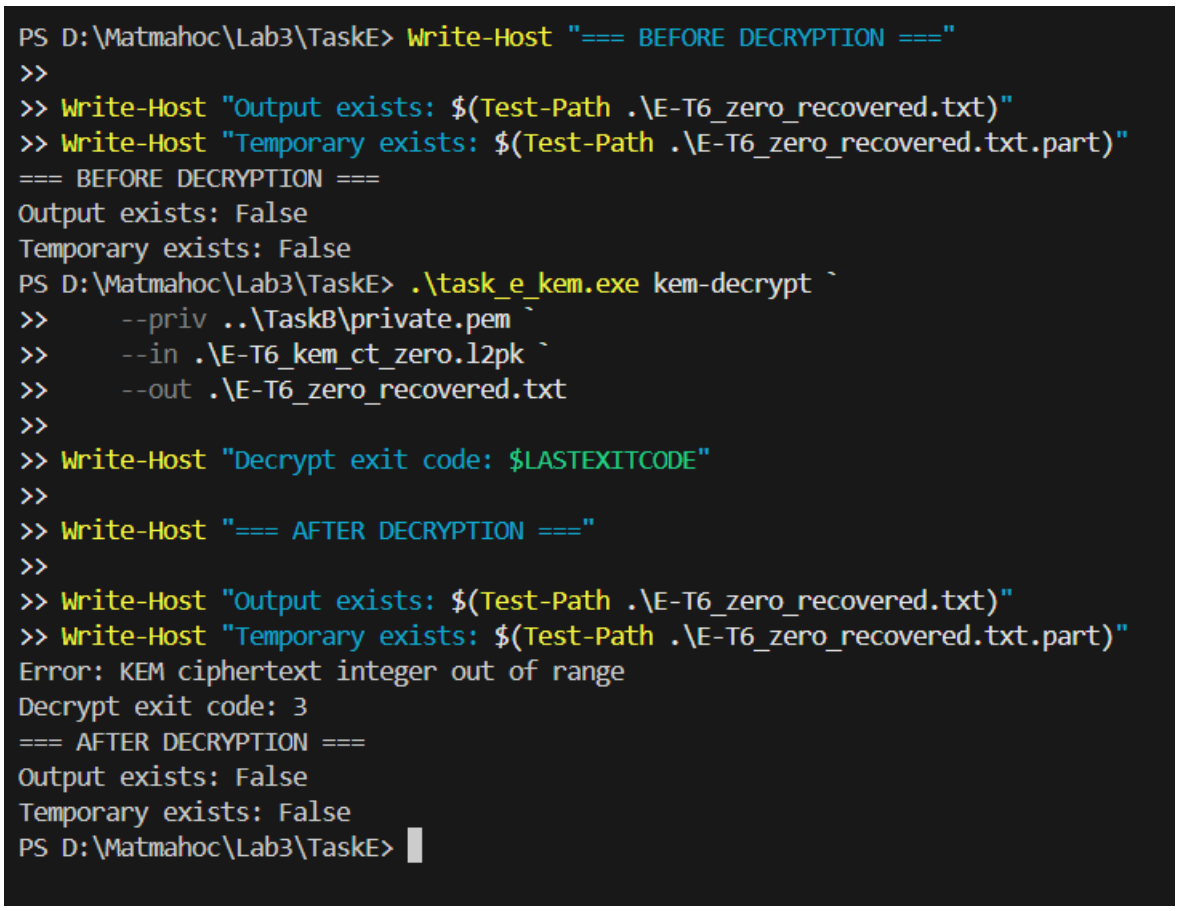
Hình 107. Tạo gói kiểm thử có giá trị KEM ciphertext lớn hơn hoặc bằng modulus RSA.

```
PS D:\Matmahoc\Lab3\TaskE> Write-Host "=== BEFORE DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\E-T6_ge_n_recovered.txt)"
>> Write-Host "Temporary exists: $(Test-Path .\E-T6_ge_n_recovered.txt.part)"
=== BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskE> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T6_kem_ct_ge_n.l2pk `
>>     --out .\E-T6_ge_n_recovered.txt
>>
>> Write-Host "Decrypt exit code: $LASTEXITCODE"
>>
>> Write-Host "=== AFTER DECRYPTION ==="
>>
>> Write-Host "Output exists: $(Test-Path .\E-T6_ge_n_recovered.txt)"
>> Write-Host "Temporary exists: $(Test-Path .\E-T6_ge_n_recovered.txt.part)"
Error: KEM ciphertext integer out of range
Decrypt exit code: 3
=== AFTER DECRYPTION ===
Output exists: False
Temporary exists: False
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 108. Kiểm thử giá trị KEM ciphertext lớn hơn hoặc bằng modulus RSA. Chương trình phát hiện ciphertext không hợp lệ, từ chối giải mã với mã thoát 3 và không tạo tệp đầu ra hoặc để lại tệp tạm.

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>>
>> $modified = [byte[]]$original.Clone()
>>
>> # Đọc độ dài kem_ct
>> $klen = [int]$original[8] * 256 + [int]$original[9]
>>
>> if ($klen -ne 384) {
>>     throw "Độ dài kem_ct không đúng 384 byte"
>> }
>>
>> # Đặt toàn bộ kem_ct bằng 0
>> for ($i = 10; $i -lt (10 + $klen); $i++) {
>>     $modified[$i] = 0x00
>> }
>>
>> # Lưu thành một package riêng
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T6_kem_ct_zero.l2pk"),
>>     $modified
>> )
>>
>> Write-Host "=== E-T6.3 ZERO KEM CIPHERTEXT ==="
>> Write-Host "KEM ciphertext length: $klen bytes"
>>
>> Write-Host "Modified kem_ct (first 16 bytes):"
>> Write-Host ([BitConverter]::ToString($modified, 10, 16))
>>
>> Write-Host "All KEM bytes are zero: $(-not [bool](@($modified[10..(9 + $klen)] | Where-Object { $_ -ne 0 }).Count))"
>>
>> Write-Host "Package size: $($modified.Length) bytes"
=== E-T6.3 ZERO KEM CIPHERTEXT ===
KEM ciphertext length: 384 bytes
Modified kem_ct (first 16 bytes):
00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
All KEM bytes are zero: True
Package size: 507 bytes
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 109. Đặt toàn bộ KEM ciphertext thành giá trị 0 trong gói L2PK.



Hình 110. Kiểm thử KEM ciphertext bằng 0 trong gói L2PK. Chương trình phát hiện giá trị ciphertext không thuộc miền hợp lệ của RSASVE, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm.

E-T7

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>>
>> $modified = [byte[]]$original.Clone()
>>
>> # Đọc độ dài KEM ciphertext
>> $klen = [int]$original[8] * 256 + [int]$original[9]
>>
>> if ($klen -ne 384) {
>>     throw "Độ dài kem_ct không đúng 384 byte"
>> }
>>
>> # Đảo một bit tại byte cuối của KEM ciphertext
>> $offset = 10 + $klen - 1
>> $modified[$offset] = $modified[$offset] -bxor 0x01
>>
>> # Lưu package kiểm thử
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T7_tampered_kem_ct.l2pk"),
>>     $modified
>> )
>>
>> Write-Host "=== E-T7.1 KEM CIPHERTEXT TAMPERING ==="
>> Write-Host "KEM ciphertext length: $klen bytes"
>> Write-Host ("Offset: {0} (0x{0:X})" -f $offset)
>> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
>> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
>> Write-Host ("XOR difference: {0:X2}" -f ($original[$offset] -bxor $modified[$offset]))
>> Write-Host "Package size: $($modified.Length) bytes"
=== E-T7.1 KEM CIPHERTEXT TAMPERING ===
KEM ciphertext length: 384 bytes
Offset: 393 (0x189)
Original byte: 92
Modified byte: 93
XOR difference: 01
Package size: 507 bytes
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 111. Thay đổi một bit tại byte cuối của KEM ciphertext.

```
PS D:\Matmahoc\Lab3\TaskE> Write-Host "=== E-T7.2 BEFORE DECRYPTION ==="
>>
>> $outputExists = Test-Path .\E-T7_recovered.txt
>> $tempExists = Test-Path .\E-T7_recovered.txt.part
>>
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>>
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại, cần kiểm tra trước khi giải mã"
>> }
=== E-T7.2 BEFORE DECRYPTION ===
Output exists: False
Temporary exists: False
```

Hình 112. Kiểm tra tệp bản rõ và tệp tạm chưa tồn tại trước khi giải mã gói có KEM ciphertext bị thay đổi.

```
PS D:\Matmahoc\Lab3\TaskE> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T7_tampered_kem_ct.l2pk `
>>     --out .\E-T7_recovered.txt
>>
>> $exitCode = $LASTEXITCODE
>>
>> Write-Host "=== E-T7.3 DECRYPTION RESULT ==="
>> Write-Host "Decrypt exit code: $exitCode"
>>
>> Write-Host "=== AFTER DECRYPTION ==="
>>
>> $outputExists = Test-Path .\E-T7_recovered.txt
>> $tempExists = Test-Path .\E-T7_recovered.txt.part
>>
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>>
>> Write-Host "=== E-T7 FINAL CHECK ==="
>>
>> $passed = ($exitCode -eq 4) -and (-not $outputExists) -and (-not $tempExists)
>>
>> Write-Host "E-T7 PASS: $passed"
Error: decryption failed
=== E-T7.3 DECRYPTION RESULT ===
Decrypt exit code: 4
=== AFTER DECRYPTION ===
Output exists: False
Temporary exists: False
=== E-T7 FINAL CHECK ===
E-T7 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 113. Kiểm thử thay đổi một bit trong KEM ciphertext. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp bản rõ hoặc để lại tệp tạm.

E-T8

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>> $modified = [byte[]]$original.Clone()
>> $offset = 394
>> $modified[$offset] = $modified[$offset] -bxor 0x01
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T8_bad_keyid.l2pk"),
>>     $modified
>> )
>> Write-Host "=== E-T8.1 KEY_ID TAMPERING ==="
>> Write-Host ("Offset: {0}" -f $offset)
>> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
>> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
>> Write-Host "Package size: $($modified.Length) bytes"
>> $outputExists = Test-Path .\E-T8_keyid_recovered.txt
>> $tempExists = Test-Path .\E-T8_keyid_recovered.txt.part
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T8_bad_keyid.l2pk `
>>     --out .\E-T8_keyid_recovered.txt
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path .\E-T8_keyid_recovered.txt
>> $tempExists = Test-Path .\E-T8_keyid_recovered.txt.part
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = (($code -eq 3) -or ($code -eq 4)) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T8.1 PASS: $passed"
=== E-T8.1 KEY_ID TAMPERING ===
Offset: 394
Original byte: 0B
Modified byte: 0A
Package size: 507 bytes
Error: decryption failed
Decrypt exit code: 4
Output exists: False
Temporary exists: False
E-T8.1 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 114. Thay đổi một bit trong trường key_id và kiểm thử khả năng phát hiện dữ liệu bị sửa đổi.

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>> $modified = [byte[]]$original.Clone()
>> $offset = 426
>> $modified[$offset] = $modified[$offset] -bxor 0x01
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T8_bad_nonce.l2pk"),
>>     $modified
>> )
>> Write-Host "=== E-T8.2 NONCE TAMPERING ==="
>> Write-Host ("Offset: {0}" -f $offset)
>> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
>> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
>> Write-Host "Package size: $($modified.Length) bytes"
>> $outputExists = Test-Path .\E-T8_nonce_recovered.txt
>> $tempExists = Test-Path .\E-T8_nonce_recovered.txt.part
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T8_bad_nonce.l2pk `
>>     --out .\E-T8_nonce_recovered.txt
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path .\E-T8_nonce_recovered.txt
>> $tempExists = Test-Path .\E-T8_nonce_recovered.txt.part
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = ($code -eq 4) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T8.2 PASS: $passed"
=== E-T8.2 NONCE TAMPERING ===
Offset: 426
Original byte: F8
Modified byte: F9
Package size: 507 bytes
Error: decryption failed
Decrypt exit code: 4
Output exists: False
Temporary exists: False
E-T8.2 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 115. Thay đổi một bit trong nonce và kiểm thử khả năng từ chối giải mã.

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>> $modified = [byte[]]$original.Clone()
>> $offset = 446
>> $modified[$offset] = $modified[$offset] -bxor 0x01
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T8_bad_ciphertext.l2pk"),
>>     $modified
>> )
>> Write-Host "=== E-T8.3 CIPHERTEXT TAMPERING ==="
>> Write-Host ("Offset: {0}" -f $offset)
>> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
>> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
>> Write-Host "Package size: $($modified.Length) bytes"
>> $outputExists = Test-Path .\E-T8_ciphertext_recovered.txt
>> $tempExists = Test-Path .\E-T8_ciphertext_recovered.txt.part
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T8_bad_ciphertext.l2pk `
>>     --out .\E-T8_ciphertext_recovered.txt
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path .\E-T8_ciphertext_recovered.txt
>> $tempExists = Test-Path .\E-T8_ciphertext_recovered.txt.part
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = ($code -eq 4) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T8.3 PASS: $passed"
=== E-T8.3 CIPHERTEXT TAMPERING ===
Offset: 446
Original byte: BD
Modified byte: BC
Package size: 507 bytes
Error: decryption failed
Decrypt exit code: 4
Output exists: False
Temporary exists: False
E-T8.3 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 116. Thay đổi một bit trong ciphertext và kiểm thử khả năng từ chối giải mã.

```
PS D:\Matmahoc\Lab3\TaskE> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>> $modified = [byte[]]$original.Clone()
>> $offset = $original.Length - 1
>> $modified[$offset] = $modified[$offset] -bxor 0x01
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T8_bad_tag.l2pk"),
>>     $modified
>> )
>> Write-Host "=== E-T8.4 GCM TAG TAMPERING ==="
>> Write-Host ("Offset: {0}" -f $offset)
>> Write-Host ("Original byte: {0:X2}" -f $original[$offset])
>> Write-Host ("Modified byte: {0:X2}" -f $modified[$offset])
>> Write-Host "Package size: $($modified.Length) bytes"
>> $outputExists = Test-Path .\E-T8_tag_recovered.txt
>> $tempExists = Test-Path .\E-T8_tag_recovered.txt.part
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T8_bad_tag.l2pk `
>>     --out .\E-T8_tag_recovered.txt
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path .\E-T8_tag_recovered.txt
>> $tempExists = Test-Path .\E-T8_tag_recovered.txt.part
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = ($code -eq 4) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T8.4 PASS: $passed"
=== E-T8.4 GCM TAG TAMPERING ===
Offset: 506
Original byte: 8E
Modified byte: 8F
Package size: 507 bytes
Error: decryption failed
Decrypt exit code: 4
Output exists: False
Temporary exists: False
E-T8.4 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 117. Kiểm thử tính toàn vẹn của package L2PK chế độ RSASVE-HKDF-AES-256-GCM khi lần lượt thay đổi một bit trong các trường key_id, nonce, ciphertext và authentication tag. Chương trình từ chối các package không hợp lệ và không tạo tệp bản rõ hoặc để lại tệp tạm.

E-T9

```
PS D:\Matmahoc\Lab3\TaskE> Write-Host "=== E-T9.1 WRONG PRIVATE KEY ==="
>> $wrongKey = "..\TaskD\D-T9_wrong_private.pem"
>> $output = ".\E-T9_wrongkey_recovered.txt"
>> if (-not (Test-Path $wrongKey)) {
>>     throw "Không tìm thấy khóa RSA kiểm thử của Task D"
>> }
>> Write-Host "Wrong private key exists: $(Test-Path $wrongKey)"
>> $outputExists = Test-Path $output
>> $tempExists = Test-Path "$output.part"
>> Write-Host "Before - Output exists: $outputExists"
>> Write-Host "Before - Temporary exists: $tempExists"
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv $wrongKey `
>>     --in .\E-T5_message.l2pk `
>>     --out $output
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path $output
>> $tempExists = Test-Path "$output.part"
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = ($code -eq 4) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T9.1 PASS: $passed"
=== E-T9.1 WRONG PRIVATE KEY ===
Wrong private key exists: True
Before - Output exists: False
Before - Temporary exists: False
Error: decryption failed
Decrypt exit code: 4
Output exists: False
Temporary exists: False
E-T9.1 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 118. Kiểm thử giải mã gói KEM bằng khóa RSA Private không tương ứng. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp đầu ra.

```
PS D:\Matmahoc\Lab3\TaskE> Write-Host "=== E-T9.2 WRONG PACKAGE MODE ==="
>> $package = "..\TaskD\D-T3_first.l2pk"
>> $output = ".\E-T9_wrongmode_recovered.txt"
>> if (-not (Test-Path $package)) {
>>     throw "Không tìm thấy package Task D"
>> }
>> $data = [IO.File]::ReadAllBytes(
>>     (Resolve-Path $package).Path
>> )
>> Write-Host ("Package mode: 0x{0:X2}" -f $data[5])
>> Write-Host "Package size: $($data.Length) bytes"
>> if ($data[5] -ne 0x01) {
>>     throw "Package không có mode 0x01"
>> }
>> $outputExists = Test-Path $output
>> $tempExists = Test-Path "$output.part"
>> Write-Host "Before - Output exists: $outputExists"
>> Write-Host "Before - Temporary exists: $tempExists"
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in $package `
>>     --out $output
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path $output
>> $tempExists = Test-Path "$output.part"
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = ($code -eq 3) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T9.2 PASS: $passed"
=== E-T9.2 WRONG PACKAGE MODE ===
Package mode: 0x01
Package size: 500 bytes
Before - Output exists: False
Before - Temporary exists: False
Error: unexpected package mode (expected KEM 0x02)
Decrypt exit code: 3
Output exists: False
Temporary exists: False
E-T9.2 PASS: True
PS D:\Matmahoc\Lab3\TaskE>
```

Hình 119. Kiểm thử sử dụng package L2PK chế độ RSA-OAEP (0x01) cho chức năng KEM-decrypt. Chương trình phát hiện chế độ không phù hợp và trả mã thoát 3.

```
PS D:\Matmahoc\Lab3\TaskE> Write-Host "=== E-T9.3 UNSUPPORTED VERSION ==="
>> $original = [IO.File]::ReadAllBytes(
>>     (Resolve-Path .\E-T5_message.l2pk).Path
>> )
>> $modified = [byte[]]$original.Clone()
>> $offset = 4
>> $modified[$offset] = 0x02
>> [IO.File]::WriteAllBytes(
>>     (Join-Path (Get-Location).Path "E-T9_bad_version.l2pk"),
>>     $modified
>> )
>> Write-Host ("Offset: {0}" -f $offset)
>> Write-Host ("Original version: 0x{0:X2}" -f $original[$offset])
>> Write-Host ("Modified version: 0x{0:X2}" -f $modified[$offset])
>> Write-Host "Package size: $($modified.Length) bytes"
>> $output = ".\E-T9_version_recovered.txt"
>> $outputExists = Test-Path $output
>> $tempExists = Test-Path "$output.part"
>> Write-Host "Before - Output exists: $outputExists"
>> Write-Host "Before - Temporary exists: $tempExists"
>> if ($outputExists -or $tempExists) {
>>     throw "File đầu ra đã tồn tại"
>> }
>> .\task_e_kem.exe kem-decrypt `
>>     --priv ..\TaskB\private.pem `
>>     --in .\E-T9_bad_version.l2pk `
>>     --out $output
>> $code = $LASTEXITCODE
>> $outputExists = Test-Path $output
>> $tempExists = Test-Path "$output.part"
>> Write-Host "Decrypt exit code: $code"
>> Write-Host "Output exists: $outputExists"
>> Write-Host "Temporary exists: $tempExists"
>> $passed = ($code -eq 3) -and (-not $outputExists) -and (-not $tempExists)
>> Write-Host "E-T9.3 PASS: $passed"
=== E-T9.3 UNSUPPORTED VERSION ===
Offset: 4
Original version: 0x01
Modified version: 0x02
Package size: 507 bytes
Before - Output exists: False
Before - Temporary exists: False
Error: unsupported package version
Decrypt exit code: 3
Output exists: False
Temporary exists: False
```

Hình 120. Kiểm thử thay đổi trường version từ 0x01 thành 0x02. Chương trình từ chối phiên bản không được hỗ trợ, trả mã thoát 3 và không để lại tệp bản rõ hoặc tệp tạm

Tích hợp giao diện dòng lệnh CLI

1. Mã nguồn

Mã nguồn: src/main.cpp (phiên bản tích hợp CLI trong thư mục nộp bài).

```
#include "task_cli.hpp"
// Giao diện dòng lệnh chung cho các phần thực hành.
#include <openssl/crypto.h>
#include <iostream>
#include <string>
#include <vector>


namespace task_a { int entry(int argc, char** argv); }
namespace task_b { int entry(int argc, char** argv); }
namespace task_c { int entry(int argc, char** argv); }
namespace task_d { int entry(int argc, char** argv); }
namespace task_e { int entry(int argc, char** argv); }


namespace {
void usage() {
    std::cout
        << "Lab03 - OpenSSL RSA/KEM\n"
        << "Cac lenh:\n"
        << "  mod-add <a> <b> <m> | mod-sub <a> <b> <m>\n"
        << "  mod-mul <a> <b> <m> | mod-exp <a> <b> <m>\n"
        << "  gcd <a> <b> | mod-inverse <a> <m>\n"
        << "  rsa-params --bits <128..1024> [--show-private]\n"
        << "  keygen --bits <2048|3072|4096> --out-dir <dir>\n"
        << "  key-convert --type public|private --in <key> --out <key> --outform PEM|DER\n"
        << "  key-info --pub <key> | key-check --priv <key> --pub <key>\n"
        << "  os2ip --hex <hex> | i2osp --int <n> --len <L>\n"
        << "  raw-rsa --bits <512..1024> | oaep-demo --pub <key> --priv <key>\n"
        << "  encrypt --pub <key> --in <file> --out <l2pk> [--force]\n"
        << "  decrypt --priv <key> --in <l2pk> --out <file> [--force]\n"
        << "  kem-selftest --pub <key> --priv <key>\n"
        << "  kem-encrypt --pub <key> --in <file> --out <l2pk> [--force]\n"
        << "  kem-decrypt --priv <key> --in <l2pk> --out <file> [--force]\n"
        << "  task-c-all | task-d-all\n"
        << "  lab03 help | lab03 version | lab03 <lenh> --help\n";
}

bool matches(const std::string& command, std::initializer_list<const char*> options) {
    for (const char* option : options) if (command == option) return true;
    return false;
}
}


int main(int argc, char** argv) {
    if (argc < 2) { usage(); return 1; }
    const std::string cmd(argv[1]);
    if (cmd == "help" || cmd == "--help") { usage(); return 0; }
    if (cmd == "version") { std::cout << "Linked OpenSSL: " << OpenSSL_version(OPENSSL_VERSION) << '\n';
return 0; }
    if (argc == 3 && std::string(argv[2]) == "--help") { usage(); return 0; }
    if (matches(cmd,{"mod-add","mod-sub","mod-mul","mod-exp","gcd","mod-inverse"}))
        return task_a::entry(argc,argv);
    if (matches(cmd,{"rsa-params","keygen","key-convert","key-info","key-check"}))
        return task_b::entry(argc,argv);
    if (matches(cmd,{"os2ip","i2osp","raw-rsa","oaep-demo"}))
        return task_c::entry(argc,argv);
    if (matches(cmd,{"encrypt","decrypt"})) return task_d::entry(argc,argv);
    if (matches(cmd,{"kem-selftest","kem-encrypt","kem-decrypt"})) return task_e::entry(argc,argv);
    if (cmd == "task-c-all") {
        char arg[] = "--all-full";
        char* local[] = {argv[0],arg,nullptr};
        return task_c::entry(2,local);
    }
    if (cmd == "task-d-all") {
        char arg[] = "--all";
        char* local[] = {argv[0],arg,nullptr};
        return task_d::entry(2,local);
    }
    std::cerr << "Khong tim thay lenh: " << cmd << '\n';
    usage();
    return 1;
}
```

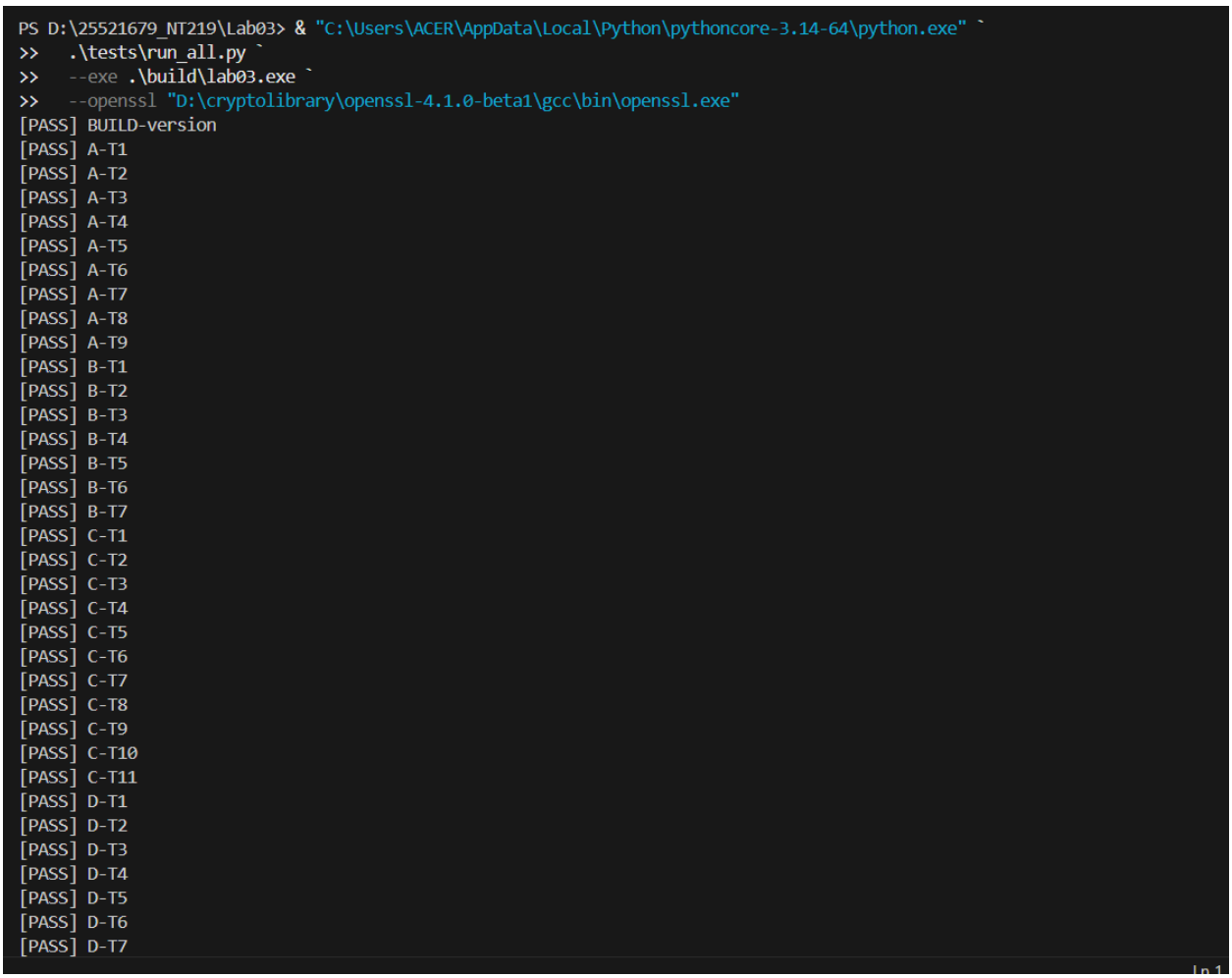
2. Hình ảnh thực hiện và kết quả kiểm thử

```
PS D:\25521679_NT219\Lab03> $env:Path = "D:\cryptolibrary\openssl-4.1.0-beta1\gcc\bin;C:\msys64\mingw64\bin;C:\msys64\ucrt64\bin;$env:Path"
PS D:\25521679_NT219\Lab03> cmake -S . -B build -G "MinGW Makefiles" `
>>   -DCMAKE_CXX_COMPILER=C:/msys64/mingw64/bin/g++.exe `
>>   -DCMAKE_MAKE_PROGRAM=C:/msys64/ucrt64/bin/mingw32-make.exe `
>>   -DCMAKE_BUILD_TYPE=Release `
>>   -DOPENSSL_ROOT_DIR=D:/cryptolibrary/openssl-4.1.0-beta1/gcc `
>>   -DOPENSSL_USE_STATIC_LIBS=TRUE `
>>   -DPython3_EXECUTABLE=C:/Users/ACER/AppData/Local/Python/pythoncore-3.14-64/python.exe
-- The CXX compiler identification is GNU 15.2.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/msys64/mingw64/bin/g++.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found OpenSSL: D:/cryptolibrary/openssl-4.1.0-beta1/gcc/lib/libcrypto.a (found suitable version "4.1.0", minimum required is "3.0") found compone
nts: Crypto
-- Found Python3: C:/Users/ACER/AppData/Local/Python/pythoncore-3.14-64/python.exe (found suitable version "3.14.0", minimum required is "3.9") foun
d components: Interpreter
-- Configuring done (2.1s)
-- Generating done (0.1s)
-- Build files have been written to: D:/25521679_NT219/Lab03/build
PS D:\25521679_NT219\Lab03> cmake --build build --parallel 4
[ 10%] Building CXX object CMakeFiles/labcore.dir/src/task_a.cpp.obj
[ 20%] Building CXX object CMakeFiles/labcore.dir/src/task_b.cpp.obj
[ 30%] Building CXX object CMakeFiles/labcore.dir/src/task_c.cpp.obj
[ 40%] Building CXX object CMakeFiles/labcore.dir/src/task_d.cpp.obj
[ 50%] Building CXX object CMakeFiles/labcore.dir/src/task_e.cpp.obj
[ 60%] Linking CXX static library liblabcore.a
[ 60%] Built target labcore
[ 70%] Building CXX object CMakeFiles/lab03.dir/src/main.cpp.obj
[ 80%] Building CXX object CMakeFiles/api_checks.dir/tests/api_checks.cpp.obj
[ 90%] Linking CXX executable lab03.exe
[100%] Linking CXX executable api_checks.exe
[100%] Built target lab03
[100%] Built target api_checks
PS D:\25521679_NT219\Lab03>
```

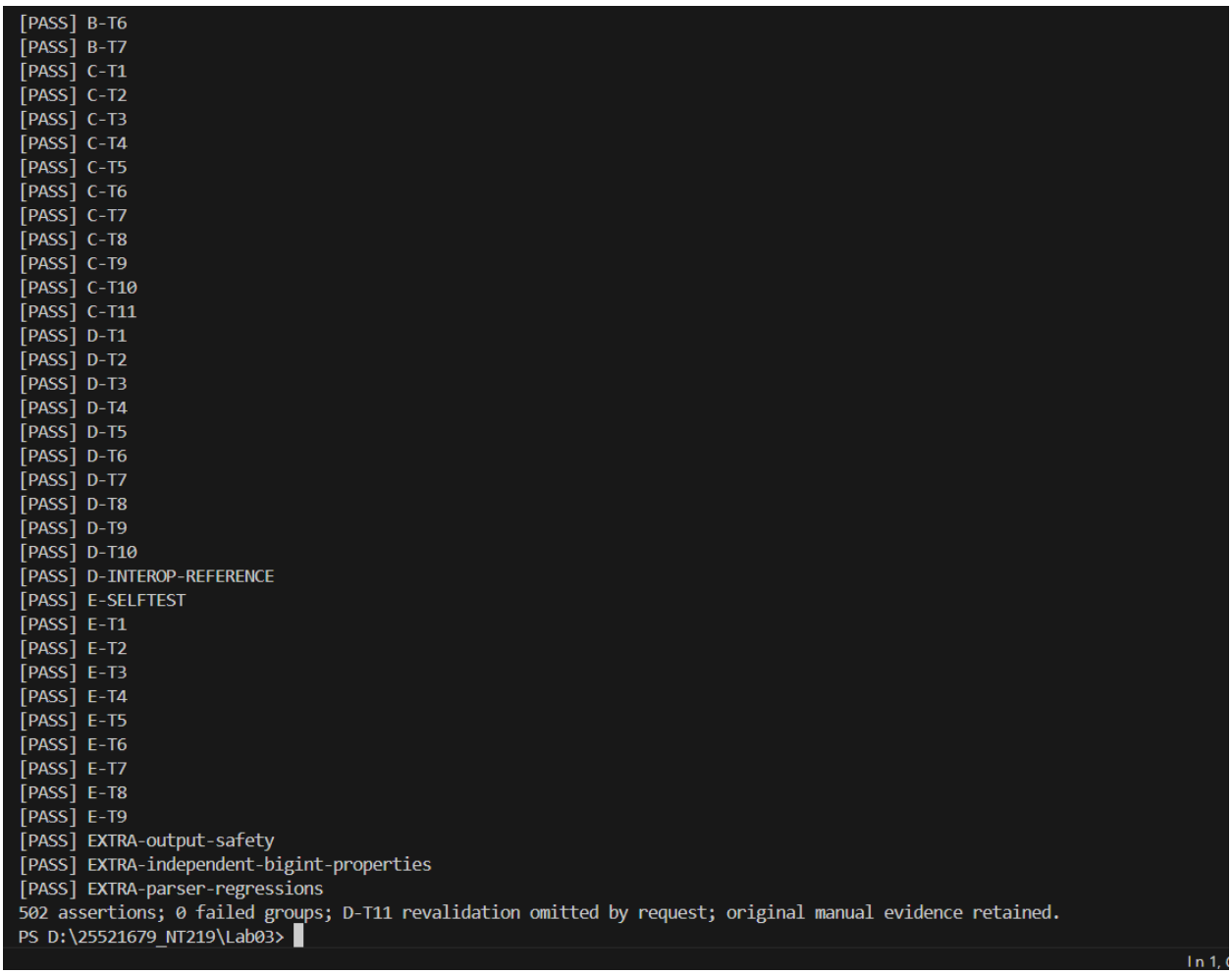
Hình 121. Biên dịch thành công chương trình CLI Lab03 tích hợp Task A–E bằng CMake.

```
PS D:\25521679_NT219\Lab03> .\build\lab03.exe version
Linked OpenSSL: OpenSSL 4.1.0-beta1 23 Sep 2026
PS D:\25521679_NT219\Lab03> .\build\lab03.exe help
Lab03 - OpenSSL RSA/KEM
Cac lenh:
  mod-add <a> <b> <m> | mod-sub <a> <b> <m>
  mod-mul <a> <b> <m> | mod-exp <a> <b> <m>
  gcd <a> <b> | mod-inverse <a> <m>
  rsa-params --bits <128..1024> [--show-private]
  keygen --bits <2048|3072|4096> --out-dir <dir>
  key-convert --type public|private --in <key> --out <key> --outform PEM|DER
  key-info --pub <key> | key-check --priv <key> --pub <key>
  os2ip --hex <hex> | i2osp --int <n> --len <L>
  raw-rsa --bits <512..1024> | oaep-demo --pub <key> --priv <key>
  encrypt --pub <key> --in <file> --out <l2pk> [--force]
  decrypt --priv <key> --in <l2pk> --out <file> [--force]
  kem-selftest --pub <key> --priv <key>
  kem-encrypt --pub <key> --in <file> --out <l2pk> [--force]
  kem-decrypt --priv <key> --in <l2pk> --out <file> [--force]
  task-c-all | task-d-all
  lab03 help | lab03 version | lab03 <lenh> --help
PS D:\25521679_NT219\Lab03>
```

Hình 122. Xác minh phiên bản OpenSSL liên kết và danh sách lệnh của chương trình CLI Lab03.



Hình 123. Kết quả kiểm thử tự động các nhóm Task A, Task B, Task C và Task D của chương trình CLI.



Hình 124. Kết quả tổng hợp bộ kiểm thử CLI: 502 phép kiểm tra và không có nhóm thất bại.

Phụ lục ảnh

Danh mục dưới đây dẫn đến vị trí hình trong từng phần thực hành. Các số trang được cập nhật theo thứ tự mới: mã nguồn trước, hình ảnh thực hiện sau.

Hình 1. Tạo cấu trúc thư mục cho Lab 03. 4
Hình 2. Cấu hình biên dịch OpenSSL trong Visual Studio Code. 4
Hình 3. Biên dịch chương trình kiểm tra môi trường bằng GCC MinGW64, sử dụng header và thư viện tĩnh OpenSSL 4.1.0-beta1 trên Windows. 4
Hình 4. Xác minh phiên bản OpenSSL sử dụng khi biên dịch và thực thi chương trình, với kết quả thành công và mã thoát bằng 0. 4
Hình 5. Biên dịch chương trình Task A bằng GCC MinGW64 và thư viện tĩnh OpenSSL 4.1.0-beta1. 10
Hình 6. Kiểm thử phép lũy thừa modulo với OpenSSL BIGNUM và xác nhận chương trình kết thúc thành công. 10
Hình 7. Kiểm thử phép cộng và phép trừ modulo. 11
Hình 8. Kiểm thử phép nhân modulo và ước chung lớn nhất. 12
Hình 9. Kiểm thử lũy thừa modulo bằng thuật toán bình phương và nhân. 12
Hình 10. Kiểm thử nghịch đảo modulo và trường hợp không tồn tại nghịch đảo. 13
Hình 11. Sinh số nguyên tố xác suất có độ dài 3072 bit bằng OpenSSL BIGNUM để minh họa bước chuẩn bị tham số RSA. .. 13
Hình 12. Kiểm thử các phép toán BIGNUM theo bộ A-T1 đến A-T5 và xác minh mã thoát của chương trình. 14
Hình 13. Kiểm thử các phép toán trên số nguyên lớn vượt phạm vi 64 bit bằng OpenSSL BIGNUM, với các kết quả A-T6 đến A-T8 và mã thoát thành công. 14
Hình 14. Kiểm thử khả năng phát hiện dữ liệu không hợp lệ và trả về mã lỗi theo yêu cầu của Task A. 15
Hình 15. Kiểm thử dữ liệu hệ thập lục phân, số âm và giới hạn độ dài đầu vào. 15
Hình 16. Biên dịch chương trình Task B bằng GCC MinGW64 và thư viện tĩnh OpenSSL 4.1.0-beta1. 24
Hình 17. Giao diện chương trình Task B. 25
Hình 18. Kiểm thử quá trình xây dựng tham số RSA và đối chiếu kết quả giải mã bằng số mũ bí mật và CRT. 25
Hình 19. Sinh cặp khóa RSA 3072 bit bằng OpenSSL và lưu dưới định dạng PEM, DER. 25
Hình 20. Cấu trúc thư mục Task B sau khi tạo bốn file khóa RSA bằng OpenSSL. 26
Hình 21. Kiểm tra sự tương ứng của cặp khóa RSA thông qua bốn tổ hợp khóa riêng và khóa công khai ở định dạng PEM và DER. 26
Hình 22. Hiển thị thông tin khóa công khai RSA 3072 bit, bao gồm modulus n, số mũ e, Key ID và biểu diễn khóa theo định dạng PEM. 27
Hình 23. Đối chiếu thông tin khóa công khai RSA và giá trị Key ID giữa hai định dạng PEM và DER. 27
Hình 24. Kiểm thử chuyển đổi khóa RSA giữa định dạng PEM và DER, xác nhận nội dung khóa không thay đổi sau quá trình chuyển đổi hai chiều. 27
Hình 25. Kiểm thử phát hiện hai khóa RSA không thuộc cùng một cặp, chương trình trả về MISMATCH và mã lỗi 4. 28
Hình 26. Kiểm thử khả năng phát hiện và từ chối file khóa công khai DER bị cắt mất dữ liệu, với mã lỗi 3. 28
Hình 27. Kiểm thử khả năng từ chối khóa công khai RSA ở cấu trúc PKCS#1 thay vì SPKI, với mã lỗi 3. 28
Hình 28. Kiểm thử khả năng từ chối khóa RSA 1024 bit không đáp ứng yêu cầu độ dài khóa tối thiểu 2048 bit. 28
Hình 29. Kiểm tra tính hợp lệ của khóa riêng RSA 3072 bit ở hai định dạng PEM và DER bằng OpenSSL CLI. 29
Hình 30. Kiểm tra danh sách quyền truy cập NTFS đối với các file khóa riêng RSA trên Windows 29
Hình 31. Cấu hình và xác minh quyền truy cập NTFS đối với hai file khóa riêng RSA trên Windows. 30
Hình 32. Xác minh giá trị Key ID bằng thuật toán SHA-256 trên khóa công khai SPKI DER thông qua OpenSSL CLI. 30
Hình 33. Kiểm thử cơ chế ngăn ghi đè các file khóa RSA đã tồn tại và xác minh khóa riêng không bị thay đổi. 30
Hình 34. Quan sát dữ liệu nhị phân và phân tích các byte mở đầu của khóa công khai RSA dưới định dạng DER. 31
Hình 35. Phân tích cấu trúc ASN.1 của khóa công khai RSA dưới định dạng DER bằng OpenSSL 4.1.0-beta1. 31
Hình 36. Thực nghiệm thay đổi một bit trong dữ liệu modulus của khóa công khai RSA và kiểm tra sự không tương ứng với khóa riêng ban đầu. 31
Hình 37. Thực nghiệm thay đổi một bit trong bản sao khóa công khai RSA dưới định dạng DER và kiểm tra tác động đến tính hợp lệ và sự tương ứng của cặp khóa. 32
Hình 38. Biên dịch chương trình Task C bằng GCC MinGW64, sử dụng thư viện OpenSSL 4.1.0-beta1 trên Windows. 45
Hình 39. Biên dịch mã nguồn Task C bằng GCC MinGW64 với chuẩn C++17 và thư viện OpenSSL 4.1.0-beta1 trên Windows. 45
Hình 40. Kết quả thực hiện ba phần của Task C gồm chuyển đổi OS2IP/I2OSP, kiểm tra các tính chất của Textbook RSA và thử nghiệm RSA-OAEP với khóa 2048 bit. Tất cả 16 kiểm thử đều thành công. 46
Hình 41. Kết quả kiểm thử chuyển đổi OS2IP/I2OSP, các tính chất của Textbook RSA và mã hóa RSA-OAEP với khóa 2048 bit. 47

Hình 42. Kết quả kiểm thử C-T7, RSA học thuật 512 bit và RSA-OAEP 3072 bit; toàn bộ 26 kiểm tra đều thành công. 47
Hình 43. Kiểm thử chức năng OS2IP qua giao diện dòng lệnh với hai dữ liệu hợp lệ và một dữ liệu không hợp lệ. Chương trình chuyển đổi chính xác các giá trị và trả mã lỗi 1 khi đầu vào không đúng định dạng. 48
Hình 44. Kết quả kiểm thử chức năng I2OSP với các giá trị và độ dài biểu diễn khác nhau. Chương trình tạo chuỗi byte đúng kích thước và trả mã lỗi 1 khi giá trị vượt quá phạm vi biểu diễn. 48
Hình 45. Kiểm thử chức năng RSA học thuật với khóa 512 bit, bao gồm mã hóa và giải mã, kiểm tra phạm vi thông điệp, tính xác định và khả năng biến đổi bản mã. Cả bốn kiểm tra đều thành công. 48
Hình 46. Kiểm thử RSA-OAEP với cặp khóa RSA 3072 bit định dạng PEM được tạo ở Task B. Kết quả cho thấy cả năm phép kiểm thử đều thành công, bao gồm tính ngẫu nhiên, giải mã, giới hạn độ dài thông điệp và phát hiện bản mã bị thay đổi. 49
Hình 47. Kiểm thử chức năng RSA-OAEP với cặp khóa RSA 3072 bit định dạng DER được tạo ở Task B. Chương trình hoàn thành cả năm phép kiểm thử, bao gồm tính ngẫu nhiên, khả năng giải mã, giới hạn độ dài thông điệp và phát hiện thay đổi bản mã. 49
Hình 48. Kiểm thử chức năng RSA-OAEP với cặp khóa không tương ứng. Chương trình phát hiện sự không khớp giữa khóa công khai và khóa bí mật, từ chối thực hiện thao tác và trả về mã lỗi 4. 49
Hình 49. Kiểm thử khả năng xử lý khóa DER không hợp lệ bằng cách tạo bản sao chỉ chứa 32 byte đầu của khóa công khai. Chương trình từ chối dữ liệu bị cắt ngắn và trả về mã lỗi 3. 50
Hình 50. Phân tích bốn byte đầu của khóa công khai RSA định dạng DER và kiểm chứng phép chuyển đổi OS2IP/I2OSP. Chuỗi 30 82 01 A2 được chuyển thành số nguyên 813826466 và khôi phục chính xác về biểu diễn ban đầu với độ dài 4 byte. 50
Hình 51. Quan sát kích thước và dữ liệu nhị phân của bản mã RSA-OAEP sử dụng khóa RSA 3072 bit. Bản rõ có độ dài 14 byte, trong khi bản mã có độ dài 384 byte, tương ứng với kích thước modulus RSA. 51
Hình 52. Thực nghiệm thay đổi một bit tại offset 0x50 của bản mã RSA-OAEP. Giá trị byte thay đổi từ D1 thành D0 thông qua phép XOR với 01. Giá trị băm SHA-256 của hai tệp khác nhau, xác nhận dữ liệu bản mã đã bị thay đổi. 51
Hình 53. Kiểm tra giải mã bản mã RSA-OAEP sau khi thay đổi một bit. OpenSSL 3.1.4 trả về thông báo “oaep decoding error” và mã thoát 1, cho thấy bản mã không vượt qua bước kiểm tra định dạng OAEP. 52
Hình 54. Đối chiếu dữ liệu nhị phân của bản mã RSA-OAEP trước và sau khi thay đổi một bit. Tại offset 0x50, giá trị byte chuyển từ D1 thành D0, trong khi các byte còn lại trên cùng dòng không thay đổi. 52
Hình 55. Thực hiện mã hóa lại bản rõ bằng RSA-OAEP với SHA-256 và MGF1-SHA256 trên OpenSSL 4.1.0-beta1. Quá trình mã hóa hoàn tất với mã thoát 0. 53
Hình 56. So sánh hai bản mã RSA-OAEP được tạo từ cùng bản rõ và cùng khóa công khai. Cả hai bản mã đều dài 384 byte nhưng có giá trị băm SHA-256 khác nhau, cho thấy kết quả mã hóa không cố định giữa các lần thực hiện. 53
Hình 57. Giải mã bản mã thứ hai bằng OpenSSL 3.1.4 với khóa bí mật RSA tương ứng. Quá trình giải mã thành công với mã thoát 0 và kết quả đối chiếu bản rõ trả về True. 54
Hình 58. Kiểm thử cơ chế mã hóa lai RSA-OAEP-SHA256 kết hợp AES-256-GCM. Chương trình hoàn thành 12/12 phép thử, bao gồm kiểm tra tính toàn vẹn, tính ngẫu nhiên và khả năng từ chối dữ liệu không hợp lệ. 64
Hình 59. Kiểm thử cơ chế mã hóa lai RSA-OAEP và AES-256-GCM bằng cặp khóa RSA được tạo từ Task B. Chương trình hoàn thành 12/12 phép kiểm thử, khôi phục chính xác bản rõ và trả về mã thoát 0. 65
Hình 60. Quan sát kích thước và cấu trúc phần đầu của gói dữ liệu demo.l2pk. Mười byte đầu thể hiện mã nhận dạng L2PK, phiên bản 1, chế độ RSA-OAEP, trường reserved bằng 0 và độ dài wrapped key là 384 byte. 66
Hình 61. Phân tích dữ liệu nhị phân của gói RSA-OAEP kết hợp AES-256-GCM tại các offset cuối. Kết quả xác định nonce dài 12 byte, trường ct_len biểu diễn giá trị 14, ciphertext dài 14 byte và authentication tag dài 16 byte, phù hợp với định dạng L2PK phiên bản 1. 66
Hình 62. Trích xuất và kiểm chứng các trường nonce, ciphertext length, ciphertext và authentication tag trong gói dữ liệu L2PK. Các trường được đọc theo offset và độ dài quy định, với tổng kích thước gói là 444 byte. 67
Hình 63. Kiểm thử mã hóa và giải mã tệp rỗng bằng RSA-OAEP kết hợp AES-256-GCM. Gói dữ liệu có kích thước 430 byte, quá trình mã hóa và giải mã đều trả về mã thoát 0. Tệp khôi phục có kích thước 0 byte và giá trị băm SHA-256 trùng với tệp ban đầu. 68
Hình 64. Kiểm thử mã hóa và giải mã tệp văn bản UTF-8 có dấu tiếng Việt bằng RSA-OAEP kết hợp AES-256-GCM. Hai thao tác đều trả về mã thoát 0; nội dung khôi phục chính xác và giá trị SHA-256 trùng khớp với tệp gốc. 69
Hình 65. Kiểm thử mã hóa và giải mã tệp PDF có kích thước 1.138.881 byte bằng RSA-OAEP kết hợp AES-256-GCM. Hai thao tác đều thành công với mã thoát 0; tệp khôi phục có kích thước và giá trị SHA-256 trùng khớp với tệp gốc. 70
Hình 66. Tạo tệp ZIP dùng cho kiểm thử mã hóa và giải mã. 70
Hình 67. Kiểm thử mã hóa và giải mã tệp ZIP bằng RSA-OAEP kết hợp AES-256-GCM. Gói dữ liệu có kích thước lớn hơn tệp gốc 430 byte; hai thao tác đều trả về mã thoát 0 và giá trị SHA-256 của tệp khôi phục trùng khớp với tệp ban đầu. 71
Hình 68. Xác định tệp ảnh PNG và kích thước dữ liệu đầu vào. 71
Hình 69. Kiểm thử mã hóa và giải mã tệp ảnh PNG bằng RSA-OAEP kết hợp AES-256-GCM. Hai thao tác thành công với mã thoát 0; ảnh khôi phục có kích thước và giá trị SHA-256 trùng khớp hoàn toàn với ảnh gốc. 72
Hình 70. Đối chiếu trực quan ảnh PNG gốc và ảnh được khôi phục sau khi giải mã bằng RSA-OAEP kết hợp AES-256-GCM. Hai ảnh hiển thị cùng nội dung; kết quả so sánh SHA-256 xác nhận dữ liệu được bảo toàn chính xác từng byte. 73
Hình 71. Tạo tệp nhị phân ngẫu nhiên dung lượng 50 MiB. 73

Hình 72. Kiểm thử mã hóa và giải mã tệp nhị phân ngẫu nhiên dung lượng 50 MiB bằng RSA-OAEP kết hợp AES-256-GCM. Chương trình trả về mã thoát 0 ở cả hai thao tác; tệp khôi phục có kích thước 52.428.800 byte và giá trị SHA-256 trùng khớp với tệp gốc. 74
Hình 73. Kiểm thử mã hóa và giải mã bằng cặp khóa RSA định dạng DER. Chương trình hoàn thành hai thao tác với mã thoát 0, tạo package 500 byte và khôi phục dữ liệu có giá trị SHA-256 trùng khớp với bản rõ ban đầu. 75
Hình 74. Kiểm thử mã hóa bằng khóa công khai RSA định dạng PEM và giải mã bằng khóa bí mật định dạng DER. Chương trình trả về mã thoát 0 cho cả hai thao tác, khôi phục bản rõ có giá trị SHA-256 trùng khớp với dữ liệu gốc. 76
Hình 75. Kiểm thử mã hóa bằng khóa công khai RSA định dạng DER và giải mã bằng khóa bí mật định dạng PEM. Chương trình thực hiện thành công hai thao tác với mã thoát 0, đồng thời khôi phục dữ liệu có giá trị SHA-256 trùng khớp với bản rõ. 77
Hình 76. Mã hóa cùng một tệp hai lần và đối chiếu kích thước, SHA-256 của hai gói dữ liệu. 78
Hình 77. Kiểm chứng tính ngẫu nhiên của cơ chế mã hóa lai RSA-OAEP kết hợp AES-256-GCM. Hai lần mã hóa cùng một tệp bằng cùng khóa công khai tạo ra các package dài 500 byte nhưng có giá trị SHA-256 và nonce khác nhau. 78
Hình 78. Thay đổi một bit trong ciphertext 79
Hình 79. Kiểm thử thay đổi một bit trong ciphertext của gói L2PK. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp đầu ra hoặc để lại tệp tạm, bảo đảm dữ liệu chưa được xác thực không bị công bố. 79
Hình 80. Thay đổi một bit trong authentication tag của gói L2PK. 80
Hình 81. Kiểm thử thay đổi một bit trong authentication tag của gói dữ liệu L2PK. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp bản rõ hoặc để lại tệp tạm, bảo đảm dữ liệu chưa được xác thực không bị công bố. 80
Hình 82. Thay đổi một bit trong nonce AES-GCM của gói L2PK. 81
Hình 83. Kiểm thử thay đổi một bit trong nonce AES-GCM của gói L2PK. Chương trình từ chối giải mã với mã thoát 4, đồng thời không tạo tệp bản rõ hoặc để lại tệp tạm. 81
Hình 84. Thay đổi một bit trong trường wrapped key RSA-OAEP. 82
Hình 85. Kiểm thử thay đổi một bit tại offset 15 trong trường wrapped key RSA-OAEP. Chương trình từ chối giải mã với mã thoát 4, không tạo tệp bản rõ hoặc để lại tệp tạm, đồng thời sử dụng thông báo lỗi thống nhất với trường hợp xác thực GCM thất bại. 82
Hình 86. Kiểm thử trường reserved không hợp lệ trong header L2PK. Chương trình phát hiện giá trị reserved khác 0, từ chối giải mã với mã thoát 3 và không tạo tệp đầu ra hoặc để lại tệp tạm. 83
Hình 87. Kiểm thử trường version không hợp lệ trong header L2PK. Chương trình phát hiện phiên bản 0x02 không được hỗ trợ, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm. 84
Hình 88. Kiểm thử mã nhận dạng Magic không hợp lệ trong header L2PK. Sau khi thay đổi Magic từ L2PK thành M2PK, chương trình từ chối giải mã với mã thoát 3 và không tạo tệp đầu ra hoặc để lại tệp tạm. 85
Hình 89. Kiểm thử gói L2PK bị cắt bỏ một byte cuối của authentication tag. Chương trình phát hiện kích thước không hợp lệ, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm. 86
Hình 90. Thêm byte 0xA5 vào cuối gói dữ liệu L2PK. 87
Hình 91. Kiểm thử gói L2PK có một byte dư ở cuối. Sau khi thêm byte 0xA5 khiến kích thước tăng từ 500 lên 501 byte, chương trình phát hiện lỗi không khớp tổng độ dài package, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm. 87
Hình 92. Đặt trường ct_len thành giá trị 2^63 trong gói L2PK. 88
Hình 93. Kiểm thử trường ct_len có giá trị 2^63 trong header L2PK. Chương trình phát hiện độ dài ciphertext không phù hợp với kích thước package thực tế, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm. 89
Hình 94. Tạo khóa bí mật RSA 3072 bit không tương ứng để kiểm thử giải mã. 89
Hình 95. Kiểm thử giải mã gói L2PK bằng khóa RSA Private không tương ứng với khóa Public mã hóa. Chương trình từ chối giải mã với mã thoát 4, không tạo tệp bản rõ hoặc để lại tệp tạm. 90
Hình 96. Kiểm thử giải mã package L2PK bằng khóa RSA-2048 trong khi trường klen khai báo 384 byte tương ứng RSA-3072. Chương trình phát hiện sự không khớp kích thước khóa, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm. 91
Hình 97. Hà thực hiện mã hóa tệp Ha_messages.txt bằng khóa công khai RSA của Thắng trên hệ điều hành Fedora. Chương trình tạo thành công gói tin from_Ha.l2pk từ dữ liệu bản rõ 17 byte. 91
Hình 98. Giải mã thành công gói dữ liệu do nhóm Hà tạo bằng chương trình Task D của nhóm nhận. Quá trình sử dụng khóa bí mật RSA tương ứng, trả về mã thoát 0 và khôi phục nội dung bản rõ. 92
Hình 99. Mã hóa thông điệp phản hồi bằng chương trình Task D và khóa công khai RSA của nhóm Hà. Chương trình tạo thành công gói from_Thang.l2pk theo định dạng L2PK phiên bản 1 và trả về mã thoát 0. 92
Hình 100. Hà thực hiện giải mã gói tin from_Thang.l2pk do Thắng tạo. Chương trình thông báo giải mã thành công, khôi phục 40 byte dữ liệu và hiển thị chính xác nội dung bản rõ ban đầu, chứng minh khả năng tương thích giữa hai chương trình. 92
Hình 101. Biên dịch chương trình Task E bằng GCC MinGW64 và thư viện OpenSSL. 104
Hình 102. Kiểm thử tổng hợp Task E với RSASVE, HKDF-SHA256 và AES-256-GCM; cả 9 nhóm kiểm thử đều thành công. 105
Hình 103. Mã hóa tệp văn bản bằng RSASVE-HKDF-AES-256-GCM và kiểm tra kích thước gói L2PK. 105
Hình 104. Kiểm thử giải mã gói L2PK chế độ RSASVE-HKDF-AES-256-GCM. Chương trình khôi phục thành công tệp bản rõ và xác nhận giá trị SHA-256 của tệp gốc và tệp sau giải mã trùng khớp. 106
Hình 105. Thay đổi trường klen từ 384 thành 385 byte để tạo gói kiểm thử không hợp lệ. 107
Hình 106. Kiểm thử trường klen không hợp lệ trong gói L2PK chế độ RSASVE. Sau khi thay đổi klen từ 384 thành 385 byte, chương trình từ chối giải mã với mã thoát 3, không tạo tệp bản rõ hoặc để lại tệp tạm. 108

Hình 107. Tạo gói kiểm thử có giá trị KEM ciphertext lớn hơn hoặc bằng modulus RSA. 109
Hình 108. Kiểm thử giá trị KEM ciphertext lớn hơn hoặc bằng modulus RSA. Chương trình phát hiện ciphertext không hợp lệ, từ chối giải mã với mã thoát 3 và không tạo tệp đầu ra hoặc để lại tệp tạm. 110
Hình 109. Đặt toàn bộ KEM ciphertext thành giá trị 0 trong gói L2PK. 111
Hình 110. Kiểm thử KEM ciphertext bằng 0 trong gói L2PK. Chương trình phát hiện giá trị ciphertext không thuộc miền hợp lệ của RSASVE, trả mã thoát 3 và không tạo tệp bản rõ hoặc để lại tệp tạm. 112
Hình 111. Thay đổi một bit tại byte cuối của KEM ciphertext. 113
Hình 112. Kiểm tra tệp bản rõ và tệp tạm chưa tồn tại trước khi giải mã gói có KEM ciphertext bị thay đổi. 113
Hình 113. Kiểm thử thay đổi một bit trong KEM ciphertext. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp bản rõ hoặc để lại tệp tạm. 114
Hình 114. Thay đổi một bit trong trường key_id và kiểm thử khả năng phát hiện dữ liệu bị sửa đổi. 115
Hình 115. Thay đổi một bit trong nonce và kiểm thử khả năng từ chối giải mã. 116
Hình 116. Thay đổi một bit trong ciphertext và kiểm thử khả năng từ chối giải mã. 117
Hình 117. Kiểm thử tính toàn vẹn của package L2PK chế độ RSASVE-HKDF-AES-256-GCM khi lần lượt thay đổi một bit trong các trường key_id, nonce, ciphertext và authentication tag. Chương trình từ chối các package không hợp lệ và không tạo tệp bản rõ hoặc để lại tệp tạm. 118
Hình 118. Kiểm thử giải mã gói KEM bằng khóa RSA Private không tương ứng. Chương trình từ chối giải mã với mã thoát 4 và không tạo tệp đầu ra. 119
Hình 119. Kiểm thử sử dụng package L2PK chế độ RSA-OAEP (0x01) cho chức năng KEM-decrypt. Chương trình phát hiện chế độ không phù hợp và trả mã thoát 3. 120
Hình 120. Kiểm thử thay đổi trường version từ 0x01 thành 0x02. Chương trình từ chối phiên bản không được hỗ trợ, trả mã thoát 3 và không để lại tệp bản rõ hoặc tệp tạm 121
Hình 121. Biên dịch thành công chương trình CLI Lab03 tích hợp Task A–E bằng CMake. 123
Hình 122. Xác minh phiên bản OpenSSL liên kết và danh sách lệnh của chương trình CLI Lab03. 124
Hình 123. Kết quả kiểm thử tự động các nhóm Task A, Task B, Task C và Task D của chương trình CLI. 125
Hình 124. Kết quả tổng hợp bộ kiểm thử CLI: 502 phép kiểm tra và không có nhóm thất bại. 126